

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH

FLIXZONE - NỀN TẢNG STREAMMING PHIM

NGÀNH: KHOA HỌC MÁY TÍNH

Tên hội đồng : Hội đồng 2L Đồ án chuyên ngành

GVHD(s) : Trần Trương Tuấn Phát, M.Eng.

—o0o—

SVTH1 : Lâm Vũ - 2213994

SVTH2 : Võ Quang Đại Việt - 2213954

SVTH3 : Nguyễn Hải Trung - 2213698

TP. HỒ CHÍ MINH, 12/2025

Mục lục

1	Giới Thiệu	7
1.1	Tóm Tắt	7
1.2	Danh Sách Thành Phần	7
2	Tổng Quan Hệ Thống	9
2.1	Bối Cảnh	9
2.2	Các Bên Liên Quan	9
2.3	Phân tích yêu cầu chức năng	10
2.3.1	Yêu cầu chức năng cho Guest	10
2.3.2	Register user	10
2.3.3	Premium user	11
2.3.4	Admin	11
2.3.5	Nhà quảng cáo	12
2.3.6	Payment Provider	12
2.4	Phân tích yêu cầu phi chức năng	13
3	Mô hình hóa hệ thống	15
3.1	Usecase tổng quan của cả hệ thống	15
3.2	Admin hệ thống	16
3.3	Người dùng khách - Guest User	20
3.4	Người dùng thường - Regular User	24
3.5	Người dùng trả phí - Premium User	33
3.6	Sơ đồ thiết kế hành vi	37
3.6.1	Tìm kiếm nội dung	37
3.6.2	Xem phim	39
3.6.3	Admin tạo tập phim mới	41
3.6.4	Người dùng viết đánh giá phim	42
3.6.5	Người dùng đăng ký tài khoản	44
3.6.6	Admin tải video mới lên hệ thống	46
3.6.7	Quản lý theo dõi phim	48
3.6.8	Thiết lập phim mới	50
3.6.9	Cài đặt thông báo	51
4	Kiến trúc phần mềm	53
4.1	Tổng Quan Kiến Trúc	53
4.2	Công Nghệ Sử Dụng	53
4.3	Các Thành Phần Chính	54
4.4	Góc Nhìn Phân Rã	55
4.4.1	Sơ Đồ Chính	55
4.4.2	Danh Mục Thành Phần	56
4.4.3	Hướng Dẫn Thay Đổi	59
4.4.4	Lý Do Thiết Kế	59
4.5	Góc Nhìn Thành Phần - Kết Nối	60
4.5.1	Sơ đồ chính	60
4.5.2	Danh Sách Thành Phần	60
4.5.3	Hướng Dẫn Thay Đổi	63
4.5.4	Lý Do Thiết Kế	63
4.6	Góc Nhìn Triển Khai	65

4.6.1	Sơ Đồ Chính	65
4.6.2	Danh Sách Thành Phần	65
4.6.3	Hướng Dẫn Thay Đổi	67
4.6.4	Lý Do Thiết Kế	67
5	Cơ sở dữ liệu	68
5.1	Dịch vụ xác thực và phân quyền (Auth Service)	68
5.1.1	Mô hình dữ liệu	68
5.1.2	Ảnh xạ mô hình dữ liệu	69
5.2	Dịch vụ quản lý nội dung (Content Service)	70
5.2.1	Mô hình dữ liệu	70
5.2.2	Ảnh xạ mô hình dữ liệu	72
5.3	Dịch vụ xử lý và truyền phát video (Streaming Service)	75
5.3.1	Mô hình dữ liệu	75
5.3.2	Ảnh xạ mô hình dữ liệu	77
5.4	Dịch vụ thông báo (Notification Service)	78
5.4.1	Mô hình dữ liệu	78
5.4.2	Ảnh xạ mô hình dữ liệu	80
5.5	Dịch vụ đánh giá và tương tác (Rating Service)	81
5.5.1	Mô hình dữ liệu	81
5.5.2	Ảnh xạ mô hình dữ liệu	83
5.6	Dịch vụ thanh toán (Payment Service)	84
5.6.1	Mô hình dữ liệu	84
5.6.2	Ảnh xạ mô hình dữ liệu	85
5.7	Dịch vụ phân tích dữ liệu (Analytics Service)	86
5.7.1	Mô hình dữ liệu	86
5.7.2	Ảnh xạ mô hình dữ liệu	87
5.8	Dịch vụ gợi ý (Recommendation Service)	88
5.8.1	Mô hình dữ liệu	89
5.8.2	Ảnh xạ mô hình dữ liệu	90
6	Hệ thống gợi ý phim cá nhân hóa	94
6.1	Mục tiêu	94
6.2	Dữ liệu đầu vào	94
6.3	Dữ liệu đầu ra	94
6.4	Phương pháp	95
6.4.1	Biểu diễn dữ liệu	95
6.4.2	Mô hình tương tác người dùng-phim	95
6.4.3	Dự đoán mức độ phù hợp	95
6.4.4	Huấn luyện mô hình	96
6.4.5	Triển khai trong hệ thống FlixZone	96
7	Kết Luận	98
7.1	Những Gì Đã Làm Được	98
7.1.1	Thiết Kế Kiến Trúc	98
7.1.2	Phát Triển Backend	98
7.1.3	Phát Triển Frontend	98
7.1.4	Tài Liệu Kỹ Thuật	98
7.1.5	Triển Khai	98
7.2	Nhận Xét	99



8	Hướng Phát Triển Tiếp Theo	100
9	Phụ lục	103
9.1	Hồ sơ quyết định kiến trúc	103

Danh sách hình vẽ

1	Usecase tổng thể của hệ thống	15
2	Sơ đồ hoạt động của use case Search Films	37
3	Sơ đồ hoạt động của usecase Xem phim	39
4	Sơ đồ hoạt động của Usecase Tạo tập phim mới	41
5	Sơ đồ hoạt động của người dùng viết đánh phim	42
6	Sơ đồ hoạt động của Usecase Người dùng đăng ký tài khoản	44
7	Sơ đồ hoạt động cho Usecase Admin tải video mới lên hệ thống và Xử lý Video	46
8	Sơ đồ hoạt động của Usecase Theo dõi phim	48
9	Sơ đồ hoạt động cho Usecase quản lý content của Admin	50
10	Sơ đồ hoạt động cho Usecase cài đặt thông báo	51
11	Sơ đồ tổng quan kiến trúc hệ thống FlixZone	53
12	Sơ đồ góc nhìn phân rã hệ thống FlixZone	55
13	Sơ đồ thành phần - kết nối	60
14	Sơ đồ góc nhìn triển khai hệ thống Streaming	65
15	Sơ đồ EERD cho Auth Service	68
16	Sơ đồ mapping EERD cho Auth Service	69
17	Sơ đồ EERD cho Content Service	71
18	Sơ đồ mapping EERD cho Content Service	72
19	Sơ đồ EERD cho Streaming Service	76
20	Sơ đồ mapping EERD cho Stream Service	77
21	Sơ đồ EERD cho Notification Service	79
22	Sơ đồ mapping EERD cho Notification Service	80
23	Sơ đồ EERD cho Rating Service	82
24	Sơ đồ mapping EERD cho Rating Service	83
25	Sơ đồ EERD cho Payment Service	85
26	Sơ đồ mapping EERD cho Payment Service	85
27	Sơ đồ EERD cho Analytics Service	87
28	Sơ đồ mapping cho Analytics Service	87
29	Sơ đồ EERD cho Recommendation Service	89
30	Sơ đồ mapping cho Recommendation Service	90
31	Sơ đồ Gantt cho hướng phát triển trong tương lai	100

Danh sách bảng

1	Use case: Setup Movie (Content Service + Streaming Service)	16
2	Use case: View Analytics (Admin)	17
3	Use case: Delete/Block/Ban User (Admin)	17
4	Use case: Create Episode (Admin)	18
5	Use case: Upload Video (Streaming Service + Cloudflare Stream)	19
6	Use case: Register (Guest User)	20
7	Use case: Search Films (Guest User)	21
8	Use case: View Film Description (Guest User)	22
9	Use case: Watch Trailer (Guest User - YouTube Source)	23
10	Use case: Sign In (User Service)	24
11	Use case: Watch Movie (Streaming Service)	25
12	Use case: Rate Content (Rating Service + Content Service)	26
13	Use case: Reset Password (User Service)	27
14	Use case: View Profile (User Service)	27
15	Use case: Update Profile (User Service)	28
16	Use case: Log Out (User Service)	28
17	Use case: Set up Notification (Notification Service)	29
18	Use case: Init Payment (Payment Service)	30
19	Use case: View History (Payment Service)	30
20	Use case: Get Recommendations (Regular User)	31
21	Use case: Be Tracked (Implicit Collection)	31
22	Use case: Subscribe to Series (Notification Service)	32
23	Use case: Watch movies without ads (Premium)	33
24	Use case: Watch trailers (Premium)	33
25	Use case: View film description (Premium)	34
26	Use case: View actor descriptions (Premium)	34
27	Use case: Track Usage (Premium)	35
28	Use case: Download (Premium)	35
29	Use case: Login (Premium)	36
30	Use case: Reset password (Premium)	36
31	Danh sách thực thể Auth Service	68
32	Danh sách quan hệ Auth Service	69
33	Chi tiết bảng users	69
34	Chi tiết bảng admins	70
35	Danh sách thực thể Content Service	71
36	Danh sách quan hệ Content Service	72
37	Chi tiết bảng contents	73
38	Chi tiết bảng single_movies	73
39	Chi tiết bảng series	73
40	Chi tiết bảng seasons	74
41	Chi tiết bảng episodes	74
42	Chi tiết bảng genres	74
43	Chi tiết bảng persons	74
44	Chi tiết bảng content_genres	75
45	Chi tiết bảng content_persons	75
46	Danh sách thực thể Streaming Service	76
47	Danh sách quan hệ Streaming Service	76

48	Chi tiết bảng stream_assets	77
49	Chi tiết bảng stream_renditions	78
50	Chi tiết bảng caption_tracks	78
51	Danh sách thực thể Notification Service	79
52	Danh sách quan hệ Notification Service	79
53	Chi tiết collection notifications	80
54	Chi tiết collection notification_preferences	80
55	Chi tiết collection series_subscriptions	81
56	Danh sách thực thể Rating Service	82
57	Danh sách quan hệ Rating Service	82
58	Chi tiết collection comments	83
59	Chi tiết collection ratings	83
60	Chi tiết collection ratings	84
61	Chi tiết collection likes	84
62	Danh sách thực thể Payment Service	85
63	Danh sách quan hệ Payment Service	85
64	Chi tiết bảng payments	86
65	Danh sách thực thể Analytics Service	87
66	Chi tiết bảng watch_history	88
67	Chi tiết bảng rating_history	88
68	Chi tiết bảng payment_history	88
69	Danh sách thực thể Recommendation Service	90
70	Chi tiết bảng watch_history	91
71	Chi tiết bảng rating_history	91
72	Chi tiết bảng like_history	91
73	Chi tiết bảng content_item	92
74	Chi tiết bảng user_embedding	92
75	Chi tiết bảng item_embedding	92
76	Chi tiết bảng model_config	92
77	Chi tiết bảng recommendation_list	93
78	Phân công nhiệm vụ phát triển hệ thống FlixZone	97

1 Giới Thiệu

1.1 Tóm Tắt

Tài liệu này trình bày kiến trúc phần mềm của hệ thống **FlixZone** – một nền tảng xem phim trực tuyến được xây dựng theo kiến trúc microservices, hỗ trợ phát video thích ứng (adaptive bitrate streaming), quản lý nội dung đa dạng, hệ thống đánh giá và gợi ý cá nhân hóa dựa trên trí tuệ nhân tạo.

Các góc nhìn kiến trúc khác nhau được sử dụng nhằm cung cấp cái nhìn toàn diện về hệ thống, bao gồm góc nhìn phân rã, thành phần-kết nối và triển khai. Ngoài ra, tài liệu còn trình bày chi tiết các thiết kế hành vi, luồng xử lý nghiệp vụ, cơ chế truyền thông bất đồng bộ thông qua message queue, cũng như việc ứng dụng trí tuệ nhân tạo trong hệ thống gợi ý nội dung và quản lý dữ liệu đa mô hình (polyglot persistence) trong hệ thống FlixZone.

1.2 Danh Sách Thành Phần

Báo cáo kiến trúc phần mềm của hệ thống được tổ chức thành các phần chính sau:

- Giới thiệu:** Giới thiệu cách tổ chức nội dung, phạm vi và mục tiêu của từng phần trong tài liệu kiến trúc.
- Tổng Quan Hệ Thống:** Trình bày bối cảnh hình thành hệ thống, các bên liên quan, yêu cầu chức năng và phi chức năng – làm cơ sở cho việc thiết kế kiến trúc hệ thống.
- Mô Hình Hóa Hệ Thống:** Mô tả các nghiệp vụ cốt lõi thông qua sơ đồ use case, bảng mô tả chi tiết và sơ đồ thiết kế hành vi.
 - Use Case tổng quan của cả hệ thống:** Sơ đồ tổng quan thể hiện các actor và use case chính.
 - Admin hệ thống:** Các use case của quản trị viên bao gồm thiết lập phim, tải video, quản lý nội dung.
 - Người dùng khách (Guest User):** Các use case cho phép truy cập mà không cần đăng nhập như duyệt phim, tìm kiếm.
 - Người dùng thường (Regular User):** Các use case sau khi đăng nhập như xem phim, đánh giá, theo dõi phim bộ.
 - Người dùng trả phí (Premium User):** Các use case đặc quyền như xem không quảng cáo, chất lượng cao, nội dung độc quyền.
 - Sơ đồ thiết kế hành vi:** Các activity diagrams mô tả luồng xử lý chi tiết.
 - Tìm kiếm nội dung (Content Discovery and Search)
 - Xem phim (Video Streaming Playback)
 - Admin tạo tập phim mới (Create Episode with Notification)
 - Người dùng viết đánh giá phim (User Rating and Review)
 - Người dùng đăng ký tài khoản (User Registration)
 - Admin tải video mới lên hệ thống (Video Upload and Processing)
 - Quản lý theo dõi phim (Series Subscription Management)
 - Thiết lập phim mới (Admin Content Management)
 - Cài đặt thông báo (Notification Preferences Management)
- Kiến Trúc Phần Mềm:** Trình bày kiến trúc hệ thống theo nhiều góc nhìn khác nhau.

- (a) **Tổng Quan Kiến Trúc:** Giới thiệu kiểu kiến trúc Event-Driven Microservices và các công nghệ chính được lựa chọn (NestJS, PostgreSQL, MongoDB, Redis, Kafka, Elasticsearch).
 - (b) **Góc Nhìn Phân Rã:** Phân tích hệ thống thành các microservices độc lập: Users Service, Content Service, Streaming Service, Rating Service, Notification Service, Payment Service, Recommendation Service.
 - (c) **Góc Nhìn Thành Phần – Kết Nối:** Trình bày các thành phần trong thời gian chạy, luồng giao tiếp qua API Gateway (Apache APISIX), hệ thống xác thực JWT, và message queue (Redis/Bull, Kafka).
 - (d) **Góc Nhìn Triển Khai:** Mô tả môi trường triển khai với Docker và Docker Compose, hạ tầng container, tích hợp Cloudflare Stream CDN và cách hệ thống vận hành trong thực tế.
5. **Cơ Sở Dữ Liệu:** Mô tả mô hình dữ liệu và ánh xạ cho từng dịch vụ.
- (a) Dịch vụ xác thực và phân quyền (Auth Service)
 - (b) Dịch vụ quản lý nội dung (Content Service)
 - (c) Dịch vụ xử lý và truyền phát video (Streaming Service)
 - (d) Dịch vụ thông báo (Notification Service)
 - (e) Dịch vụ đánh giá và tương tác (Rating Service)
 - (f) Dịch vụ thanh toán (Payment Service)
 - (g) Dịch vụ phân tích dữ liệu (Analytics Service)
 - (h) Dịch vụ đề xuất phim (Recommendation Service)
6. **Hệ Thống Gợi Ý Phim Cá Nhân Hóa:** Trình bày cách hệ thống ứng dụng học máy trong mô-đun gợi ý nội dung.
7. **Phụ Lục:**
- (a) **Hồ Sơ Quyết Định Kiến Trúc (ADR):** Phân tích các quyết định kiến trúc quan trọng và lý do lựa chọn, bao gồm:
 - ADR-01: Lựa Chọn Kiến Trúc Event-Driven Microservices.
 - ADR-02: Lựa Chọn Sử Dụng API Gateway (Apache APISIX).
 - ADR-03: Lựa Chọn Cloudflare Stream cho Video Streaming.
 - ADR-04: Lựa Chọn Elasticsearch cho Tìm Kiếm Nội Dung.
 - ADR-05: Lựa Chọn Message Queue Kết Hợp (Redis/Bull + Kafka).
 - ADR-06: Lựa Chọn Polyglot Persistence (PostgreSQL + MongoDB).
 - ADR-07: Lựa Chọn JWT cho Authentication.
 - ADR-08: Lựa Chọn Containerization với Docker.

2 Tổng Quan Hệ Thống

2.1 Bối Cảnh

Nhu cầu tiêu thụ nội dung số, đặc biệt là phim trực tuyến, tăng mạnh trong nhiều năm gần đây. Quy mô thị trường video streaming toàn cầu đã vượt 544 tỷ USD vào năm 2023^{1,2} và được dự đoán đạt trên 1 nghìn tỷ USD trước năm 2030, với tốc độ tăng trưởng kép hàng năm (CAGR) khoảng 21–25%. Lượng người dùng cũng tăng liên tục: tính riêng tại Việt Nam, nhóm dịch vụ OTT trả phí ghi nhận hơn 15 triệu người dùng năm 2023,^{3,4} tăng gần 35% so với năm trước.

Để đáp ứng quy mô và yêu cầu kỹ thuật ngày càng cao, các hệ thống xem phim trực tuyến lớn như Netflix thường lựa chọn kiến trúc microservices,^{5,8} nơi mỗi dịch vụ đảm nhiệm một phạm vi nghiệp vụ rõ ràng (user, movie, payment, recommendation, ads) và giao tiếp thông qua API Gateway⁹ hoặc message queue.⁶ Kiến trúc này cho phép mở rộng độc lập từng dịch vụ, triển khai linh hoạt, dễ theo dõi và duy trì tính sẵn sàng cao.⁷ Kết hợp với các công nghệ như HLS/DASH cho adaptive bitrate streaming,^{10,11} kho đối tượng (object storage) và CDN để phân phối nội dung,¹² hệ thống có thể phục vụ được lượng người dùng lớn mà vẫn đảm bảo trải nghiệm ổn định.

Từ những nhu cầu và ràng buộc kỹ thuật trên, nhóm lựa chọn xây dựng **FlixZone** như một mô hình hệ thống hoàn chỉnh để mô phỏng bài toán thực tế trong phạm vi học thuật. Thay vì chỉ dừng ở mức CRUD đơn giản, FlixZone tích hợp đầy đủ các nghiệp vụ thiết yếu của một nền tảng xem phim hiện đại: phân tách frontend-backend, API Gateway, microservices độc lập,⁵ quản lý tài khoản Free/Premium, quy trình xử lý giao dịch qua nhà cung cấp thanh toán nội địa (VNPay,¹⁶ MoMo¹⁷) với các yêu cầu về bảo mật và idempotency,^{18,19} cùng khả năng phân phối quảng cáo thông qua dịch vụ Ads.

Điểm nổi bật của FlixZone nằm ở việc ứng dụng AI trong mô-đun gợi ý nội dung,^{13,14} cho phép cá nhân hóa danh mục phim dựa trên hành vi và lịch sử xem của từng người dùng theo các phương pháp được trình bày trong.¹⁵ Việc kết hợp giữa kiến trúc microservices, nghiệp vụ kinh doanh thực tế và mô-đun AI giúp FlixZone vượt xa mô hình bài tập lập trình cơ bản, trở thành một hệ thống mẫu giúp người học rèn luyện toàn diện các kỹ năng quan trọng của kỹ sư phần mềm:^{20,21} phân tích yêu cầu, thiết kế kiến trúc theo Domain-Driven Design,²² xử lý giao dịch, quản trị vận hành và triển khai hệ thống tại quy mô gần với thực tế doanh nghiệp.

2.2 Các Bên Liên Quan

Hệ thống FlixZone phục vụ nhiều nhóm người dùng với đặc điểm và nhu cầu sử dụng khác nhau. Mỗi nhóm đóng vai trò cụ thể trong hoạt động tổng thể của nền tảng, đồng thời tạo nên bối cảnh cho các yêu cầu chức năng và phi chức năng của hệ thống.

- **Khách (Guest):** người dùng chưa đăng nhập. Khách có thể truy cập các trang thông tin phim, xem trailer và đọc mô tả, nhưng không thể xem nội dung đầy đủ. Đây là nhóm người dùng tiềm năng có khả năng chuyển đổi sang tài khoản Free hoặc Premium.
- **Người dùng miễn phí (Free User):** đã đăng ký và đăng nhập. Nhóm này được phép xem phim với quảng cáo và giới hạn chất lượng truyền phát, thực hiện tìm kiếm, thích (like), bình luận, và theo dõi lịch sử xem. Hành vi của họ cung cấp dữ liệu đầu vào quan trọng cho thuật toán gợi ý phim.
- **Người dùng trả phí (Premium User):** đã đăng nhập và đã nâng cấp tài khoản qua cổng thanh toán hợp lệ. Người dùng Premium có quyền truy cập toàn bộ nội dung, không bị quảng cáo, lựa chọn chất lượng cao (tối đa 4K) và tải xuống phim để xem ngoại tuyến. Đây là nhóm tạo doanh thu trực tiếp cho hệ thống.
- **Quản trị viên (Admin):** chịu trách nhiệm vận hành hệ thống nội dung và người dùng. Admin thực hiện quản lý danh mục phim (thêm, chỉnh sửa, xoá, cập nhật trạng thái), xử lý tài khoản, phân quyền Premium/Free, khóa/mở tài khoản, và theo dõi báo cáo thống kê như lượt xem hoặc doanh thu.

- **Nhà quảng cáo (Advertiser):** cung cấp nội dung quảng cáo xuất hiện trong luồng xem của người dùng Free. Họ cấu hình chiến dịch, quản lý vị trí hiển thị, phân phối quảng cáo theo nhóm đối tượng và theo dõi hiệu quả chiến dịch.
- **Bên cung cấp dịch vụ thanh toán (Payment Provider):** các đơn vị trung gian như MoMo và VNPay chịu trách nhiệm xử lý giao dịch nâng cấp Premium, cung cấp API xác thực và phản hồi trạng thái thanh toán theo chuẩn bảo mật, đồng thời đảm bảo không tiết lộ thông tin nhạy cảm của người dùng.

2.3 Phân tích yêu cầu chức năng

2.3.1 Yêu cầu chức năng cho Guest

Guest là người dùng chưa đăng nhập vào hệ thống. Các chức năng được cung cấp ở mức cơ bản nhằm giới thiệu nội dung và thu hút người dùng đăng ký tài khoản.

1. Xem danh sách và thông tin phim

- Guest có thể xem danh sách phim đang có trên hệ thống.
- Thông tin hiển thị bao gồm: tên phim, poster, thể loại, mô tả ngắn, thời lượng, diễn viên và đánh giá.
- Không được phép xem nội dung phim đầy đủ.

2. Xem trailer phim

- Guest được phép xem trailer để nắm nội dung tổng quan của phim.
- Trailer được phát ở chất lượng mặc định, không hỗ trợ tùy chỉnh nâng cao.

3. Tìm kiếm phim cơ bản

- Cho phép tìm kiếm phim theo tên hoặc thể loại.
- Kết quả tìm kiếm chỉ bao gồm thông tin giới thiệu, không kèm quyền xem phim.

4. Định hướng đăng ký và đăng nhập

- Khi Guest chọn xem phim đầy đủ hoặc thực hiện chức năng nâng cao, hệ thống yêu cầu đăng nhập hoặc đăng ký.
- Hiển thị thông báo và nút chuyển hướng đến trang đăng ký/đăng nhập.

Ví dụ: Một người dùng truy cập FlixZone lần đầu có thể duyệt danh sách phim, xem trailer và đọc mô tả, nhưng khi bấm vào nút “Xem phim” hệ thống sẽ yêu cầu đăng nhập hoặc đăng ký tài khoản.

2.3.2 Register user

Register user là người dùng đã đăng ký tài khoản miễn phí (Free user). Ngoài các chức năng của Guest, người dùng được cấp thêm các quyền sau:

1. Quản lý tài khoản người dùng

- Đăng ký tài khoản bằng email/mật khẩu hoặc đăng nhập bằng Google/Facebook.
- Cập nhật thông tin cá nhân: tên hiển thị, ảnh đại diện, mật khẩu.

2. Xem phim có giới hạn

- Được phép xem phim đầy đủ nội dung.
- Chất lượng video bị giới hạn (ví dụ: tối đa 720p).
- Phim có chèn quảng cáo trong quá trình phát.

3. Tương tác nội dung

- Thích (like) phim.
- Bình luận và xem bình luận của người dùng khác.

4. Lưu trữ lịch sử và sở thích

- Hệ thống lưu lại lịch sử xem phim của người dùng.
- Cho phép thêm phim vào danh sách yêu thích.

5. Tìm kiếm nâng cao

- Tìm kiếm phim theo diễn viên, đạo diễn hoặc từ khoá liên quan.

Ví dụ: Một người dùng đăng ký tài khoản miễn phí có thể xem trọn bộ phim với quảng cáo, để lại bình luận và lưu phim yêu thích để xem lại sau.

2.3.3 Premium user

Premium user là người dùng đã thanh toán gói dịch vụ trả phí, được cấp quyền truy cập đầy đủ vào hệ thống.

1. Xem phim không giới hạn

- Xem toàn bộ phim không bị chèn quảng cáo.
- Hỗ trợ nhiều mức chất lượng video từ 360p đến 4K.

2. Trải nghiệm nâng cao

- Tua nhanh, tua chậm và tiếp tục xem từ vị trí đã dừng.
- Ưu tiên tốc độ tải và phát video ổn định.

3. Cá nhân hoá nội dung

- Nhận gợi ý phim dựa trên lịch sử xem và hành vi người dùng.
- Trang chủ được cá nhân hoá theo sở thích.

4. Quản lý gói dịch vụ

- Gia hạn, huỷ hoặc thay đổi gói Premium.
- Xem lịch sử thanh toán và hoá đơn.

Ví dụ: Một Premium user có thể xem phim chất lượng 4K không quảng cáo và nhận danh sách phim gợi ý phù hợp với sở thích cá nhân.

2.3.4 Admin

Admin là người quản trị hệ thống, chịu trách nhiệm quản lý nội dung và người dùng.

1. Quản lý phim

- Thêm, sửa, xoá phim và cập nhật trạng thái phát hành.
- Upload video, trailer, phụ đề và metadata phim.

2. Quản lý người dùng

- Xem danh sách người dùng.
- Khoá/mở khoá tài khoản.
- Gán vai trò Free hoặc Premium cho người dùng.

3. Quản lý hệ thống

- Theo dõi số lượt xem, doanh thu và phim phổ biến.
- Xem báo cáo và thống kê hệ thống.

Ví dụ: Admin có thể đăng tải một bộ phim mới, theo dõi số lượt xem và quản lý trạng thái Premium của người dùng.

2.3.5 Nhà quảng cáo

Nhà quảng cáo là đối tác bên ngoài sử dụng FlixZone để phân phối quảng cáo đến người dùng, chủ yếu là Guest và Register user.

1. Quản lý chiến dịch quảng cáo

- Tạo mới, chỉnh sửa và hủy các chiến dịch quảng cáo.
- Khai báo thông tin chiến dịch: tên chiến dịch, thời gian chạy, ngân sách và đối tượng mục tiêu.

2. Quản lý nội dung quảng cáo

- Upload video quảng cáo hoặc banner quảng cáo.
- Cấu hình vị trí hiển thị quảng cáo (trước phim, giữa phim, banner).

3. Nhắm mục tiêu người dùng

- Lựa chọn nhóm người dùng theo tiêu chí: loại tài khoản (Guest/Free), thể loại phim quan tâm, lịch sử xem.
- Giới hạn tần suất hiển thị quảng cáo cho mỗi người dùng.

4. Theo dõi và thống kê

- Xem số lượt hiển thị (impression), lượt nhấp (click) và thời gian xem quảng cáo.
- Thống kê chi phí và hiệu quả của từng chiến dịch.

Ví dụ: Một nhà quảng cáo có thể tạo chiến dịch quảng cáo phim hành động, cấu hình hiển thị trước các bộ phim cùng thể loại và theo dõi hiệu quả chiến dịch theo thời gian thực.

2.3.6 Payment Provider

Payment Provider là hệ thống bên thứ ba (ví dụ: VNPay, MoMo) chịu trách nhiệm xử lý các giao dịch thanh toán.

1. Xử lý thanh toán

- Nhận yêu cầu thanh toán gói Premium từ FlixZone.
- Thực hiện xác thực giao dịch và xử lý thanh toán an toàn.

2. Phản hồi trạng thái giao dịch

- Gửi kết quả giao dịch (thành công/thất bại/hủy) về cho hệ thống FlixZone.
- Cung cấp mã giao dịch và thông tin xác nhận.

3. Hỗ trợ đối soát

- Cung cấp API để tra cứu lịch sử giao dịch.
- Hỗ trợ đối soát doanh thu và hoàn tiền khi cần thiết.

Ví dụ: Khi một người dùng nâng cấp tài khoản lên Premium, FlixZone gửi yêu cầu thanh toán đến VNPay; sau khi giao dịch thành công, Payment Provider phản hồi trạng thái để hệ thống kích hoạt quyền Premium cho người dùng.

2.4 Phân tích yêu cầu phi chức năng

- **NFR-01 — Hiệu năng (Performance)** Ứng dụng xem phim đòi hỏi tốc độ phản hồi cao để đảm bảo trải nghiệm xem mượt mà.
 - Trang chủ tải trong dưới 2 giây nhằm giảm tỷ lệ thoát phiên.
 - Thay đổi chất lượng video (HLS/DASH) phản hồi dưới 3 giây để không gián đoạn luồng xem.
 - Các yêu cầu này liên quan trực tiếp đến cảm nhận của người dùng về độ mượt và sự ổn định của dịch vụ.
- **NFR-02 — Bảo mật (Security)** Đặc thù của hệ thống xem phim bao gồm quản lý tài khoản và giao dịch thanh toán Premium.
 - Mật khẩu phải được mã hóa bằng bcrypt.
 - Sử dụng JWT cho xác thực phiên và HTTPS trong toàn bộ quá trình thanh toán.
 - Bảo vệ trước các tấn công XSS/CSRF.
 - Đảm bảo an toàn cho dữ liệu cá nhân, lịch sử xem và giao dịch người dùng.
- **NFR-03 — Khả năng mở rộng (Scalability)** Nền tảng xem phim có lưu lượng truy cập lớn vào giờ cao điểm.
 - Hỗ trợ tối thiểu 100 người dùng xem đồng thời.
 - Scale ngang qua Kubernetes khi có nhu cầu tăng tải.
 - Mục tiêu kiểm thử tải đạt 500 RPS để mô phỏng nhiều yêu cầu phát video cùng lúc.
- **NFR-04 — Tương thích trình duyệt (Cross-Browser Compatibility)** Người dùng truy cập qua nhiều thiết bị và trình duyệt khác nhau.
 - Player video, phụ đề, các nút điều khiển phải hoạt động đồng nhất trên Chrome, Firefox, Safari.
 - Tránh lỗi vỡ bố cục hoặc mất chức năng khi xem phim trên thiết bị di động.
- **NFR-05 — Tính sẵn sàng (Availability)** Người dùng mong muốn dịch vụ xem phim hoạt động liên tục 24/7.
 - Duy trì uptime trên 99% để không gián đoạn phát phim.
 - Sao lưu dữ liệu hàng tuần và giám sát hệ thống qua Prometheus/Grafana.
 - Đặc biệt quan trọng khi có phim mới hoặc sự kiện thu hút nhiều người xem.
- **NFR-06 — Giao diện và trải nghiệm người dùng (UI/UX)** Trải nghiệm xem phim phụ thuộc mạnh vào sự rõ ràng và mượt mà của giao diện.
 - Tìm kiếm, chọn phim, bật phụ đề, đổi chất lượng phải nhanh và trực quan.
 - Thiết kế tối giản giúp người dùng tập trung vào nội dung.
- **NFR-07 — Phục hồi lỗi (Fault Tolerance)** Lỗi mạng hoặc gián đoạn nhỏ không được phép làm dừng luồng xem.
 - Tự động retry các request segment video khi gặp lỗi.
 - Ghi log lỗi và cung cấp thông báo thân thiện khi sự cố nghiêm trọng xảy ra.

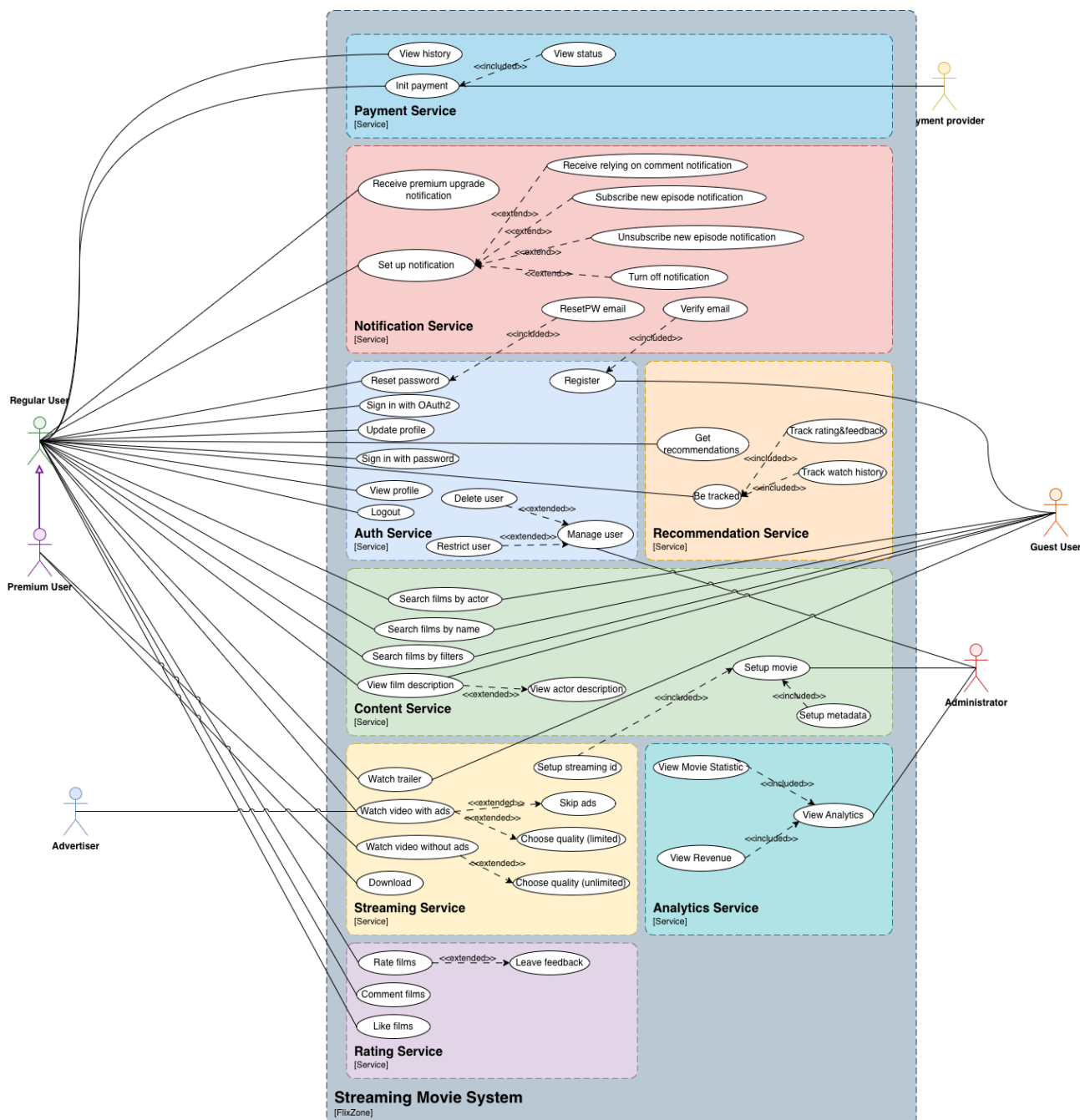


- Đảm bảo dịch vụ xem phim không bị treo khi một module đơn lẻ gặp lỗi.

3 Mô hình hóa hệ thống

Phần này xác định rõ các bên sử dụng, bối cảnh nghiệp vụ và yêu cầu cốt lõi của FlixZone. Hệ thống được thiết kế theo hướng microservices để đáp ứng nhu cầu mở rộng, tích hợp thanh toán nâng cấp Premium, phân phối quảng cáo cho nhà quảng cáo và cá nhân hoá nội dung dựa trên hành vi người dùng. Những yêu cầu được trình bày là cơ sở cho thiết kế kiến trúc, mô hình dữ liệu, cũng như kế hoạch kiểm thử và triển khai của hệ thống lần này.

3.1 Usecase tổng quan của cả hệ thống



Hình 1: Usecase tổng thể của hệ thống

3.2 Admin hệ thống

Setup Movie (ADM-UC-01)	
Use-case ID	ADM-UC-01
Use-case name	Thiết lập phim (Setup Movie)
Use-case Overview	Admin cấu hình thông tin phim và luồng phát (streaming) thông qua <i>Content Service</i> và <i>Streaming Service</i> .
Actors	Administrator
Pre-Conditions	Admin đã đăng nhập.
Trigger	Admin chọn chức năng <i>Setup Movie</i> trong giao diện quản lý nội dung.
Normal Flow	- Admin yêu cầu setup phim từ giao diện hệ thống. - Hệ thống hiển thị giao diện upload phim và gọi <i>Streaming Service</i>. - Admin upload phim, phim sẽ được tải và lưu trong Cloudflare Platform. - Hệ thống hiển thị giao diện cấu hình và gọi <i>Content Service</i>. <i>Includes: Setup metadata</i> – Admin nhập/chỉnh sửa thông tin metadata (tên, diễn viên, mô tả). <i>Includes: Setup streaming id</i> – Hệ thống tạo hoặc gán ID luồng phát cho phim. - Admin xác nhận hoàn tất thiết lập. - Hệ thống lưu trạng thái và cập nhật dữ liệu.
Post Conditions	Phim có đầy đủ metadata và streaming ID sẵn sàng để phát hành.
Alternative Flow	AF1: <i>Extension: View film description</i> – Admin xem lại mô tả chi tiết phim trước khi setup để đảm bảo tính chính xác.
Exception Flow	EF1: Lỗi kết nối <i>Content Service</i> → thông báo lỗi, không lưu metadata. EF2: Streaming ID bị trùng hoặc không hợp lệ → yêu cầu tạo lại ID mới.

Bảng 1: Use case: Setup Movie (Content Service + Streaming Service)

View Analytics (ADM-UC-02)	
Use-case ID	ADM-UC-02
Use-case name	Xem phân tích/ thống kê
Use-case Overview	Admin xem số liệu doanh thu và thống kê phim tổng hợp từ hệ thống.
Actors	Quản trị viên; <i>Analytics service</i>
Pre-Conditions	Admin đã đăng nhập vào hệ thống.
Trigger	Admin chọn chức năng phân tích trên giao diện quản trị.
Normal Flow	- Admin chọn mục "Phân tích & Thống kê" trên giao diện hệ thống quản trị. - Hệ thống điều hướng Admin đến trang lựa chọn loại báo cáo (Thống kê Phim hoặc Thống kê Doanh thu). - Hệ thống (<i>Analytics Service</i>) tự động đồng bộ và tổng hợp dữ liệu mới nhất từ các dịch vụ liên quan (Payment, Streaming, Rating...) để thực hiện tính toán phân tích. - Hệ thống hiển thị Dashboard phân tích trực quan với các biểu đồ và số liệu đã được xử lý.
Post Conditions	Bảng điều khiển hiển thị số liệu mới nhất; có thể xuất báo cáo.
Alternative Flow	AF1: Lọc dữ liệu: Tại Dashboard, Admin chọn khoảng thời gian hoặc tiêu chí cụ thể để xem chi tiết.
Exception Flow	EF1: Dịch vụ phân tích không phản hồi (Timeout/Error) → Hệ thống hiển thị thông báo lỗi và cung cấp nút "Tải lại".

Bảng 2: Use case: View Analytics (Admin)

Delete/Block/Ban User (ADM-UC-03)	
Use-case ID	ADM-UC-03
Use-case name	Xoá/Chặn/Cấm người dùng
Use-case Overview	Admin áp dụng biện pháp xử lý tài khoản theo chính sách vi phạm.
Actors	Quản trị viên; <i>User service</i>
Pre-Conditions	Có quyền quản trị người dùng; người dùng mục tiêu tồn tại.
Trigger	Chọn người dùng trong <i>Manage Users</i> .
Normal Flow	- Chọn hành động: <i>Delete</i>, <i>Block</i> (tạm thời), hoặc <i>Ban</i> (vĩnh viễn). - Nhập lý do và thời hạn (nếu <i>Block</i>). - Xác nhận; hệ thống gọi <i>User service</i> cập nhật trạng thái, thu hồi phiên đăng nhập, ghi audit.
Post Conditions	Tài khoản ở trạng thái tương ứng; hành động được lưu vết.
Alternative Flow	AF1: Gỡ chặn/mở khoá theo yêu cầu hỗ trợ sau khi xác minh.
Exception Flow	EF1: Tài khoản đang trong giao dịch thanh toán → hoãn thao tác, thông báo xử lý sau.

Bảng 3: Use case: Delete/Block/Ban User (Admin)



Create Episode (ADM-UC-04)	
Use-case ID	ADM-UC-04
Use-case name	Tạo tập phim mới (Create Episode)
Use-case Overview	Admin thêm tập phim mới vào phim bộ. Hệ thống tự động thông báo đến người dùng đã theo dõi phim bộ đó.
Actors	Administrator
Pre-Conditions	Admin đã đăng nhập; Phim bộ và Mùa phim đã tồn tại.
Trigger	Admin chọn "Thêm tập mới" trong trang quản lý phim bộ.
Normal Flow	- Admin chọn mùa phim cần thêm tập. - Hệ thống hiển thị form nhập thông tin tập phim. - Admin nhập: số tập, tiêu đề, mô tả, thời lượng, ngày phát hành. - Admin xác nhận tạo tập phim. - Hệ thống lưu tập phim và gửi thông báo đến người dùng theo dõi. - Hệ thống hiển thị thông báo thành công.
Post Conditions	Tập phim được lưu; Người theo dõi nhận thông báo tập mới.
Alternative Flow	AF1: Không có người theo dõi → Hệ thống bỏ qua gửi thông báo.
Exception Flow	EF1: Số tập đã tồn tại → Thông báo lỗi, yêu cầu nhập số tập khác. EF2: Dữ liệu không hợp lệ → Hiển thị lỗi, yêu cầu sửa thông tin.

Bảng 4: Use case: Create Episode (Admin)

Upload Video (ADM-UC-05)	
Use-case ID	ADM-UC-05
Use-case name	Tải lên video (Upload Video)
Use-case Overview	Admin tải video lên hệ thống. Video được xử lý tự động (chuyển đổi định dạng, tạo nhiều độ phân giải) và sẵn sàng phát trực tuyến.
Actors	Administrator; <i>Streaming Service</i> ; <i>Cloudflare Stream</i>
Pre-Conditions	Admin đã đăng nhập với quyền quản trị; Nội dung (phim/tập phim) đã được tạo trong hệ thống.
Trigger	Admin chọn "Tải lên video" trong trang quản lý nội dung.
Normal Flow	- Admin chọn nội dung cần gắn video và nhấn "Tải lên video". - Hệ thống tạo phiên tải lên và cung cấp URL upload trực tiếp đến CDN. - Admin chọn file video và bắt đầu tải lên. - Video được tải trực tiếp lên Cloudflare Stream. - CDN tự động xử lý video: chuyển đổi định dạng, tạo nhiều độ phân giải (480p, 720p, 1080p), và tạo hình thu nhỏ. - CDN gửi thông báo trạng thái xử lý về hệ thống qua webhook. - Hệ thống cập nhật trạng thái video thành "Sẵn sàng phát". - Hệ thống hiển thị thông báo thành công cho Admin.
Post Conditions	Video được lưu trữ trên CDN và sẵn sàng phát trực tuyến; Nội dung được gắn streaming ID.
Alternative Flow	AF1: Thay thế video: Admin tải video mới để thay thế video cũ → Hệ thống xóa video cũ và xử lý video mới.
Exception Flow	EF1: Xử lý video thất bại → Hệ thống đánh dấu trạng thái "Lỗi" và thông báo Admin. EF2: Mất kết nối khi tải lên → Admin có thể thử lại từ đầu.

Bảng 5: Use case: Upload Video (Streaming Service + Cloudflare Stream)

3.3 Người dùng khách - Guest User

Register (GUEST-UC-01)	
Use-case ID	GUEST-UC-01
Use-case name	Đăng ký tài khoản (Register)
Use-case Overview	Người dùng khách tạo tài khoản mới thông qua <i>User Service</i> để trở thành Regular User.
Actors	Guest User; <i>User Service</i>
Pre-Conditions	Người dùng chưa đăng nhập.
Trigger	Người dùng chọn nút "Register" trên giao diện.
Normal Flow	- Người dùng nhập thông tin đăng ký (email, mật khẩu, họ tên). - Hệ thống gửi dữ liệu đến <i>User Service</i> để kiểm tra tính hợp lệ. <i>User Service</i> tạo bản ghi người dùng mới với trạng thái chờ kích hoạt. - Hệ thống thông báo đăng ký thành công và yêu cầu xác thực (nếu có). - Người dùng khách xác nhận thông qua hướng dẫn được gửi về email.
Post Conditions	Tài khoản mới được tạo; Người dùng chuyển sang trạng thái chờ xác thực hoặc đăng nhập.
Alternative Flow	AF1: Đăng ký bằng OAuth2.
Exception Flow	EF1: Email đã tồn tại trong <i>User Service</i> → hiển thị thông báo lỗi, yêu cầu đăng nhập. EF2: Mật khẩu không đúng quy định bảo mật → yêu cầu nhập lại.

Bảng 6: Use case: Register (Guest User)



Search Films (GUEST-UC-02)	
Use-case ID	GUEST-UC-02
Use-case name	Tìm kiếm phim (Search Films)
Use-case Overview	Guest User tìm kiếm nội dung thông qua <i>Content Service</i> theo nhiều tiêu chí khác nhau.
Actors	Guest User; <i>Content Service</i>
Pre-Conditions	Hệ thống <i>Content Service</i> đang hoạt động.
Trigger	Người dùng nhập từ khoá hoặc chọn bộ lọc tìm kiếm.
Normal Flow	- Người dùng nhập keyword để tìm kiếm vào thanh search bar. - Hệ thống gửi query đến <i>Content Service</i> tương ứng với lựa chọn của người dùng: • Search films by name: Tìm theo tên phim. • Search films by actor: Tìm theo tên diễn viên. <i>Content Service</i> trả về danh sách phim khớp với từ khoá/bộ lọc. - Hệ thống cho phép filter kết quả search theo thể loại, năm phát hành, tuổi,... - Hệ thống hiển thị kết quả tìm kiếm dưới dạng lưới (grid).
Post Conditions	Danh sách phim phù hợp được hiển thị.
Alternative Flow	AF1: Người dùng sắp xếp kết quả (Sort) theo độ phổ biến hoặc ngày phát hành.
Exception Flow	EF1: Không tìm thấy kết quả phù hợp → hiển thị thông báo "No results found" và gợi ý từ khóa khác.

Bảng 7: Use case: Search Films (Guest User)

View Film Description (GUEST-UC-03)	
Use-case ID	GUEST-UC-03
Use-case name	Xem mô tả phim (View Film Description)
Use-case Overview	Guest User xem thông tin chi tiết của một bộ phim cụ thể.
Actors	Guest User; <i>Content Service</i>
Pre-Conditions	Người dùng đã chọn một bộ phim từ danh sách được hiển thị sẵn hoặc kết quả tìm kiếm.
Trigger	Người dùng nhấn vào poster hoặc tiêu đề phim.
Normal Flow	- Hệ thống gửi content ID của phim đến <i>Content Service</i>. <i>Content Service</i> trả về metadata chi tiết (cốt truyện, trailer, diễn viên, rating). - Hệ thống hiển thị trang chi tiết phim.
Post Conditions	Thông tin chi tiết phim được hiển thị.
Alternative Flow	AF1: Extension: View actor description – Người dùng nhấn vào tên diễn viên trong trang chi tiết phim → Hệ thống hiển thị popup/trang thông tin về diễn viên đó.
Exception Flow	EF1: ID phim không tồn tại (lỗi đường dẫn) → chuyển hướng về trang chủ hoặc trang 404.

Bảng 8: Use case: View Film Description (Guest User)

Watch Trailer (GUEST-UC-04)	
Use-case ID	GUEST-UC-05
Use-case name	Xem trailer (Watch Trailer)
Use-case Overview	Guest User xem trailer thông qua trình phát nhúng (Embedded Player) hoặc chuyển hướng sang YouTube.
Actors	Guest User; <i>Content Service</i>
Pre-Conditions	Metadata của phim trong <i>Content Service</i> đã có URL trailer YouTube hợp lệ.
Trigger	Người dùng nhấn nút "Watch Trailer" trên giao diện chi tiết phim hoặc banner.
Normal Flow	- Hệ thống gọi <i>Content Service</i> để lấy Trailer URL. - Hệ thống hiển thị trình phát YouTube Embedded Player với mã video tương ứng. - Video tự động tải; người dùng thực hiện các thao tác (Play, Pause, Fullscreen) trên giao diện YouTube Player.
Post Conditions	Trailer được phát; người dùng vẫn ở lại trang hoặc quay lại trang chi tiết sau khi đóng popup.
Alternative Flow	AF1: Direct Redirect: Nếu cấu hình hệ thống không cho phép nhúng hoặc trên thiết bị di động → Hệ thống mở tab mới chuyển hướng trực tiếp sang trang YouTube/App YouTube.
Exception Flow	EF1: Video trên YouTube bị xóa hoặc để chế độ riêng tư (Private) → Player hiển thị thông báo "Video unavailable" từ phía YouTube. EF2: URL trong <i>Content Service</i> bị lỗi format → Hệ thống thông báo "Trailer not found".

Bảng 9: Use case: Watch Trailer (Guest User - YouTube Source)

3.4 Người dùng thường - Regular User

Sign In (REC-UC-01)	
Use-case ID	REC-UC-01
Use-case name	Đăng nhập (Sign In)
Use-case Overview	Người dùng truy cập vào hệ thống thông qua <i>User Service</i> để sử dụng các tính năng cá nhân hoá.
Actors	Regular User; <i>User Service</i>
Pre-Conditions	Người dùng đã có tài khoản (đã đăng ký).
Trigger	Người dùng chọn "Sign In" trên trang chủ.
Normal Flow	1. Người dùng chọn phương thức đăng nhập bằng Mật khẩu (<i>Sign in with password</i>). 2. Nhập Email và Mật khẩu. 3. Hệ thống gửi thông tin đến <i>User Service</i> để xác thực. 4. <i>User Service</i> xác nhận hợp lệ và trả về token phiên làm việc (Session/JWT). 5. Hệ thống chuyển hướng người dùng về trang chủ với trạng thái đã đăng nhập.
Post Conditions	Người dùng truy cập được các chức năng của thành viên; Dữ liệu phiên được lưu.
Alternative Flow	AF1: Sign in with OAuth2: Người dùng chọn đăng nhập qua Google/Facebook → Hệ thống chuyển hướng sang Provider xác thực → <i>User Service</i> tạo/cập nhật user và cấp token. AF2: Forgot Password: Người dùng chọn "Reset password" → Hệ thống kích hoạt luồng gửi email khôi phục (<i>Notification Service</i>).
Exception Flow	EF1: Sai tên đăng nhập hoặc mật khẩu → Hiện thị thông báo lỗi. EF2: Tài khoản bị khoá (<i>Restrict user</i>) → Thông báo lý do và liên hệ admin.

Bảng 10: Use case: Sign In (User Service)

Watch Movie (REC-UC-02)	
Use-case ID	REC-UC-02
Use-case name	Xem phim (Watch Movie)
Use-case Overview	Người dùng xem phim qua <i>Streaming Service</i> . Hệ thống phân biệt luồng xử lý giữa tài khoản thường (Ads) và Premium (No Ads).
Actors	Regular User; Premium User; <i>Streaming Service</i> ; Advertiser
Pre-Conditions	Người dùng đã đăng nhập; Phim ở trạng thái sẵn sàng.
Trigger	Người dùng nhấn "Play" tại trang chi tiết phim.
Normal Flow	- Hệ thống kiểm tra gói dịch vụ của người dùng (Regular hay Premium). Nếu là Regular User: - Hệ thống kích hoạt <i>Watch video with ads</i>. - Gọi <i>Advertiser</i> để tải quảng cáo pre-roll. - Người dùng xem quảng cáo (có thể <i>Skip ads</i> sau 5s nếu cho phép). <i>Streaming Service</i> phát phim với chất lượng giới hạn (Limited Quality). - Hệ thống ghi nhận lịch sử xem (Tracking).
Post Conditions	Phim được phát; Lịch sử xem được cập nhật.
Alternative Flow	AF1: Premium User Flow: - Hệ thống kích hoạt <i>Watch video without ads</i>. - Bỏ qua bước gọi Advertiser. <i>Streaming Service</i> phát phim ngay lập tức. - Người dùng tùy chọn chất lượng cao nhất (<i>Choose quality unlimited</i>). AF2: Download (Premium only): Người dùng chọn tải xuống → <i>Streaming Service</i> chuẩn bị file offline được mã hoá DRM.
Exception Flow	EF1: Gói Premium hết hạn ngay lúc đang xem → Hệ thống hiển thị thông báo và chuyển sang luồng Regular (kèm quảng cáo).

Bảng 11: Use case: Watch Movie (Streaming Service)

Rate Content (REC-UC-03)	
Use-case ID	REC-UC-03
Use-case name	Đánh giá nội dung (Rate Content)
Use-case Overview	Người dùng đánh giá phim hoặc phim bộ bằng điểm số (1-5 sao). Hệ thống cập nhật điểm đánh giá trung bình và đồng bộ với hệ thống tìm kiếm.
Actors	Regular User; <i>Rating Service</i> ; <i>Content Service</i>
Pre-Conditions	Người dùng đã đăng nhập; Nội dung cần đánh giá tồn tại trong hệ thống.
Trigger	Người dùng nhấn vào biểu tượng sao đánh giá trên trang chi tiết phim.
Normal Flow	1. Người dùng xem trang chi tiết phim và chọn số sao đánh giá (1-5). 2. Hệ thống gửi yêu cầu đánh giá đến <i>Rating Service</i>. 3. <i>Rating Service</i> kiểm tra người dùng đã đánh giá nội dung này trước đó chưa. 4. Hệ thống lưu đánh giá mới hoặc cập nhật đánh giá cũ. 5. Hệ thống tính toán lại điểm đánh giá trung bình và gửi sự kiện cập nhật. 6. <i>Content Service</i> nhận sự kiện và đồng bộ điểm trung bình với hệ thống tìm kiếm. 7. Hệ thống hiển thị xác nhận đánh giá thành công.
Post Conditions	Đánh giá được lưu; Điểm trung bình được cập nhật trên trang chi tiết và kết quả tìm kiếm.
Alternative Flow	AF1: Cập nhật đánh giá: Người dùng đã đánh giá trước đó → Hệ thống cập nhật số sao mới thay vì tạo mới. AF2: Xóa đánh giá: Người dùng chọn xóa đánh giá → Hệ thống xóa và tính lại điểm trung bình.
Exception Flow	EF1: Người dùng chưa đăng nhập → Chuyển hướng đến trang đăng nhập. EF2: Lỗi kết nối dịch vụ → Hiển thị thông báo lỗi, yêu cầu thử lại.

Bảng 12: Use case: Rate Content (Rating Service + Content Service)

Reset Password (REC-UC-04)	
Use-case ID	REC-UC-04
Use-case name	Đặt lại mật khẩu (Reset Password)
Use-case Overview	Người dùng yêu cầu khôi phục mật khẩu khi bị quên, hệ thống kích hoạt gửi email qua <i>Notification Service</i> .
Actors	Regular User; <i>User Service</i> ; <i>Notification Service</i>
Pre-Conditions	Tài khoản email của người dùng đã tồn tại trong hệ thống.
Trigger	Người dùng chọn "Forgot Password" tại màn hình đăng nhập.
Normal Flow	1. Người dùng nhập địa chỉ email đăng ký. 2. <i>User Service</i> kiểm tra sự tồn tại của email. 3. <i>Includes: ResetPW email</i> – <i>User Service</i> gọi <i>Notification Service</i> để tạo token và gửi email hướng dẫn đặt lại mật khẩu. 4. Người dùng nhận email, nhấn vào đường dẫn xác thực. 5. Hệ thống hiển thị trang nhập mật khẩu mới. 6. Người dùng nhập mật khẩu mới và xác nhận. 7. <i>User Service</i> cập nhật mật khẩu đã mã hoá vào cơ sở dữ liệu.
Post Conditions	Mật khẩu được thay đổi; Người dùng có thể đăng nhập bằng mật khẩu mới.
Exception Flow	EF1: Email không tồn tại → Hệ thống hiển thị thông báo kiểm tra lại email. EF2: Token đặt lại mật khẩu hết hạn → Yêu cầu người dùng thực hiện lại quy trình từ đầu.

Bảng 13: Use case: Reset Password (User Service)

View Profile (REC-UC-05)	
Use-case ID	REC-UC-05
Use-case name	Xem hồ sơ cá nhân (View Profile)
Use-case Overview	Người dùng xem thông tin tài khoản và trạng thái gói dịch vụ hiện tại.
Actors	Regular User; <i>User Service</i>
Pre-Conditions	Người dùng đã đăng nhập thành công.
Trigger	Người dùng nhấn vào Avatar hoặc tên tài khoản trên thanh điều hướng.
Normal Flow	1. Hệ thống gửi yêu cầu lấy thông tin chi tiết đến <i>User Service</i>. 2. <i>User Service</i> truy xuất thông tin (Họ tên, Email, Avatar, Trạng thái gói cước Regular/Premium). 3. Hệ thống hiển thị trang hồ sơ người dùng.
Post Conditions	Thông tin người dùng được hiển thị.
Alternative Flow	AF1: View history: Từ trang profile, người dùng chuyển sang xem lịch sử thanh toán (thuộc <i>Payment Service</i>).

Bảng 14: Use case: View Profile (User Service)

Update Profile (REC-UC-06)	
Use-case ID	REC-UC-06
Use-case name	Cập nhật hồ sơ (Update Profile)
Use-case Overview	Người dùng chỉnh sửa thông tin cá nhân trong tài khoản.
Actors	Regular User; <i>User Service</i>
Pre-Conditions	Người dùng đang ở màn hình <i>View Profile</i> .
Trigger	Người dùng nhấn nút "Edit Profile" hoặc "Cập nhật thông tin".
Normal Flow	- Người dùng chỉnh sửa các trường cho phép (Họ tên, Số điện thoại, Ảnh đại diện). - Người dùng nhấn "Lưu thay đổi". - Hệ thống gửi dữ liệu mới đến <i>User Service</i>. <i>User Service</i> xác thực dữ liệu (định dạng ảnh, độ dài tên) và cập nhật vào cơ sở dữ liệu. - Hệ thống thông báo cập nhật thành công và hiển thị thông tin mới.
Post Conditions	Thông tin tài khoản được làm mới.
Exception Flow	EF1: Định dạng ảnh upload không hợp lệ hoặc quá dung lượng → Hệ thống báo lỗi và yêu cầu chọn ảnh khác. EF2: Dữ liệu nhập vào chứa ký tự cấm → Hệ thống từ chối cập nhật.

Bảng 15: Use case: Update Profile (User Service)

Log Out (REC-UC-07)	
Use-case ID	REC-UC-07
Use-case name	Đăng xuất (Log Out)
Use-case Overview	Người dùng kết thúc phiên làm việc hiện tại, hệ thống thu hồi quyền truy cập.
Actors	Regular User; <i>User Service</i>
Pre-Conditions	Người dùng đang đăng nhập.
Trigger	Người dùng chọn chức năng "Log Out" từ menu tài khoản.
Normal Flow	- Hệ thống gửi yêu cầu đăng xuất tới <i>User Service</i>. <i>User Service</i> vô hiệu hóa Token xác thực hiện tại (hoặc xóa session). - Hệ thống xóa thông tin lưu trữ cục bộ (Local Storage/Cookies) trên trình duyệt người dùng. - Điều hướng người dùng về trang chủ dành cho Khách (Guest User).
Post Conditions	Phiên làm việc kết thúc; Người dùng trở về trạng thái Guest.
Exception Flow	EF1: Lỗi kết nối mạng khi gửi yêu cầu → Hệ thống vẫn thực hiện xóa token ở phía Client (Force Logout) để đảm bảo an toàn cục bộ.

Bảng 16: Use case: Log Out (User Service)

Set up Notification (REC-UC-08)	
Use-case ID	REC-UC-08
Use-case name	Cài đặt thông báo (Set up Notification)
Use-case Overview	Người dùng cấu hình các tùy chọn nhận thông báo (phím mới, phản hồi bình luận) hoặc tắt toàn bộ thông báo từ hệ thống.
Actors	Regular User; <i>Notification Service</i>
Pre-Conditions	Người dùng đã đăng nhập vào hệ thống.
Trigger	Người dùng truy cập mục "Cài đặt thông báo" (Notification Settings).
Normal Flow	- Hệ thống hiển thị trạng thái cấu hình hiện tại từ <i>Notification Service</i>. - Người dùng thực hiện thay đổi các tùy chọn cài đặt (chi tiết trong Alternative Flows). - Người dùng nhấn nút "Lưu thay đổi" (Save Changes). - Hệ thống cập nhật Preferences vào cơ sở dữ liệu. - Hiển thị thông báo "Cập nhật thành công".
Post Conditions	Cấu hình thông báo mới có hiệu lực ngay lập tức.
Alternative Flow	AF1: Extension: Subscribe/Unsubscribe New Episode - Người dùng bật/tắt tùy chọn "Thông báo phim tập mới". - Hệ thống thêm/xoá User khỏi danh sách đăng ký nhận thông báo phim bộ. AF2: Extension: Configure Reply Notification - Người dùng bật/tắt tùy chọn "Thông báo khi có phản hồi bình luận". - Hệ thống cập nhật cờ <code>allow_reply_noti</code> cho tài khoản. AF3: Extension: Turn off notification (Global) - Người dùng kích hoạt chế độ "Tắt tất cả thông báo" (Do not disturb). - Hệ thống vô hiệu hoá toàn bộ Push Notification đến thiết bị.
Exception Flow	EF1: Không lưu được cấu hình (Lỗi Server/Mạng) → Hệ thống giữ nguyên trạng thái cũ và báo lỗi "Update failed, try again later".

Bảng 17: Use case: Set up Notification (Notification Service)

Init Payment / Upgrade Premium (REC-UC-09)	
Use-case ID	REC-UC-09
Use-case name	Khởi tạo thanh toán (Init Payment)
Use-case Overview	Người dùng thực hiện thanh toán để nâng cấp tài khoản. Quá trình này bao gồm việc nhận thông báo xác nhận và xem trạng thái giao dịch.
Actors	Regular User; <i>Payment Service</i> ; <i>Notification Service</i> ; Payment Provider
Pre-Conditions	Người dùng đã đăng nhập và chọn gói dịch vụ muốn đăng ký.
Trigger	Người dùng nhấn nút "Nâng cấp Premium".
Normal Flow	- Hệ thống gửi yêu cầu <i>Init payment</i> đến <i>Payment Service</i>. <i>Payment Service</i> tạo mã giao dịch và chuyển hướng người dùng sang cổng thanh toán (<i>Payment Provider</i>). - Người dùng hoàn tất quá trình thanh toán trên giao diện của bên thứ 3. <i>Payment Service</i> nhận tín hiệu thành công (Callback/IPN) từ Provider. - Hệ thống cập nhật trạng thái tài khoản người dùng lên Premium. <i>Includes: Receive premium upgrade notification</i> – Hệ thống gọi <i>Notification Service</i> gửi email/thông báo xác nhận nâng cấp thành công. <i>Includes: View status</i> – Hệ thống hiển thị màn hình kết quả giao dịch và hạn sử dụng gói.
Post Conditions	Tài khoản trở thành Premium; Người dùng nhận được thông báo xác nhận.
Exception Flow	EF1: Thanh toán thất bại hoặc người dùng huỷ ở bước 3 → <i>Payment Service</i> ghi nhận trạng thái "Cancelled/Failed" → Hiển thị thông báo lỗi tại bước <i>View status</i>. EF2: Lỗi gửi thông báo (<i>Notification Service</i> down) → Giao dịch vẫn thành công, nhưng thông báo sẽ được đưa vào hàng đợi gửi sau (<i>Retry mechanism</i>).

Bảng 18: Use case: Init Payment (Payment Service)

View History (REC-UC-10)	
Use-case ID	REC-UC-10
Use-case name	Xem lịch sử giao dịch (View History)
Use-case Overview	Người dùng xem lại danh sách các hoá đơn và trạng thái thanh toán cũ.
Actors	Regular/Premium User; <i>Payment Service</i>
Pre-Conditions	Người dùng đã đăng nhập.
Trigger	Người dùng chọn tab "Lịch sử thanh toán" (<i>Billing History</i>) trong trang hồ sơ.
Normal Flow	- Hệ thống gửi yêu cầu lấy dữ liệu đến <i>Payment Service</i>. <i>Payment Service</i> truy xuất danh sách giao dịch (Ngày, Mã GD, Số tiền, Trạng thái, Tên gói). - Hệ thống hiển thị bảng danh sách lịch sử giao dịch.
Post Conditions	Danh sách giao dịch được hiển thị.
Alternative Flow	AF1: Filter History: Người dùng lọc giao dịch theo năm hoặc trạng thái (Thành công/Thất bại).
Exception Flow	EF1: Chưa có giao dịch nào → Hiển thị thông báo "No transaction history".

Bảng 19: Use case: View History (Payment Service)

Get Recommendations (REC-UC-11)	
Use-case ID	REC-UC-11
Use-case name	Nhận gợi ý phim (Get recommendations)
Use-case Overview	Hệ thống hiển thị danh sách phim được cá nhân hoá dựa trên sở thích và lịch sử hoạt động của người dùng.
Actors	Regular User; <i>Recommendation Service</i>
Pre-Conditions	Người dùng đã đăng nhập và có dữ liệu lịch sử (xem/đánh giá).
Trigger	Người dùng truy cập Trang chủ hoặc trang "Dành cho bạn".
Normal Flow	- Hệ thống gửi User ID đến <i>Recommendation Service</i>. <i>Recommendation Service</i> phân tích dữ liệu người dùng (dựa trên <i>Generate recommendations</i>). - Service trả về danh sách các bộ phim phù hợp (ví dụ: "Vì bạn đã xem...", "Có thể bạn sẽ thích"). - Hệ thống hiển thị danh sách phim lên giao diện người dùng.
Post Conditions	Danh sách phim gợi ý được hiển thị.
Alternative Flow	AF1: Cold Start: Nếu là người dùng mới chưa có lịch sử → Hệ thống trả về danh sách Trending/Popular (giống Guest User).
Exception Flow	EF1: Lỗi kết nối Service → Hiển thị thông báo "Không thể tải gợi ý" hoặc hiển thị danh sách phim ngẫu nhiên từ Cache.

Bảng 20: Use case: Get Recommendations (Regular User)

Be Tracked (REC-UC-12)	
Use-case ID	REC-UC-12
Use-case name	Được theo dõi hành vi (Be tracked)
Use-case Overview	Hệ thống tự động ghi nhận các tương tác của người dùng để phục vụ thuật toán gợi ý. Bao gồm theo dõi lịch sử xem và đánh giá.
Actors	Regular User; <i>Recommendation Service</i>
Pre-Conditions	Người dùng đang tương tác với nội dung (xem phim hoặc đánh giá).
Trigger	Sự kiện ngầm: Người dùng dừng xem phim, xem xong phim, hoặc gửi đánh giá.
Normal Flow	- Hệ thống phát hiện tương tác người dùng. <i>Includes: Track watch history</i> – Nếu người dùng đang xem phim: Hệ thống gửi thông tin (Phim, thời lượng đã xem, thời điểm dừng) về Service. <i>Includes: Track rating & feedback</i> – Nếu người dùng đánh giá/bình luận: Hệ thống gửi thông tin (Số sao, nội dung) về Service. <i>Recommendation Service</i> cập nhật vector sở thích (User Profile) của người dùng.
Post Conditions	Dữ liệu hành vi mới được ghi nhận để cải thiện kết quả gợi ý lần sau.
Exception Flow	EF1: Người dùng bật chế độ ẩn danh (Incognito) → Hệ thống bỏ qua việc ghi log hành vi này.

Bảng 21: Use case: Be Tracked (Implicit Collection)

Subscribe to Series (REC-13)	
Use-case ID	REC-13
Use-case name	Theo dõi phim bộ (Subscribe to Series)
Use-case Overview	Người dùng đăng ký theo dõi phim bộ yêu thích để nhận thông báo khi có tập mới phát hành.
Actors	Regular User; <i>Notification Service</i>
Pre-Conditions	Người dùng đã đăng nhập; Phim bộ tồn tại trong hệ thống.
Trigger	Người dùng nhấn nút "Theo dõi" trên trang chi tiết phim bộ.
Normal Flow	1. Người dùng truy cập trang chi tiết phim bộ. 2. Người dùng nhấn nút "Theo dõi" (Subscribe). 3. Hệ thống gửi yêu cầu đến <i>Notification Service</i>. 4. Hệ thống kiểm tra người dùng đã theo dõi phim bộ này chưa. 5. Hệ thống lưu đăng ký theo dõi mới. 6. Hệ thống hiển thị xác nhận "Đã theo dõi thành công". 7. Khi có tập mới, người dùng nhận thông báo tự động.
Post Conditions	Người dùng được thêm vào danh sách theo dõi; Nhận thông báo khi có tập mới.
Alternative Flow	AF1: Huỷ theo dõi: Người dùng nhấn "Huỷ theo dõi" → Hệ thống xoá đăng ký và ngừng gửi thông báo. AF2: Xem danh sách theo dõi: Người dùng truy cập "Phim bộ đang theo dõi" để xem tất cả phim bộ đã đăng ký.
Exception Flow	EF1: Đã theo dõi trước đó → Hiển thị thông báo "Bạn đã theo dõi phim bộ này". EF2: Người dùng chưa đăng nhập → Chuyển hướng đến trang đăng nhập.

Bảng 22: Use case: Subscribe to Series (Notification Service)

3.5 Người dùng trả phí - Premium User

Watch movies without ads (PREM-UC-01)	
Use-case ID	PREM-UC-01
Use-case name	Xem phim không quảng cáo
Use-case Overview	Người dùng Premium xem nội dung không chèn quảng cáo. Use case <i>includes</i> Track Usage ; có thể <i>extend</i> sang <i>View film description</i> và <i>View actor descriptions</i> .
Actors	Premium User; <i>Movie service</i> ; <i>Recommendation service</i>
Pre-Conditions	Subscription Premium còn hiệu lực; nội dung khả dụng.
Trigger	Nhấn “Xem phim”.
Normal Flow	- Hệ thống xác thực trạng thái Premium và chuẩn bị nguồn HLS/DASH. - Phát nội dung chính liên tục, không chèn quảng cáo. <i>Include: Track Usage</i> – ghi play/pause/seek/completion theo thời gian thực.
Post Conditions	Lịch sử xem và thống kê được cập nhật; gợi ý được cải thiện.
Alternative/Exception	AF1 Hết hạn Premium trong khi xem → thông báo gia hạn ở lần phát tiếp theo. EF1 Lỗi stream → đề nghị đổi chất lượng/khôi phục kết nối.

Bảng 23: Use case: Watch movies without ads (Premium)

Watch trailers (PREM-UC-02)	
Use-case ID	PREM-UC-02
Use-case name	Xem trailer
Use-case Overview	Phát trailer HLS; có thể <i>extend</i> sang <i>View film description</i> và <i>View actor descriptions</i> .
Actors	Premium User; <i>Movie service</i>
Pre-Conditions	Trailer tồn tại và được phép xem.
Trigger	Nhấn “Xem trailer” từ trang phim/kết quả tìm kiếm.
Normal Flow	- Hệ thống tải manifest và player. - Trailer phát; người dùng điều khiển play/pause/chất lượng.
Post Conditions	Ghi nhận lượt xem trailer.
Exception Flow	EF1 Nguồn phát lỗi → hiển thị thông báo và gợi ý thử lại.

Bảng 24: Use case: Watch trailers (Premium)

View film description (PREM-UC-03)	
Use-case ID	PREM-UC-03
Use-case name	Xem mô tả phim
Use-case Overview	Hiển thị chi tiết phim: nội dung, thể loại, diễn viên, trailer, xếp hạng.
Actors	Premium User; <i>Movie service</i>
Pre-Conditions	Phim tồn tại trong catalog.
Trigger	Chọn một phim trong danh sách/kết quả tìm kiếm.
Normal Flow	- Tải metadata phim. - Hiển thị chi tiết; cho phép chuyển sang xem phim/trailer.
Post Conditions	Không có.
Exception Flow	EF1 Phim không khả dụng → thông báo và quay lại danh sách.

Bảng 25: Use case: View film description (Premium)

View actor descriptions (PREM-UC-04)	
Use-case ID	PREM-UC-04
Use-case name	Xem thông tin diễn viên
Use-case Overview	Hiển thị hồ sơ diễn viên và filmography; thường được mở rộng từ xem trailer/xem phim.
Actors	Premium User; <i>Movie service</i>
Pre-Conditions	Diễn viên có dữ liệu hồ sơ.
Trigger	Nhấn vào tên/ảnh diễn viên.
Normal Flow	- Tải thông tin diễn viên và các phim liên quan. - Hiển thị trang hồ sơ; cho phép điều hướng đến phim khác.
Post Conditions	Không có.
Exception Flow	EF1 Hồ sơ thiếu dữ liệu → hiển thị “đang cập nhật”.

Bảng 26: Use case: View actor descriptions (Premium)

Track Usage (PREM-UC-05)	
Use-case ID	PREM-UC-05
Use-case name	Ghi nhận hành vi xem
Use-case Overview	Thu thập sự kiện xem để phân tích/gợi ý. (Được <i>include</i> bởi Watch movies without ads).
Actors	Premium User; <i>Recommendation service</i>
Pre-Conditions	Người dùng đang xem nội dung.
Trigger	Player gửi sự kiện theo thời gian thực/batch.
Normal Flow	- Thu thập play/pause/seek/completion. - Gửi sự kiện tới Recommendation/Analytics qua API/queue. - Cập nhật lịch sử xem của người dùng.
Post Conditions	Dữ liệu hành vi được lưu; dùng cho gợi ý và thống kê.
Exception Flow	EF1 Mất kết nối → đệm cục bộ và gửi lại khi có mạng.

Bảng 27: Use case: Track Usage (Premium)

Download (PREM-UC-06)	
Use-case ID	PREM-UC-06
Use-case name	Tải xuống nội dung (offline)
Use-case Overview	Cho phép Premium tải tệp video/trailer dùng ngoại tuyến (trong đồ án có thể mô phỏng/giới hạn; không triển khai DRM).
Actors	Premium User; <i>Movie service</i>
Pre-Conditions	Nội dung cho phép tải; dung lượng còn đủ.
Trigger	Nhấn “Tải xuống/Download”.
Normal Flow	- Hệ thống tạo liên kết tải/hàng đợi job. - Người dùng bắt đầu tải; theo dõi tiến trình.
Post Conditions	Tệp lưu cục bộ; ghi lịch sử tải.
Alternative/Exception	AF1 Hủy tải giữa chừng → cho phép resume (nếu hỗ trợ). EF1 Hết dung lượng/kết nối gián đoạn → hiển thị thông báo và cho phép thử lại.

Bảng 28: Use case: Download (Premium)

Login (PREM-UC-07)	
Use-case ID	PREM-UC-07
Use-case name	Đăng nhập
Use-case Overview	Xác thực người dùng Premium để truy cập nội dung; <i>extends</i> sang Reset password .
Actors	Premium User; <i>User service</i>
Pre-Conditions	Tài khoản Verified ; đang đăng xuất.
Trigger	Chọn “Login/Đăng nhập”.
Normal Flow	- Nhập email/mật khẩu; xác thực. - Tạo phiên (JWT) kèm thông tin gói (Premium/hết hạn).
Post Conditions	Người dùng ở trạng thái đăng nhập; token hợp lệ.
Exception/Extend	- Sai thông tin/khóa tạm thời. - Người dùng chọn “Quên mật khẩu” → <i>extends</i> sang Reset password.

Bảng 29: Use case: Login (Premium)

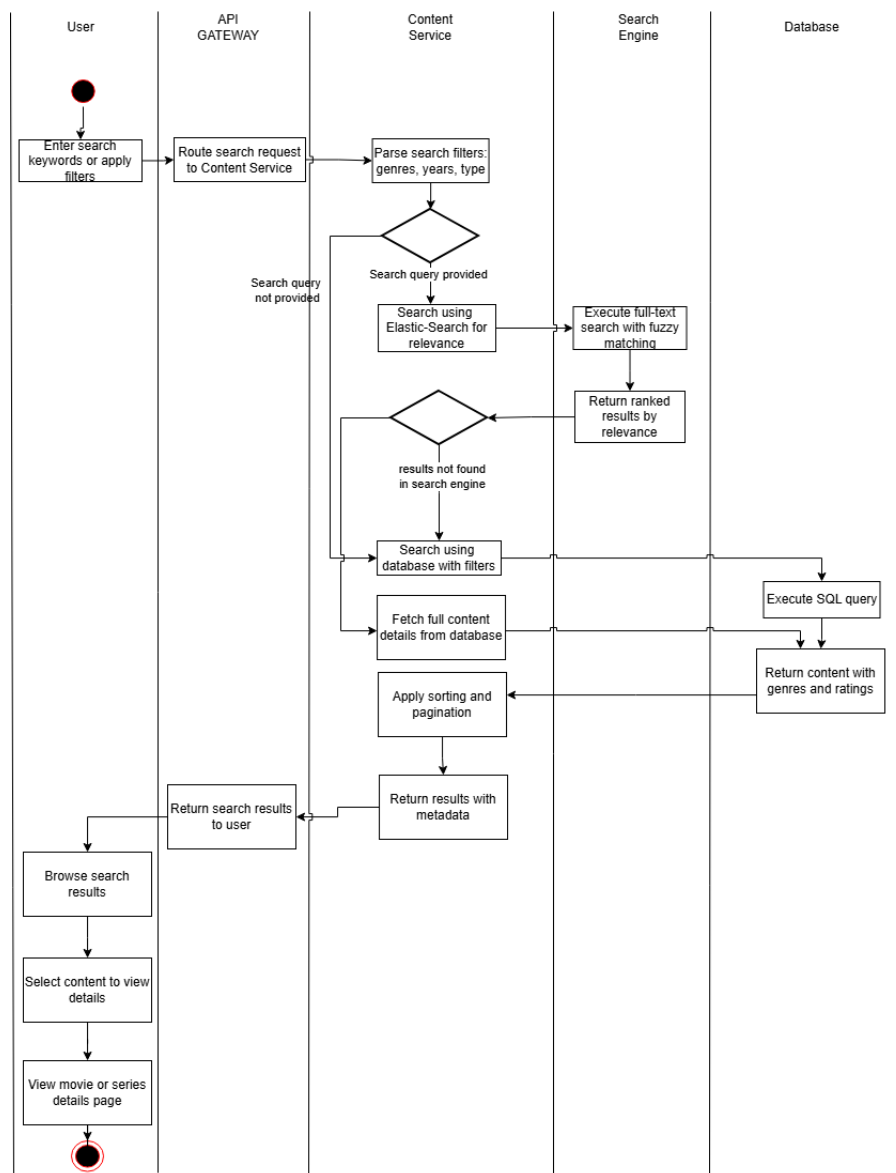
Reset password (PREM-UC-08)	
Use-case ID	PREM-UC-08
Use-case name	Khôi phục mật khẩu
Use-case Overview	Đặt lại mật khẩu qua email/token. <i>Extends</i> từ Login .
Actors	Premium User; <i>User service</i>
Pre-Conditions	Có tài khoản hợp lệ; email hoạt động.
Trigger	Chọn “Quên mật khẩu/Reset password”.
Normal Flow	- Nhập email; hệ thống gửi liên kết/token đặt lại. - Mở liên kết, nhập mật khẩu mới và xác nhận. - Hệ thống cập nhật mật khẩu; thu hồi các phiên cũ.
Post Conditions	Mật khẩu mới có hiệu lực; có thể đăng nhập lại.
Exception Flow	EF1 Email không tồn tại/không nhận được thư → cho phép gửi lại. EF2 Token hết hạn/không hợp lệ → yêu cầu tạo yêu cầu mới.

Bảng 30: Use case: Reset password (Premium)

3.6 Sơ đồ thiết kế hành vi

Dưới đây trình bày một số hoạt động nghiệp vụ chính của hệ thống dưới dạng sơ đồ hoạt động. Chi tiết nghiệp vụ đã được mô tả đầy đủ trong các bảng Usecase ở mục 3.

3.6.1 Tìm kiếm nội dung

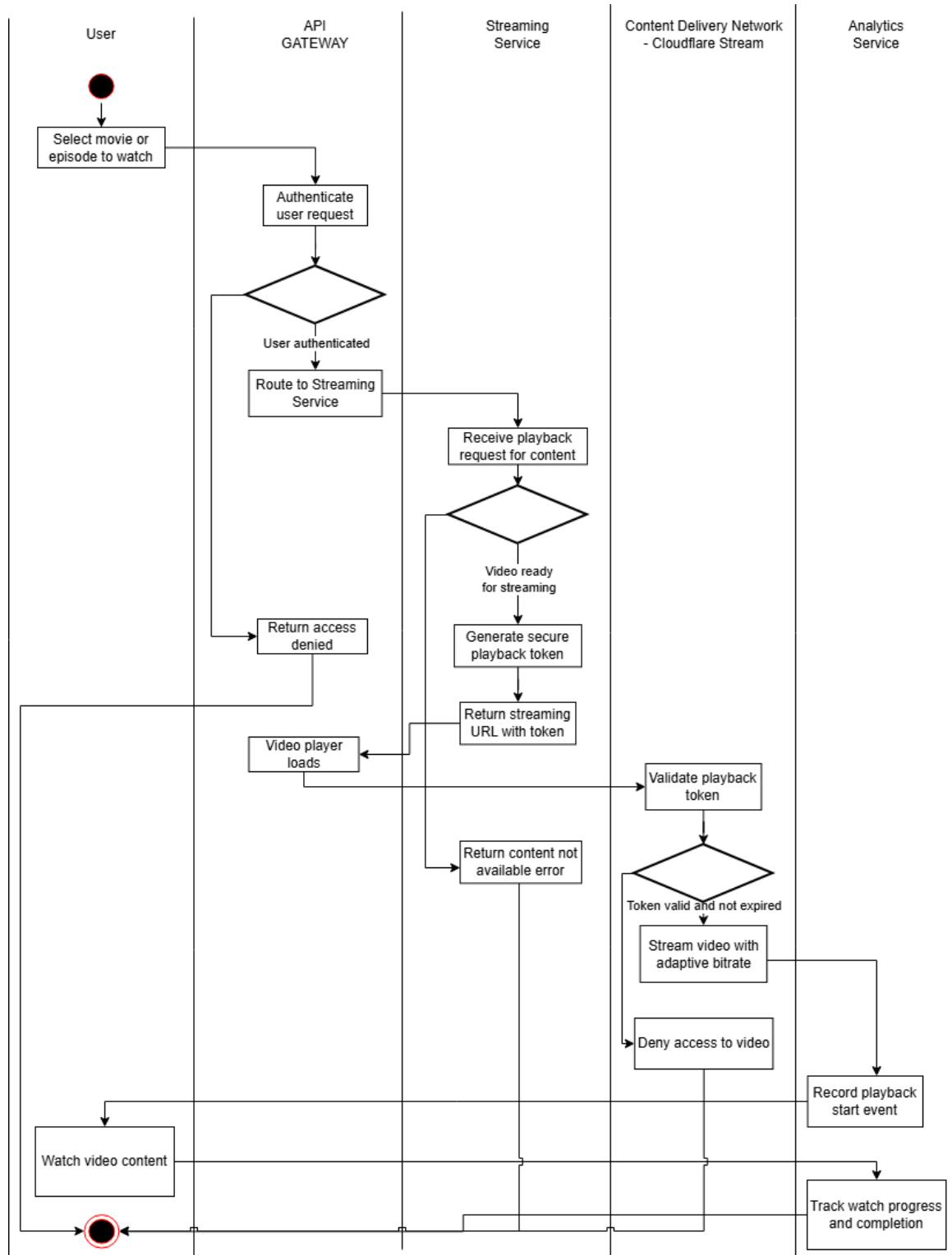


Hình 2: Sơ đồ hoạt động của use case Search Films

- Người dùng mở ứng dụng FlixZone và truy cập vào trang tìm kiếm bằng cách nhấn vào biểu tượng kính lúp hoặc thanh tìm kiếm trên giao diện chính.
- Ứng dụng hiển thị giao diện tìm kiếm với thanh nhập từ khóa và các bộ lọc nâng cao bao gồm: thể loại (genres), năm phát hành (years), loại nội dung (phim lẻ/phim bộ), và đánh giá tối thiểu.
- Người dùng nhập từ khóa tìm kiếm (ví dụ: "Avengers") hoặc chọn các bộ lọc mong muốn (ví dụ: thể loại Hành động, năm 2023-2024), sau đó nhấn nút "Tìm Kiếm".
- Ứng dụng gửi yêu cầu tìm kiếm đến API Gateway kèm theo các tham số: từ khóa (query), danh sách thể loại (genres), danh sách năm (years), loại nội dung (type), và thông tin phân trang (page, limit).
- API Gateway tiếp nhận yêu cầu và chuyển tiếp đến Content Service để xử lý logic tìm kiếm.

- Content Service phân tích các tham số tìm kiếm và xử lý theo hai trường hợp:
 - Có từ khóa tìm kiếm: Content Service gọi đến Elasticsearch để thực hiện tìm kiếm toàn văn (full-text search) với thuật toán fuzzy matching, giúp tìm được kết quả
 - Không có từ khóa (chỉ lọc): Content Service truy vấn trực tiếp cơ sở dữ liệu PostgreSQL với các điều kiện lọc theo thể loại và năm phát hành.
- Elasticsearch thực hiện tìm kiếm đa trường (multi-match) trên các thuộc tính tiêu đề (title), mô tả (description), và tên diễn viên, sau đó trả về danh sách ID nội dung được xếp hạng theo độ liên quan (relevance score).
- Content Service nhận danh sách ID từ Elasticsearch và truy vấn PostgreSQL để lấy thông tin đầy đủ của từng nội dung bao gồm: tiêu đề, ảnh poster, thể loại, năm phát hành, đánh giá trung bình, và mô tả ngắn.
- Hệ thống áp dụng logic sắp xếp (theo độ phù hợp, đánh giá, hoặc năm phát hành) và phân trang theo yêu cầu của người dùng.
- Content Service trả về kết quả tìm kiếm cho API Gateway, bao gồm danh sách nội dung và metadata phân trang (tổng số kết quả, trang hiện tại, tổng số trang).
- API Gateway chuyển tiếp kết quả về ứng dụng di động/web của người dùng.
- Ứng dụng hiển thị kết quả tìm kiếm dưới dạng lưới (grid) hoặc danh sách (list), mỗi mục bao gồm ảnh poster, tiêu đề, năm phát hành, đánh giá, và các thể loại.
- Trong quá trình tìm kiếm, hệ thống xử lý hai trường hợp đặc biệt:
 - Không tìm thấy kết quả: Ứng dụng hiển thị thông báo "Không tìm thấy kết quả phù hợp" kèm gợi ý thay đổi bộ lọc hoặc từ khóa.
 - Elasticsearch không khả dụng: Content Service tự động chuyển sang sử dụng PostgreSQL làm phương thức tìm kiếm dự phòng (fallback) với tìm kiếm ILIKE.
- Người dùng có thể nhấn vào một mục trong kết quả để xem trang chi tiết nội dung, hoặc điều chỉnh bộ lọc để tinh chỉnh kết quả tìm kiếm, hoặc cuộn xuống để tải thêm kết quả (infinite scroll/pagination).

3.6.2 Xem phim

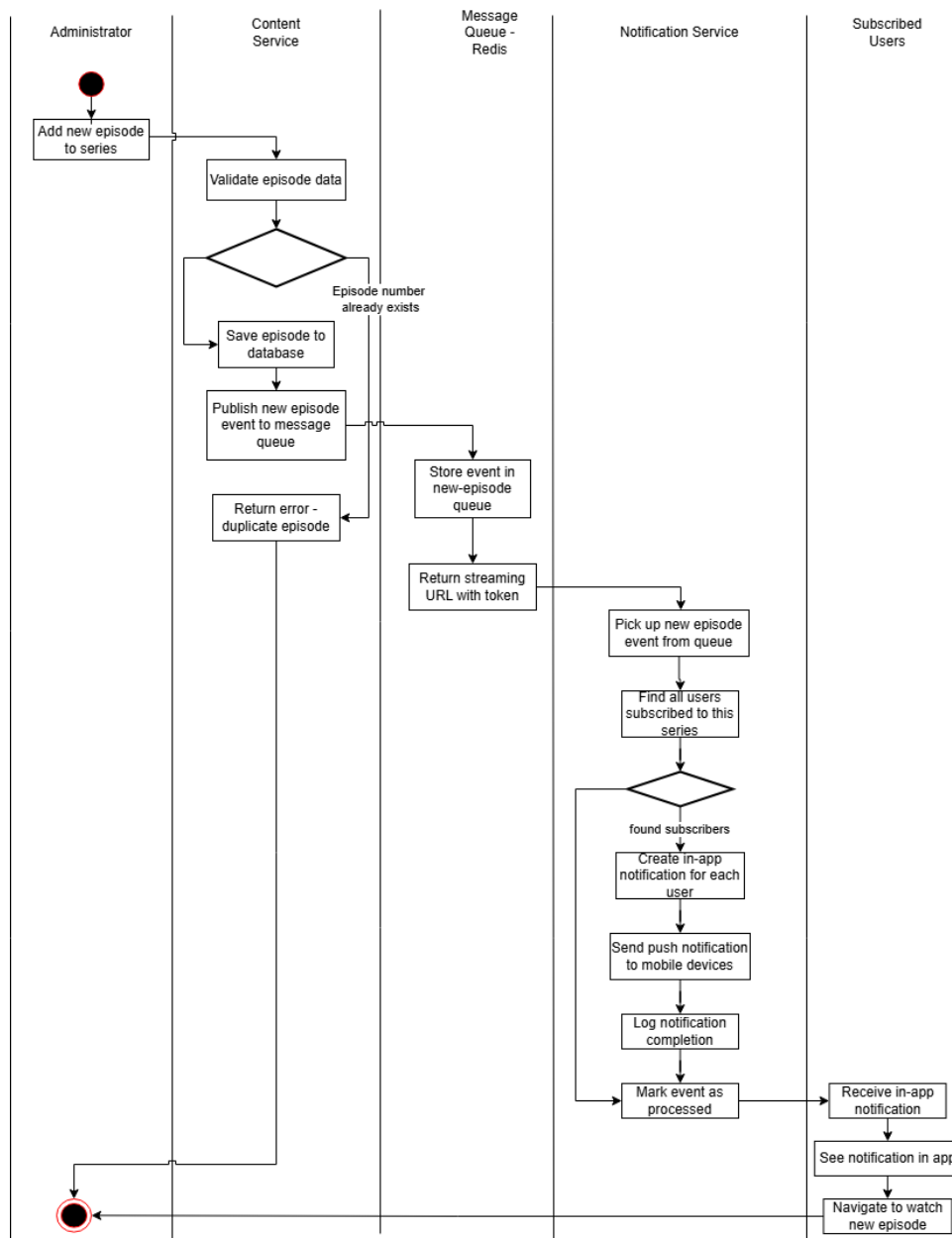


Hình 3: Sơ đồ hoạt động của usecase Xem phim

- Người dùng duyệt qua danh sách phim hoặc phim bộ trên ứng dụng FlixZone và chọn một nội dung muốn xem bằng cách nhấn vào ảnh poster hoặc tiêu đề.
- Ứng dụng hiển thị trang chi tiết nội dung bao gồm: tiêu đề, mô tả, thể loại, năm phát hành, đánh giá, thời lượng, danh sách diễn viên, và nút "Xem Phim" nổi bật.
- Người dùng nhấn nút "Xem Phim" (hoặc chọn tập cụ thể nếu là phim bộ) để bắt đầu phát video.

- Ứng dụng gửi yêu cầu phát video đến API Gateway kèm theo thông tin: ID nội dung (contentId) và token xác thực người dùng (JWT).
- API Gateway tiếp nhận yêu cầu và thực hiện xác thực người dùng thông qua JWT token:
 - Không xác thực thành công: API Gateway trả về mã lỗi 401 Unauthorized, ứng dụng hiển thị thông báo yêu cầu đăng nhập và chuyển hướng người dùng đến trang đăng nhập.
 - Xác thực thành công: API Gateway chuyển tiếp yêu cầu đến Streaming Service để xử lý.
- Streaming Service nhận yêu cầu và truy vấn cơ sở dữ liệu MongoDB để kiểm tra trạng thái video:
 - Video chưa sẵn sàng: Streaming Service trả về thông báo lỗi "Nội dung hiện không khả dụng" nếu video đang được xử lý hoặc không tồn tại.
 - Video sẵn sàng: Streaming Service tiếp tục quy trình tạo phiên phát.
- Streaming Service tạo playback token bảo mật có thời hạn (signed URL) bằng cách sử dụng Cloudflare Stream API, token này chứa thông tin: ID video, thời gian hết hạn, và chữ ký số.
- Streaming Service trả về URL phát video kèm playback token cho API Gateway, API Gateway chuyển tiếp kết quả về ứng dụng người dùng.
- Ứng dụng khởi tạo trình phát video (video player) với URL nhận được và bắt đầu tải video từ Cloudflare Stream CDN.
- Cloudflare Stream CDN tiếp nhận yêu cầu phát video và xác thực playback token:
 - Token không hợp lệ hoặc hết hạn: CDN từ chối yêu cầu và trả về lỗi 403 Forbidden, ứng dụng hiển thị thông báo "Phiên phát đã hết hạn, vui lòng thử lại".
 - Token hợp lệ: CDN bắt đầu stream video với công nghệ Adaptive Bitrate Streaming (HLS/DASH).
- CDN tự động điều chỉnh chất lượng video dựa trên tốc độ mạng của người dùng:
 - Mạng nhanh: Phát video ở độ phân giải cao (1080p, 4K).
 - Mạng chậm: Tự động giảm xuống độ phân giải thấp hơn (720p, 480p) để đảm bảo phát mượt mà không giật lag.
- Trong quá trình xem, ứng dụng định kỳ gửi thông tin tiến trình xem (watch progress) đến Streaming Service để lưu trữ vào MongoDB, bao gồm: ID nội dung, ID người dùng, thời điểm hiện tại trong video, và thời lượng đã xem.
- Analytics Service ghi nhận các sự kiện phát video: bắt đầu phát (play), tạm dừng (pause), tua (seek), và hoàn thành (complete) để phục vụ thống kê và đề xuất cá nhân hóa.
- Người dùng có thể:
 - Điều khiển phát: Tạm dừng, tua tiến/tua lùi, điều chỉnh âm lượng, bật/tắt phụ đề.
 - Chọn chất lượng: Thay đổi độ phân giải video theo ý muốn (Tự động, 1080p, 720p, 480p).
 - Xem toàn màn hình: Chuyển sang chế độ toàn màn hình để có trải nghiệm xem tốt hơn.
- Khi người dùng đóng trình phát video hoặc thoát ứng dụng, hệ thống lưu lại vị trí xem cuối cùng để người dùng có thể tiếp tục xem từ điểm dừng trong lần truy cập tiếp theo (Continue Watching).
- Nếu người dùng xem hoàn thành nội dung (>90% thời lượng), hệ thống đánh dấu nội dung là "Đã xem" và có thể hiển thị gợi ý nội dung liên quan hoặc tập tiếp theo (nếu là phim bộ).

3.6.3 Admin tạo tập phim mới



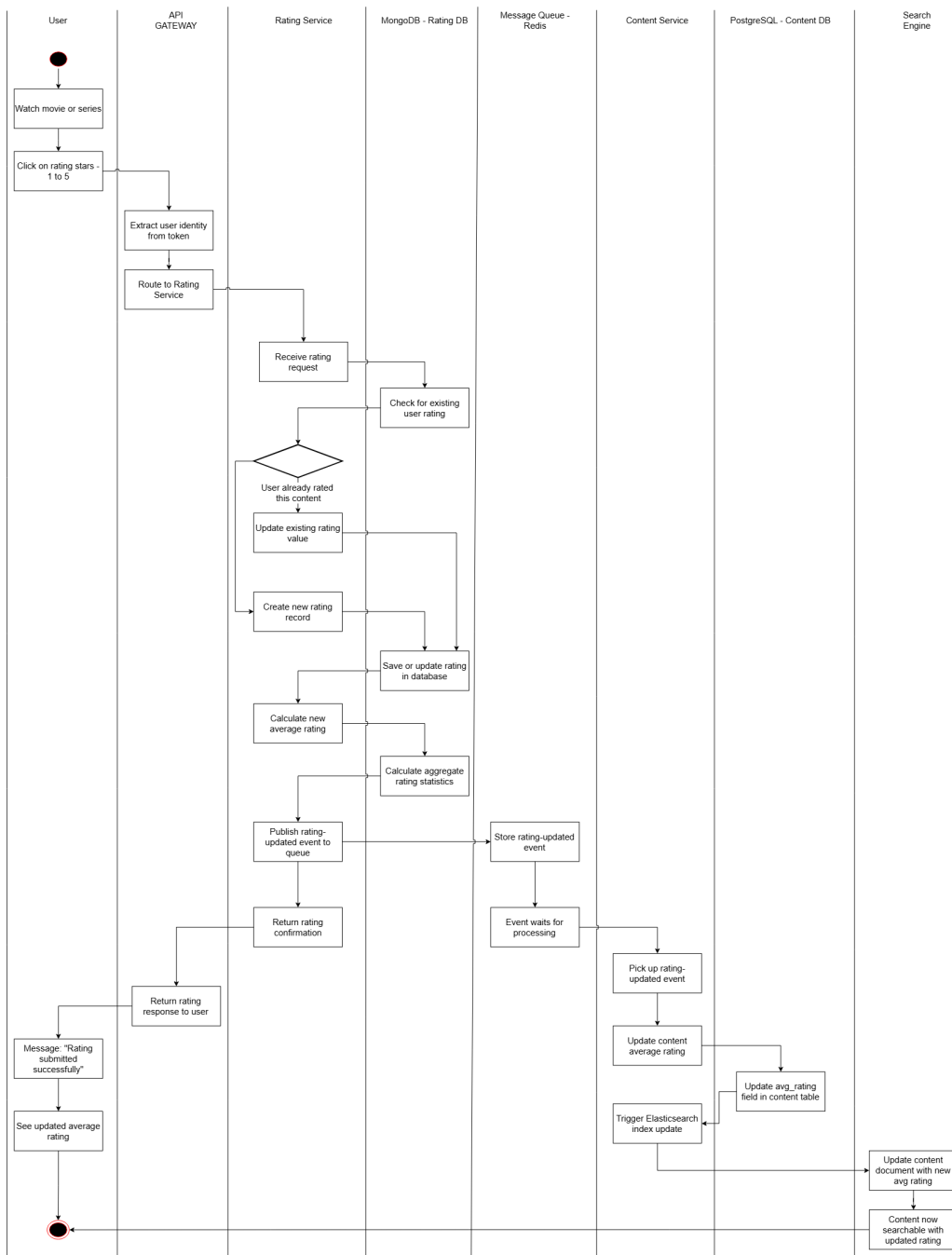
Hình 4: Sơ đồ hoạt động của Usecase Tạo tập phim mới

- Admin truy cập vào trang quản lý phim bộ và chọn mùa phim cần thêm tập mới.
- Hệ thống hiển thị form nhập thông tin tập phim bao gồm: số tập, tiêu đề, mô tả, thời lượng, và ngày phát hành.
- Admin nhập đầy đủ thông tin và nhấn nút "Tạo Tập Phim" để xác nhận.
- Hệ thống kiểm tra tính hợp lệ của dữ liệu:
 - Số tập đã tồn tại: Hệ thống hiển thị thông báo lỗi và yêu cầu Admin nhập số tập khác.
 - Dữ liệu hợp lệ: Hệ thống tiếp tục lưu tập phim vào cơ sở dữ liệu.
- Sau khi lưu thành công, hệ thống tự động tạo sự kiện thông báo tập mới và đưa vào hàng đợi xử lý.
- Hệ thống tìm kiếm danh sách người dùng đã đăng ký theo dõi phim bộ này:
 - Có người theo dõi: Hệ thống tạo thông báo và gửi notification đến từng người dùng.

– Không có người theo dõi: Hệ thống ghi nhận và bỏ qua bước gửi thông báo.

- Hệ thống đánh dấu hoàn tất xử lý sự kiện và ghi log kết quả.
- Người dùng đã theo dõi phim bộ nhận được thông báo trên website với nội dung: "Tập mới của [Tên phim] đã có! Xem ngay tập [Số tập]: [Tiêu đề tập]".
- Người dùng có thể nhấn vào thông báo để chuyển trực tiếp đến trang xem tập phim mới.
- Admin nhận được thông báo thành công và có thể tiếp tục thêm tập khác hoặc gắn video streaming cho tập vừa tạo.

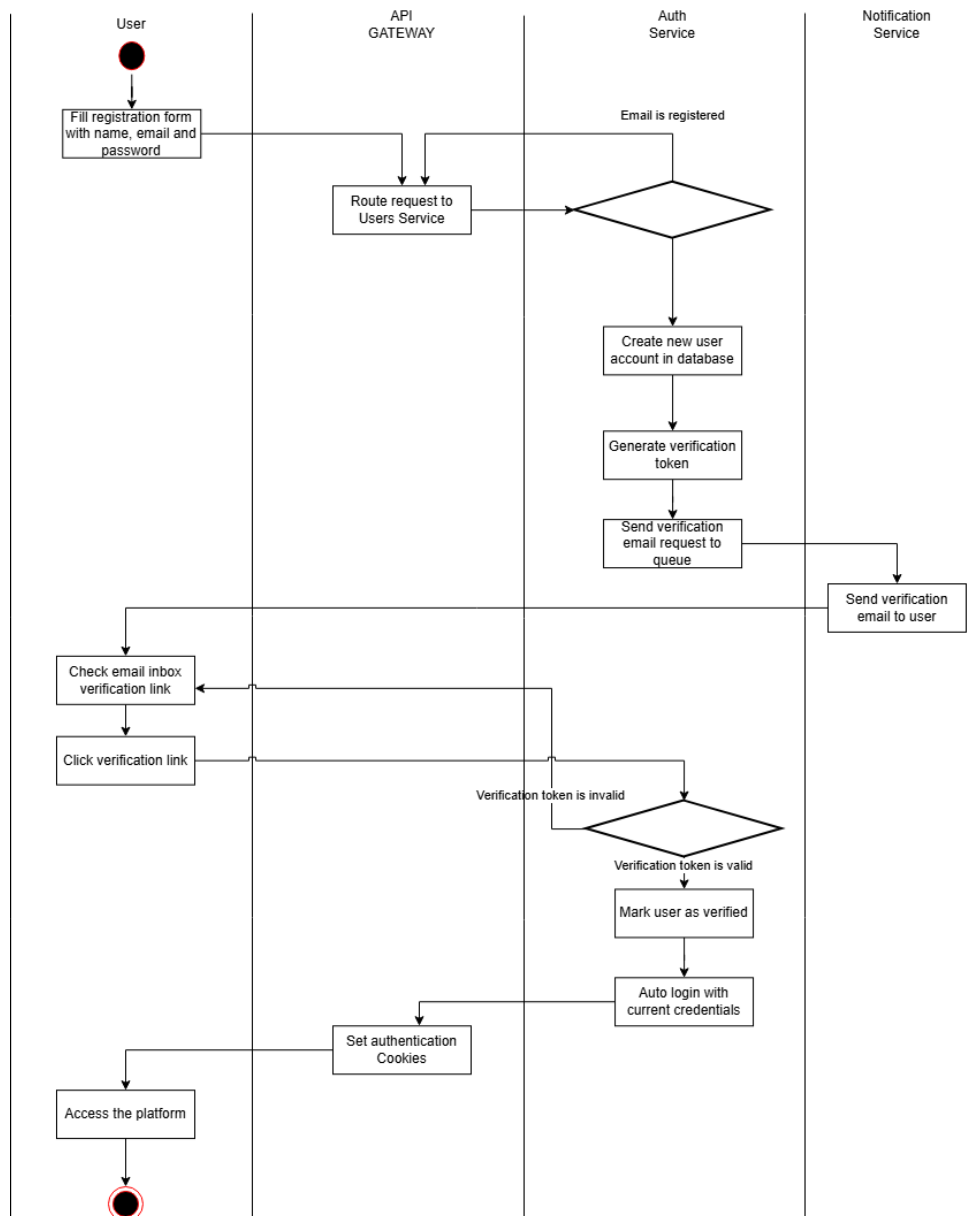
3.6.4 Người dùng viết đánh giá phim



Hình 5: Sơ đồ hoạt động của người dùng viết đánh giá phim

- Người dùng xem trang chi tiết phim hoặc phim bộ và quyết định đánh giá nội dung bằng cách nhấn vào biểu tượng sao (1-5 sao).
- Ứng dụng gửi yêu cầu đánh giá đến API Gateway kèm theo thông tin: ID nội dung, số sao đánh giá, và token xác thực người dùng (JWT).
- API Gateway trích xuất thông tin người dùng từ JWT token và chuyển tiếp yêu cầu đến Rating Service.
- Rating Service nhận yêu cầu và truy vấn cơ sở dữ liệu MongoDB để kiểm tra người dùng đã đánh giá nội dung này trước đó hay chưa:
 - Đã đánh giá trước đó: Hệ thống cập nhật số sao mới cho đánh giá hiện có.
 - Chưa đánh giá: Hệ thống tạo bản ghi đánh giá mới.
- Rating Service lưu đánh giá vào MongoDB và tính toán lại điểm đánh giá trung bình cho nội dung dựa trên tất cả đánh giá hiện có.
- Rating Service tạo sự kiện "rating-updated" chứa thông tin điểm trung bình mới và đẩy vào hàng đợi tin nhắn (Message Queue - Redis).
- Rating Service trả về phản hồi thành công cho người dùng ngay lập tức, không cần chờ các bước xử lý tiếp theo.
- Message Queue lưu trữ sự kiện và chờ Content Service xử lý bất đồng bộ.
- Content Service nhận sự kiện từ hàng đợi và cập nhật trường điểm đánh giá trung bình (avg_rating) trong bảng nội dung trên cơ sở dữ liệu PostgreSQL.
- Content Service kích hoạt cập nhật chỉ mục Elasticsearch để đồng bộ điểm đánh giá mới, giúp kết quả tìm kiếm phản ánh đúng thứ hạng dựa trên đánh giá.
- Elasticsearch cập nhật tài liệu nội dung với điểm trung bình mới, cho phép người dùng có thể lọc và sắp xếp kết quả tìm kiếm theo đánh giá.
- Người dùng thấy xác nhận đánh giá thành công và điểm đánh giá trung bình được cập nhật trên trang chi tiết phim.

3.6.5 Người dùng đăng ký tài khoản

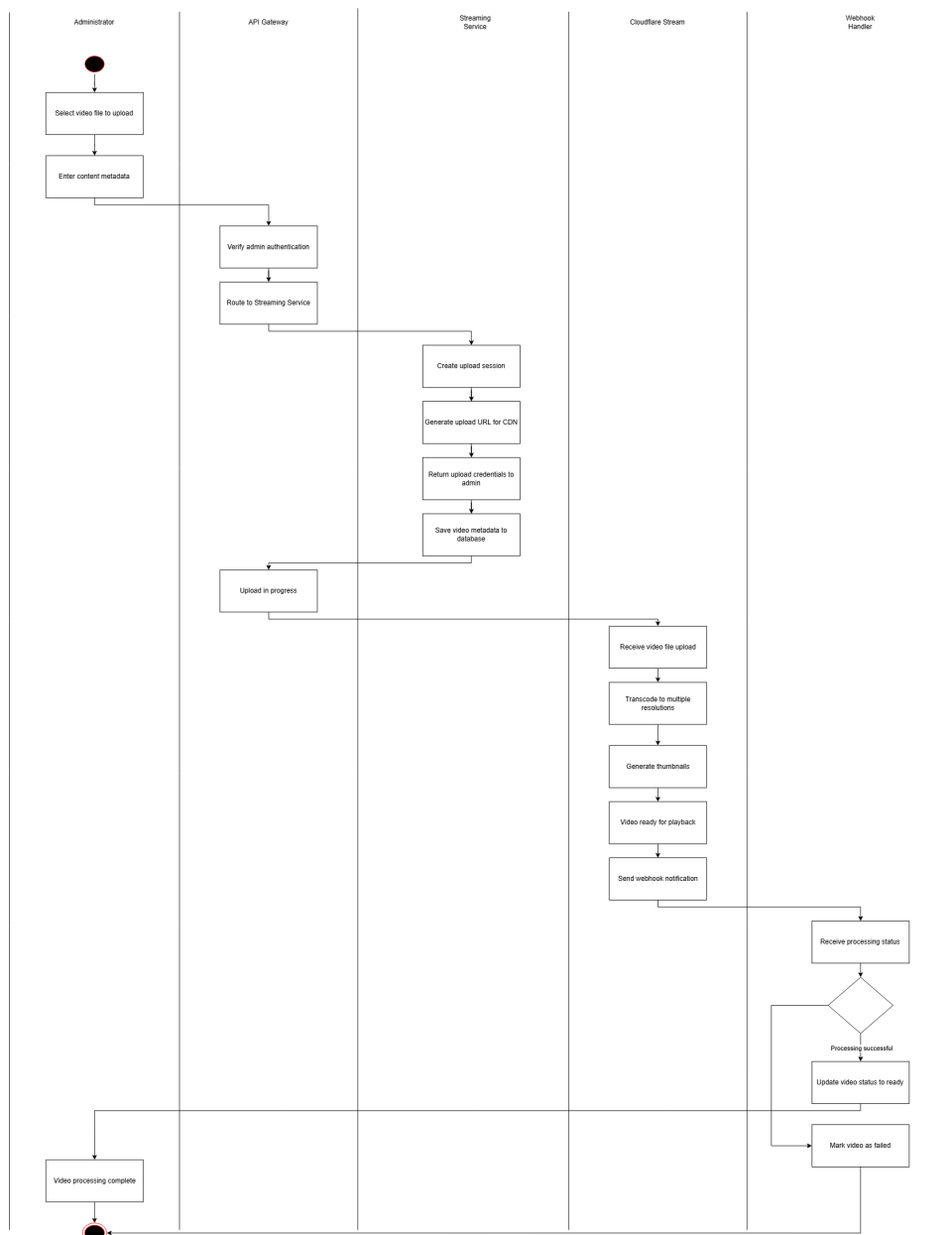


Hình 6: Sơ đồ hoạt động của Usecase Người dùng đăng ký tài khoản

- Người dùng truy cập trang chủ FlixZone và nhấn nút "Đăng Ký" để bắt đầu quá trình tạo tài khoản mới.
- Ứng dụng hiển thị form đăng ký yêu cầu người dùng nhập thông tin: họ tên, địa chỉ email, và mật khẩu.
- Người dùng điền đầy đủ thông tin và nhấn nút "Tạo Tài Khoản" để xác nhận.
- Ứng dụng gửi yêu cầu đăng ký đến API Gateway, sau đó được chuyển tiếp đến Users Service để xử lý.
- Users Service kiểm tra địa chỉ email đã được đăng ký trong hệ thống hay chưa:
 - Email đã tồn tại: Hệ thống trả về thông báo lỗi "Email đã được sử dụng" và dừng quá trình đăng ký.
 - Email chưa tồn tại: Hệ thống tiếp tục tạo tài khoản mới.
- Users Service mã hóa mật khẩu và lưu thông tin người dùng mới vào cơ sở dữ liệu PostgreSQL với trạng thái "Chưa xác thực".
- Users Service tạo token xác thực email có thời hạn và gửi yêu cầu gửi email đến hàng đợi tin nhắn.

- Notification Service nhận yêu cầu từ hàng đợi và gửi email xác thực đến địa chỉ email người dùng với đường link kích hoạt tài khoản.
- Người dùng kiểm tra hộp thư email và nhấn vào đường link xác thực trong email nhận được.
- Hệ thống kiểm tra tính hợp lệ của token xác thực:
 - Token không hợp lệ hoặc hết hạn: Hiển thị thông báo lỗi và yêu cầu người dùng gửi lại email xác thực.
 - Token hợp lệ: Hệ thống cập nhật trạng thái tài khoản thành "Đã xác thực".
- Sau khi xác thực thành công, người dùng được chuyển đến trang đăng nhập và nhập thông tin đăng nhập (email, mật khẩu).
- Users Service xác thực thông tin đăng nhập:
 - Thông tin sai: Hiển thị thông báo lỗi "Email hoặc mật khẩu không chính xác".
 - Thông tin đúng: Hệ thống tạo access token và refresh token (JWT).
- API Gateway thiết lập cookie xác thực trên trình duyệt và chuyển hướng người dùng về trang chủ với trạng thái đã đăng nhập.
- Người dùng có thể truy cập tất cả tính năng của thành viên như: xem phim, đánh giá, tạo danh sách yêu thích, và đăng ký theo dõi phim bộ.

3.6.6 Admin tải video mới lên hệ thống

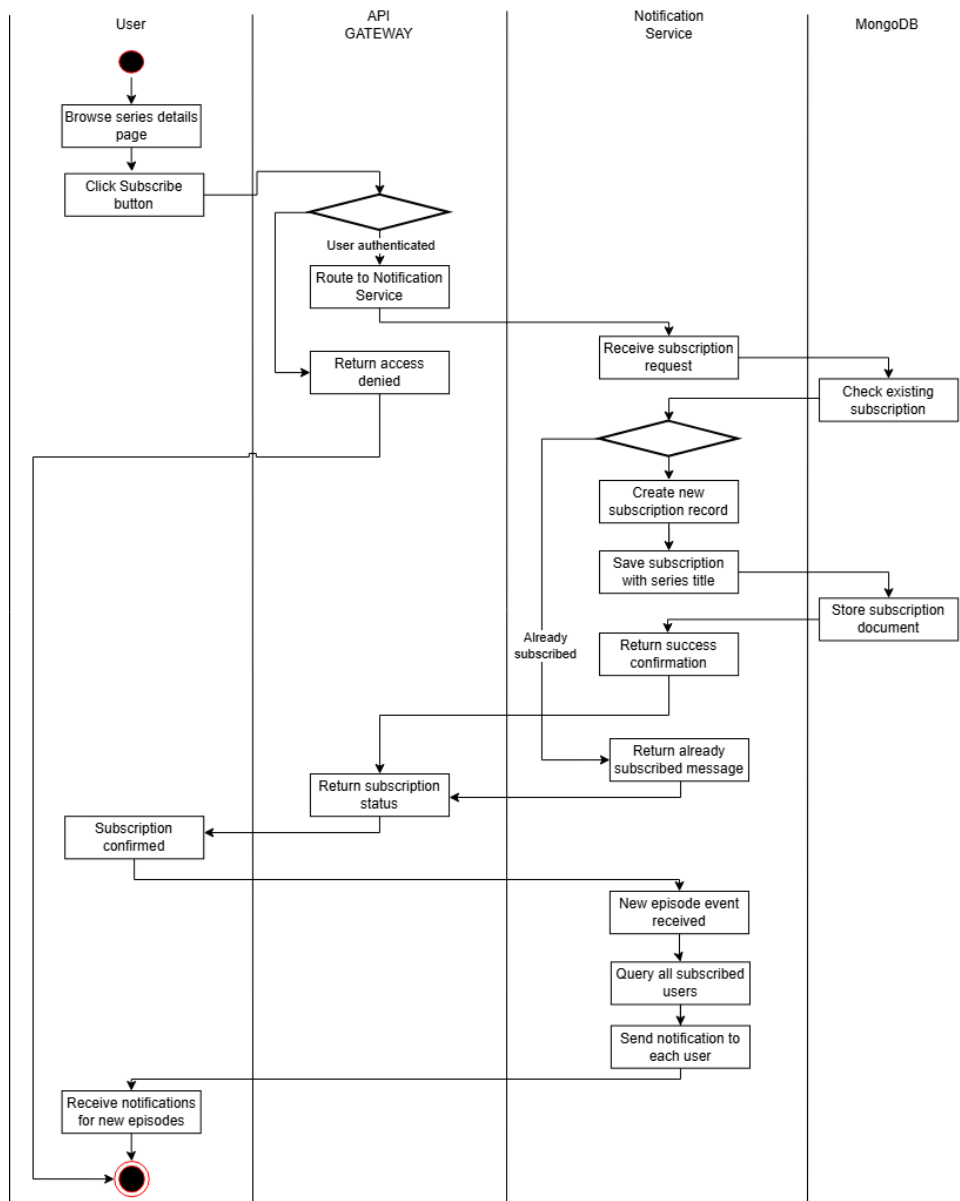


Hình 7: Sơ đồ hoạt động cho Usecase Admin tải video mới lên hệ thống và Xử lý Video

- Admin truy cập trang quản lý nội dung và chọn phim hoặc tập phim cần gắn video.
- Admin nhấn nút "Tải lên video" để bắt đầu quá trình upload.
- Ứng dụng gửi yêu cầu tạo phiên tải lên đến API Gateway, sau đó được chuyển tiếp đến Streaming Service.
- API Gateway xác thực quyền Admin từ JWT token:
 - Không có quyền Admin: Hệ thống trả về lỗi "Không có quyền truy cập".
 - Có quyền Admin: Hệ thống tiếp tục xử lý yêu cầu.
- Streaming Service tạo phiên tải lên (upload session) và gọi Cloudflare Stream API để lấy URL upload trực tiếp.
- Streaming Service lưu thông tin metadata video vào cơ sở dữ liệu MongoDB với trạng thái "Đang xử lý".

- Hệ thống trả về URL upload cho Admin, cho phép tải file video trực tiếp lên CDN mà không qua server backend.
- Admin chọn file video từ máy tính và bắt đầu tải lên Cloudflare Stream.
- Cloudflare Stream nhận file video và tự động thực hiện các bước xử lý:
 - Chuyển đổi video sang định dạng HLS/DASH để phát trực tuyến.
 - Tạo nhiều phiên bản độ phân giải: 480p, 720p, 1080p (và 4K nếu video gốc hỗ trợ).
 - Tạo hình thu nhỏ (thumbnail) tự động từ các khung hình trong video.
- Sau khi xử lý hoàn tất, Cloudflare Stream gửi thông báo webhook về Streaming Service với kết quả xử lý.
- Webhook Handler nhận thông báo và xác thực chữ ký bảo mật:
 - Chữ ký không hợp lệ: Từ chối yêu cầu và ghi log cảnh báo.
 - Chữ ký hợp lệ: Tiếp tục xử lý kết quả.
- Hệ thống kiểm tra trạng thái xử lý video:
 - Xử lý thành công: Cập nhật trạng thái video thành "Sẵn sàng phát" và lưu streaming ID.
 - Xử lý thất bại: Đánh dấu trạng thái "Lỗi" và ghi nhận lý do thất bại.
- Admin nhận thông báo hoàn tất và có thể xem trước video hoặc tiếp tục tải lên video cho nội dung khác.
- Video sẵn sàng phát trực tuyến cho người dùng với chất lượng tự động điều chỉnh theo tốc độ mạng.

3.6.7 Quản lý theo dõi phim

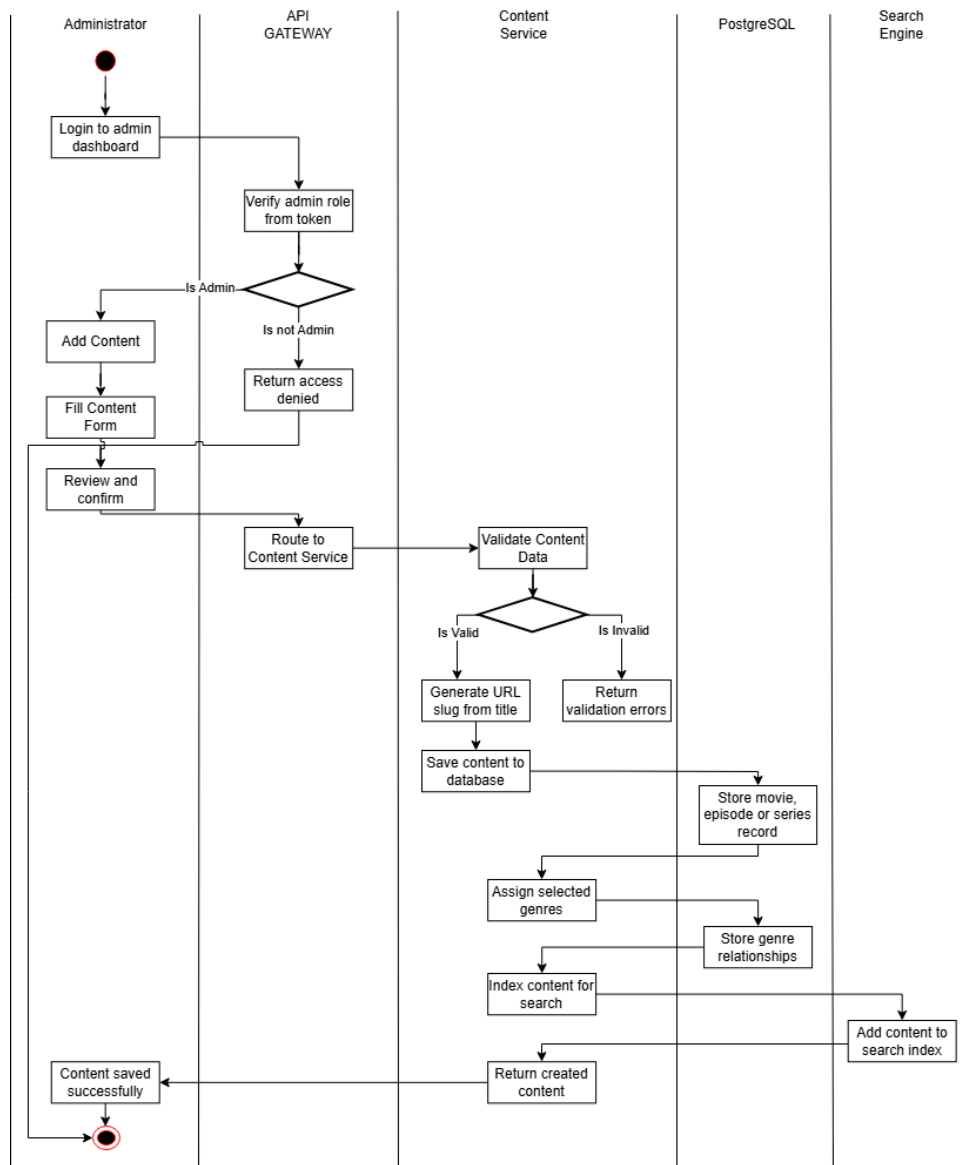


Hình 8: Sơ đồ hoạt động của Usecase Theo dõi phim

- Người dùng duyệt danh sách phim bộ hoặc tìm kiếm phim bộ yêu thích trên ứng dụng FlixZone.
- Người dùng nhấn vào phim bộ để xem trang chi tiết, bao gồm: thông tin mô tả, danh sách mùa, và nút "Theo dõi".
- Người dùng nhấn nút "Theo dõi" để đăng ký nhận thông báo khi có tập mới.
- Ứng dụng gửi yêu cầu đăng ký theo dõi đến API Gateway kèm theo thông tin: ID phim bộ, tên phim bộ, và token xác thực người dùng.
- API Gateway xác thực người dùng và chuyển tiếp yêu cầu đến Notification Service.
- Notification Service truy vấn cơ sở dữ liệu MongoDB để kiểm tra người dùng đã theo dõi phim bộ này chưa:
 - Đã theo dõi: Hệ thống trả về thông báo "Bạn đã theo dõi phim bộ này".
 - Chưa theo dõi: Hệ thống tiếp tục tạo đăng ký mới.

- Notification Service lưu thông tin đăng ký theo dõi vào MongoDB bao gồm: ID người dùng, ID phim bộ, tên phim bộ, và thời gian đăng ký.
- Hệ thống trả về xác nhận thành công cho người dùng và cập nhật giao diện nút "Theo dõi" thành "Đang theo dõi".
- Người dùng tiếp tục duyệt nội dung hoặc xem danh sách phim bộ đang theo dõi trong mục "Phim của tôi".
- Khi có tập mới phát hành (luồng xử lý sau):
 - Admin tạo tập mới trong hệ thống.
 - Content Service tạo sự kiện "new-episode" và đẩy vào hàng đợi tin nhắn.
 - Notification Service nhận sự kiện và truy vấn danh sách người dùng đã theo dõi phim bộ.
 - Notification Service gửi thông báo và gửi notification đến từng người dùng.
- Người dùng nhận thông báo trên website hoặc email: "Tập mới của [Tên phim] đã có! Xem ngay."
- Người dùng nhấn vào thông báo để chuyển trực tiếp đến trang xem tập mới.

3.6.8 Thiết lập phim mới

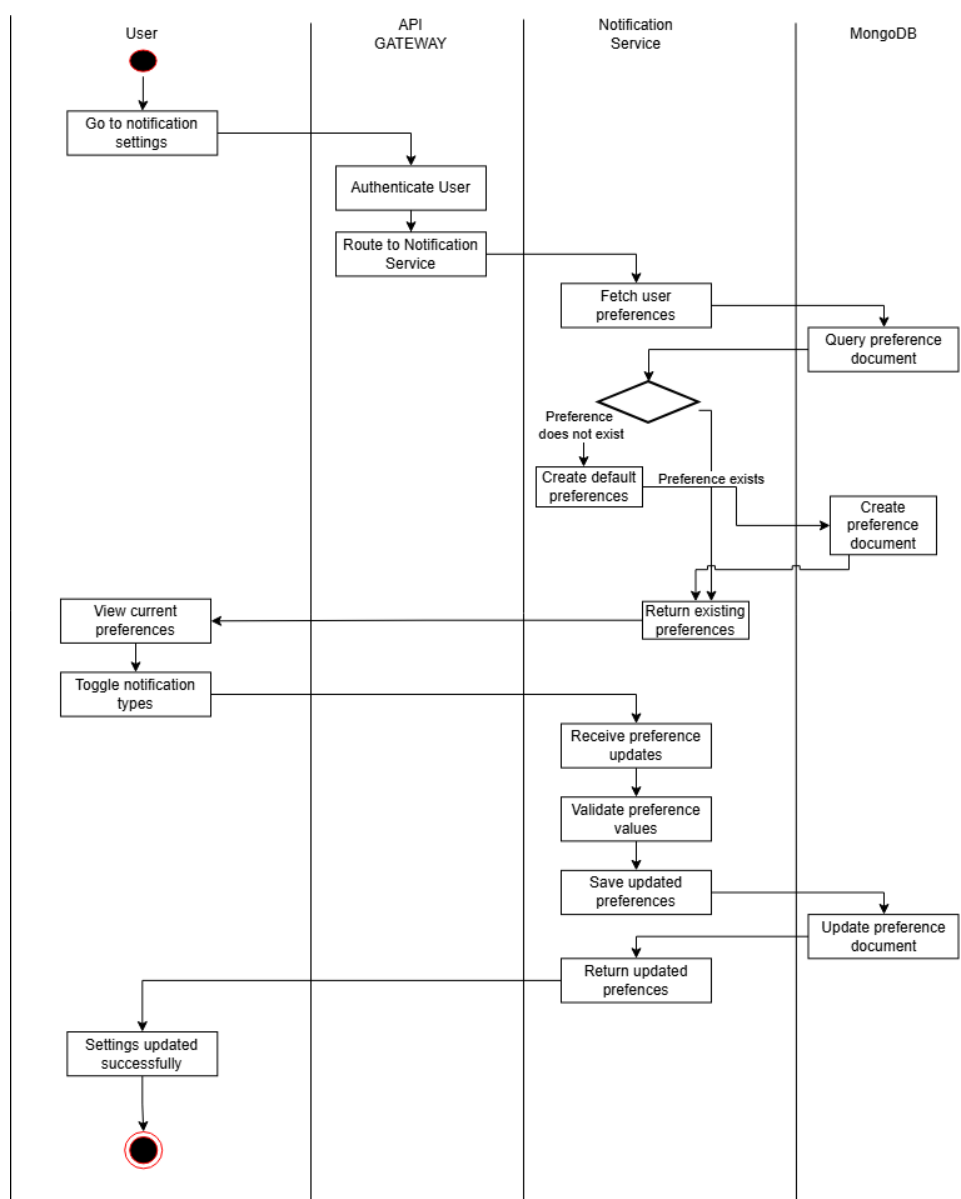


Hình 9: Sơ đồ hoạt động cho Usecase quản lý content của Admin

- Admin đăng nhập vào hệ thống quản trị FlixZone và truy cập trang quản lý nội dung.
- API Gateway xác thực JWT token và kiểm tra quyền Admin:
 - Không có quyền Admin: Hệ thống trả về lỗi "Không có quyền truy cập" và từ chối yêu cầu.
 - Có quyền Admin: Hệ thống cho phép truy cập trang quản lý nội dung.
- Admin chọn hành động muốn thực hiện: Thêm phim mới, Thêm phim bộ mới, hoặc Thêm tập phim.
- Hệ thống hiển thị form nhập liệu tương ứng với loại nội dung được chọn.
- Admin điền thông tin nội dung bao gồm: tiêu đề, mô tả, năm phát hành, thể loại, quốc gia, độ tuổi giới hạn, và các thông tin liên quan khác.
- Admin xem lại thông tin và nhấn nút "Lưu" để xác nhận tạo nội dung.
- Hệ thống gửi yêu cầu đến Content Service để xử lý:
 - Content Service kiểm tra tính hợp lệ của dữ liệu đầu vào.

- Dữ liệu không hợp lệ: Trả về danh sách lỗi validation và yêu cầu Admin sửa thông tin.
- Dữ liệu hợp lệ: Tiếp tục xử lý tạo nội dung.
- Content Service tự động tạo URL slug từ tiêu đề (ví dụ: "Avengers: Endgame" → "avengers-endgame") để sử dụng trong đường dẫn thân thiện.
- Content Service lưu thông tin nội dung vào cơ sở dữ liệu PostgreSQL.
- Content Service lưu mối quan hệ giữa nội dung và các thể loại đã chọn vào bảng liên kết.
- Content Service đẩy nội dung mới vào Elasticsearch để đánh chỉ mục tìm kiếm, cho phép nội dung xuất hiện trong kết quả tìm kiếm của người dùng.
- Hệ thống trả về xác nhận thành công kèm thông tin nội dung vừa tạo cho Admin.
- Admin có thể tiếp tục thêm nội dung khác hoặc tiến hành tải video lên cho nội dung vừa tạo.

3.6.9 Cài đặt thông báo



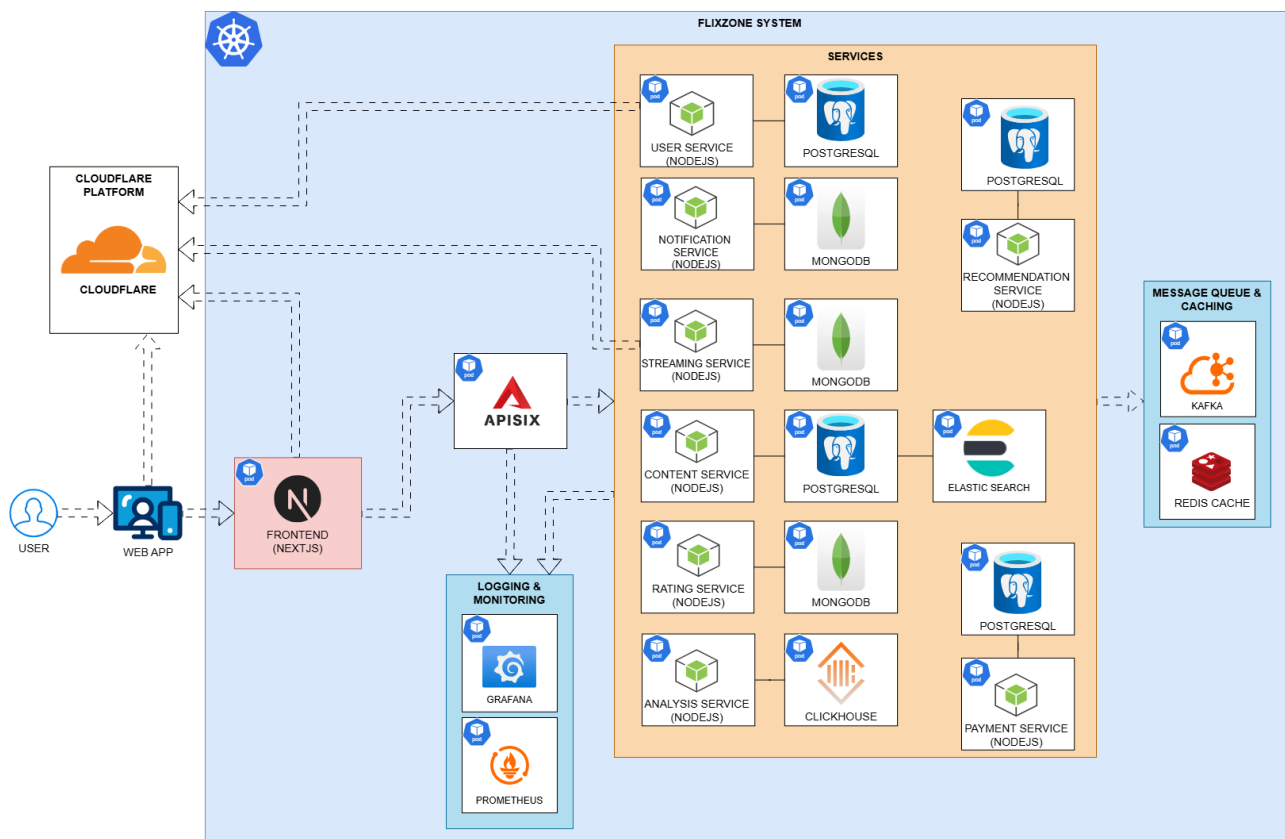
Hình 10: Sơ đồ hoạt động cho Usecase cài đặt thông báo

- Người dùng đăng nhập vào ứng dụng FlixZone và truy cập mục "Cài đặt" từ menu tài khoản.

- Người dùng chọn "Cài đặt thông báo" để quản lý các tùy chọn nhận thông báo.
- Ứng dụng gửi yêu cầu đến API Gateway để lấy cài đặt thông báo hiện tại của người dùng.
- API Gateway xác thực người dùng và chuyển tiếp yêu cầu đến Notification Service.
- Notification Service truy vấn cơ sở dữ liệu MongoDB để lấy cài đặt thông báo của người dùng:
 - Chưa có cài đặt: Hệ thống tạo cài đặt mặc định với tất cả loại thông báo được bật.
 - Đã có cài đặt: Hệ thống trả về cài đặt hiện tại.
- Ứng dụng hiển thị trang cài đặt thông báo với các tùy chọn:
 - Thông báo Email: Tập mới, Đề xuất phim, Cảnh báo tài khoản.
 - Thông báo trong ứng dụng: Tất cả loại thông báo.
- Người dùng bật/tắt các loại thông báo theo ý muốn bằng cách nhấn vào công tắc (toggle) tương ứng.
- Người dùng nhấn nút "Lưu" để xác nhận thay đổi cài đặt.
- Notification Service nhận yêu cầu cập nhật và xác thực giá trị cài đặt mới.
- Notification Service lưu cài đặt mới vào cơ sở dữ liệu MongoDB.
- Hệ thống trả về xác nhận cập nhật thành công cho người dùng.
- Từ thời điểm này, hệ thống sẽ chỉ gửi thông báo theo đúng cài đặt người dùng đã chọn.

4 Kiến trúc phần mềm

4.1 Tổng Quan Kiến Trúc



Hình 11: Sơ đồ tổng quan kiến trúc hệ thống FlixZone

Hệ thống FlixZone được xây dựng theo kiến trúc **Cloud-Native Microservices** kết hợp với mô hình **Hybrid Cloud**, tận dụng sức mạnh của Kubernetes cho việc điều phối dịch vụ và Cloudflare Platform cho việc xử lý hạ tầng video. Kiến trúc này hướng đến khả năng mở rộng linh hoạt, tối ưu chi phí băng thông và đảm bảo trải nghiệm streaming độ trễ thấp trên toàn cầu.

4.2 Công Nghệ Sử Dụng

Hệ thống tích hợp các công nghệ hiện đại nhằm giải quyết bài toán streaming quy mô lớn:

- **Kubernetes (K8s):** Đóng vai trò nền tảng điều phối, tự động mở rộng (Auto-scaling) và tự phục hồi (Self-healing) cho các container dịch vụ.
- **Node.js & TypeScript:** Được sử dụng để phát triển toàn bộ các vi dịch vụ (Microservices), tận dụng mô hình Non-blocking I/O để xử lý hàng ngàn kết nối đồng thời.
- **Cloudflare Platform:** Thay thế cho giải pháp lưu trữ và CDN truyền thống. Cloudflare Stream đảm nhận toàn bộ quy trình phức tạp gồm lưu trữ (Storage), chuyển mã (Transcoding) và phân phối nội dung (CDN) tại Edge.
- **Cloudflare Tunnel:** Thiết lập đường hầm bảo mật kết nối trực tiếp từ mạng Edge của Cloudflare vào cụm Kubernetes nội bộ mà không cần mở Port công khai.
- **Next.js:** Framework React hỗ trợ Server-Side Rendering (SSR), giúp tối ưu hóa SEO và tốc độ tải trang ban đầu cho Web App.

- **Apache Kafka:** Đóng vai trò trục xương sống (Backbone) cho giao tiếp bất đồng bộ theo kiến trúc Event-Driven, đảm bảo tính toàn vẹn dữ liệu giữa các dịch vụ.
- **API Gateway (APISIX):** Quản lý định tuyến, giới hạn tốc độ (Rate limiting) và xác thực sơ bộ trước khi request đi vào các dịch vụ backend.
- **ClickHouse:** Cơ sở dữ liệu phân tích (OLAP) cột dọc, cho phép truy vấn và tổng hợp hàng tỷ bản ghi log hành vi người dùng trong thời gian thực với tốc độ cực nhanh.
- **MongoDB:** Cơ sở dữ liệu NoSQL dùng để lưu trữ dữ liệu phi cấu trúc và có tần suất ghi cao như bình luận, đánh giá và lịch sử xem phim.
- **PostgreSQL:** Cơ sở dữ liệu quan hệ lưu trữ các dữ liệu có cấu trúc chặt chẽ như thông tin người dùng, metadata phim và giao dịch thanh toán.
- **Redis:** Hệ thống lưu trữ In-memory dùng để cache các truy vấn phổ biến và quản lý phiên làm việc (Session).
- **Elasticsearch:** Công cụ tìm kiếm Full-text search, hỗ trợ tìm kiếm phim theo tiêu đề, thể loại và diễn viên với độ chính xác cao.

4.3 Các Thành Phần Chính

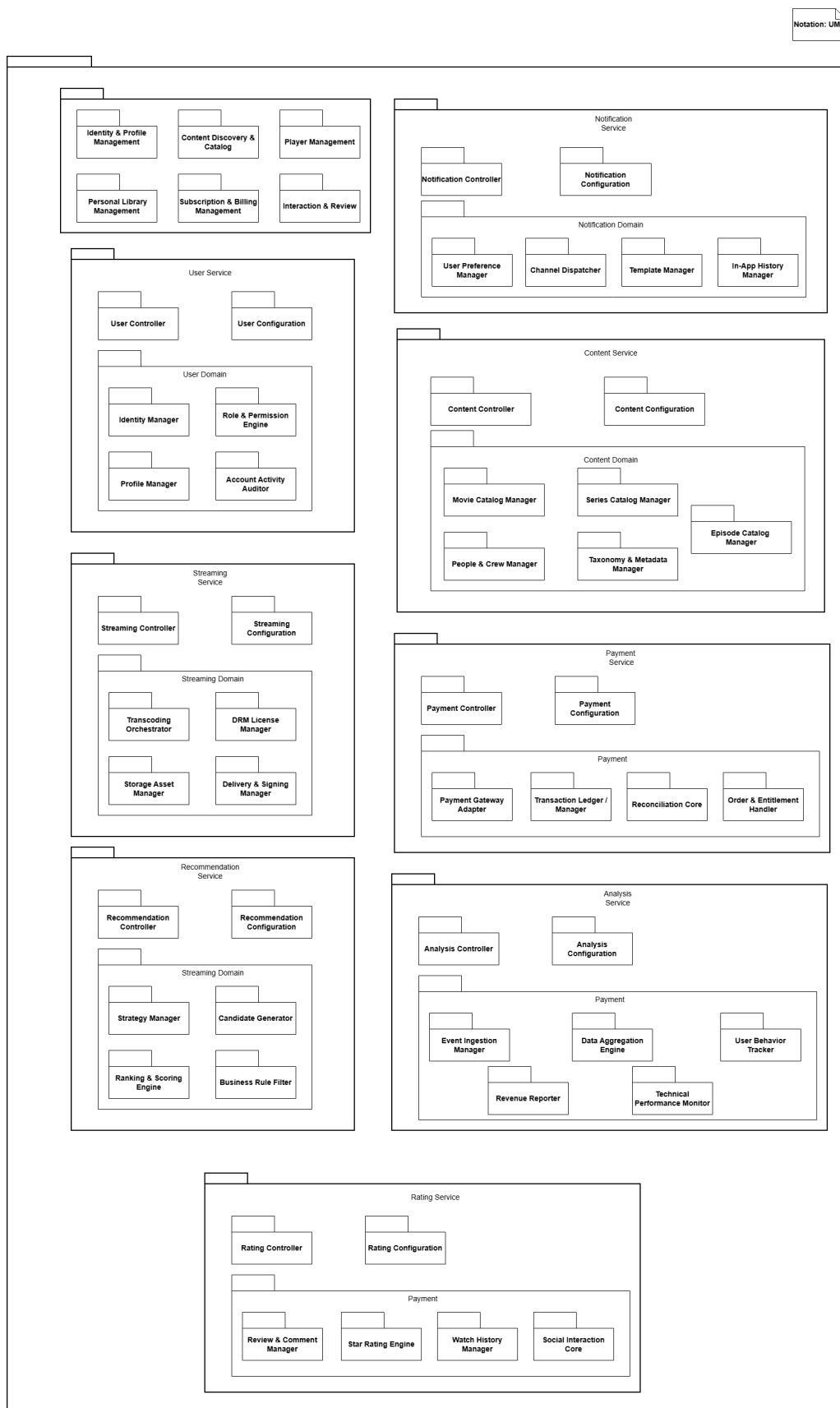
Hệ thống được chia nhỏ thành các vi dịch vụ độc lập, mỗi thành phần đảm nhận một nghiệp vụ cụ thể:

- **Streaming Service:** Đóng vai trò trung gian quản lý video. Thay vì xử lý file trực tiếp, dịch vụ này giao tiếp với Cloudflare để tạo URL upload, cấp phát token truy cập có chữ ký (Signed Token) để bảo vệ bản quyền (DRM) và ngăn chặn truy cập trái phép.
- **Content Service:** Quản lý siêu dữ liệu (metadata) của phim như tựa đề, mô tả, thông tin diễn viên và cấu trúc phân cấp (Series - Season - Episode). Dữ liệu được đồng bộ sang Elasticsearch để phục vụ tìm kiếm.
- **User Service:** Quản lý vòng đời tài khoản người dùng, xác thực (Authentication) và phân quyền (Authorization) dựa trên JWT.
- **Payment Service:** Xử lý nghiệp vụ thanh toán một lần (Lifetime Access). Dịch vụ này tích hợp với cổng thanh toán VNPay/Momo, xác thực chữ ký điện tử (IPN) và kích hoạt quyền xem phim cho người dùng.
- **Rating & Review Service:** Quản lý tương tác người dùng bao gồm bình luận, chấm điểm sao và lưu trữ tiến độ xem phim (Watch History). Sử dụng MongoDB để đáp ứng lượng dữ liệu lớn.
- **Analysis Service:** Thu thập và phân tích log sự kiện từ toàn bộ hệ thống. Dữ liệu được đẩy vào ClickHouse để tạo các báo cáo doanh thu và hành vi người dùng theo thời gian thực cho Admin Dashboard.
- **Notification Service:** Quản lý các mẫu thông báo (Templates) và điều phối việc gửi email hoặc thông báo in-app dựa trên cấu hình ưu tiên của người dùng.
- **API Gateway & Ingress:** Điểm tiếp nhận duy nhất cho mọi yêu cầu từ phía Client, định tuyến đến đúng Microservice xử lý thông qua mạng nội bộ Kubernetes.

Kiến trúc này đảm bảo sự tách biệt rõ ràng giữa logic nghiệp vụ (do Microservices xử lý) và các tác vụ nặng về hạ tầng (do Cloudflare và Kubernetes xử lý), giúp hệ thống FlixZone đạt được độ ổn định cao và dễ dàng mở rộng trong tương lai.

4.4 Góc Nhìn Phân Rã

4.4.1 Sơ Đồ Chính



Hình 12: Sơ đồ góc nhìn phân rã hệ thống FlixZone

4.4.2 Danh Mục Thành Phần

1. **Web App:** Là thành phần Frontend cung cấp giao diện tương tác chính cho người dùng cuối (Viewers) và quản trị viên (Admins). Module này không chứa logic nghiệp vụ phức tạp mà tập trung vào trải nghiệm người dùng, hiển thị dữ liệu và điều hướng.
 - (a) **Identity & Profile Management:** Quản lý các giao diện đăng nhập, đăng ký, quên mật khẩu và chuyển đổi giữa các hồ sơ người dùng (Profiles) khác nhau trong cùng một tài khoản.
 - (b) **Content Discovery & Catalog:** Hiển thị trang chủ, danh sách phim theo xu hướng, và các trang duyệt phim theo thể loại, giúp người dùng dễ dàng tìm kiếm nội dung mong muốn.
 - (c) **Player Management:** Giao diện trình phát video, bao gồm các nút điều khiển (Play/Pause), lựa chọn phụ đề, chất lượng video và xử lý các sự kiện playback.
 - (d) **Personal Library Management:** Quản lý giao diện "Danh sách của tôi" (My List) và hiển thị lịch sử các phim đang xem dở dang.
 - (e) **Subscription & Billing Management:** Cung cấp giao diện giới thiệu các gói dịch vụ trọn đời (Lifetime), so sánh quyền lợi và các bước thực hiện thanh toán.
 - (f) **Interaction & Review:** Cho phép người dùng tương tác với nội dung thông qua việc chấm điểm sao, viết bình luận hoặc báo cáo vi phạm.
2. **User Service:** Là dịch vụ chịu trách nhiệm quản lý định danh, xác thực và phân quyền người dùng trong toàn bộ hệ thống.
 - (a) **User Controller:** Cung cấp các API RESTful cho Web App và các dịch vụ nội bộ khác để thực hiện các tác vụ liên quan đến tài khoản và xác thực.
 - (b) **User Configuration:** Quản lý các tham số cấu hình như thời gian hết hạn Token, quy tắc mật khẩu và kết nối cơ sở dữ liệu người dùng.
 - (c) **User Domain:** Tập hợp các module nghiệp vụ cốt lõi của User Service:
 - **Identity Manager:** Xử lý logic đăng ký, đăng nhập và cấp phát JWT Token.
 - **Role & Permission Engine:** Quản lý vai trò (Admin/User) và kiểm tra quyền truy cập tài nguyên.
 - **Profile Manager:** Quản lý thông tin cá nhân và các profile phụ.
 - **Account Activity Auditor:** Ghi nhận lịch sử đăng nhập và phát hiện các hoạt động bất thường để bảo mật tài khoản.
3. **Content Service:** Dịch vụ quản lý toàn bộ siêu dữ liệu (metadata) và cấu trúc phân cấp của kho phim.
 - (a) **Content Controller:** Cung cấp API truy xuất thông tin chi tiết phim, danh sách tập và thông tin diễn viên.
 - (b) **Content Configuration:** Cấu hình kết nối cơ sở dữ liệu metadata và tích hợp với Search Engine.
 - (c) **Content Domain:**
 - **Movie Catalog Manager:** Quản lý thông tin các phim lẻ (Movies).
 - **Series Catalog Manager:** Quản lý cấu trúc phim bộ (Series) và các mùa (Seasons).
 - **Episode Catalog Manager:** Quản lý thông tin chi tiết từng tập phim.
 - **People & Crew Manager:** Quản lý hồ sơ diễn viên, đạo diễn và biên kịch.
 - **Taxonomy & Metadata Manager:** Quản lý hệ thống phân loại gồm thể loại, thẻ (tags) và quốc gia.

4. **Streaming Service:** Dịch vụ đóng vai trò "trái tim" kỹ thuật, xử lý việc tải lên, chuyển mã và bảo vệ bản quyền video thông qua tích hợp với Cloudflare.
- (a) **Streaming Controller:** Cung cấp API để Admin lấy link upload và User lấy token xem phim.
 - (b) **Streaming Configuration:** Chứa thông tin xác thực với Cloudflare API (Account ID, API Token) và khóa ký (Signing Keys).
 - (c) **Streaming Domain:**
 - **Transcoding Orchestrator:** Điều phối quy trình chuyển mã video và xử lý Webhook thông báo trạng thái từ Cloudflare.
 - **DRM License Manager:** Quản lý việc cấp phép bản quyền kỹ thuật số, đảm bảo chỉ người dùng hợp lệ mới có thể giải mã video.
 - **Storage Asset Manager:** Quản lý liên kết giữa ID phim trong hệ thống và ID video trên Cloudflare Stream.
 - **Delivery & Signing Manager:** Tạo ra các URL và Token có chữ ký số (Signed Tokens) để bảo mật luồng phát video.
5. **Payment Service:** Xử lý các giao dịch tài chính, tích hợp cổng thanh toán và kích hoạt quyền lợi trọn đời cho người dùng.
- (a) **Payment Controller:** Tiếp nhận yêu cầu thanh toán từ Client và nhận tín hiệu IPN (Instant Payment Notification) từ cổng thanh toán.
 - (b) **Payment Configuration:** Lưu trữ mã đối tác (Partner Code), khóa bí mật (Secret Key) của VNPAY/Momo.
 - (c) **Payment Domain:**
 - **Payment Gateway Adapter:** Module chuyển đổi giao tiếp đa hình, giúp hệ thống dễ dàng tích hợp nhiều cổng thanh toán khác nhau.
 - **Transaction Ledger:** Sổ cái ghi nhận mọi biến động trạng thái của giao dịch (Pending, Success, Failed).
 - **Reconciliation Core:** Module đối soát tự động để xử lý các giao dịch bị treo hoặc lỗi mạng.
 - **Order & Entitlement Handler:** Xử lý logic nghiệp vụ sau khi thanh toán thành công (kích hoạt gói Premium cho User).
6. **Notification Service:** Đóng vai trò là trung tâm quản lý và điều phối tất cả thông báo trong hệ thống, đảm bảo thông tin đến được người dùng qua đúng kênh và đúng thời điểm.
- (a) **Notification Controller:** Cung cấp các API RESTful để tiếp nhận yêu cầu gửi thông báo từ các dịch vụ khác (ví dụ: "Thanh toán thành công", "Có tập phim mới") và API cho phép ứng dụng Client truy xuất lịch sử thông báo.
 - (b) **Notification Configuration:** Quản lý các cấu hình kết nối với nhà cung cấp dịch vụ gửi tin (Email SMTP, Push Notification Service), cũng như các chính sách thử lại (Retry Policy) khi gửi thất bại.
 - (c) **Notification Domain:** Tập hợp các module xử lý logic nghiệp vụ gửi tin:
 - **User Preference Manager:** Kiểm tra cài đặt riêng tư của người dùng (ví dụ: tắt email quảng cáo, chỉ nhận thông báo bảo mật) trước khi thực hiện gửi tin.
 - **Channel Dispatcher:** Bộ định tuyến thông minh, quyết định xem thông báo sẽ được gửi qua Email, SMS hay In-App Notification dựa trên mức độ ưu tiên và loại sự kiện.

- **Template Manager:** Quản lý và render nội dung thông báo từ các mẫu (Templates) HTML/-Text có sẵn, điền dữ liệu động (tên user, tên phim) vào mẫu.
 - **In-App History Manager:** Lưu trữ bền vững các thông báo để hiển thị trong mục "Hộp thư" của ứng dụng và quản lý trạng thái đã đọc/chưa đọc.
7. **Recommendation Service:** Sử dụng các thuật toán học máy và phân tích dữ liệu để cá nhân hóa trải nghiệm, đề xuất các nội dung phù hợp nhất cho từng người dùng.
- (a) **Recommendation Controller:** Cung cấp các API để lấy danh sách phim gợi ý cho các ngữ cảnh khác nhau: Trang chủ (Home Feed), Phim tương tự (Related Movies), hoặc Gợi ý theo tâm trạng.
 - (b) **Recommendation Configuration:** Lưu trữ các tham số điều chỉnh thuật toán (Hyperparameters), trọng số (Weights) cho việc xếp hạng, và cấu hình cho các thử nghiệm A/B Testing.
 - (c) **Recommendation Domain:**
 - **Strategy Manager:** Lựa chọn chiến lược gợi ý phù hợp (ví dụ: Collaborative Filtering cho người dùng cũ, Popularity-based cho người dùng mới).
 - **Candidate Generator:** Thực hiện truy vấn nhanh để lọc ra tập hợp vài trăm ứng viên tiềm năng từ hàng ngàn bộ phim trong kho dữ liệu.
 - **Ranking & Scoring Engine:** Tính toán điểm số chi tiết cho từng phim ứng viên dựa trên xác suất người dùng sẽ bấm xem, từ đó sắp xếp thứ tự ưu tiên.
 - **Business Rule Filter:** Lọc cuối cùng để áp dụng các quy tắc nghiệp vụ (ví dụ: loại bỏ phim đã xem, lọc nội dung không phù hợp lứa tuổi) trước khi trả về kết quả.
8. **Rating Service:** Quản lý toàn bộ tương tác, phản hồi của người dùng đối với nội dung và lưu trữ lịch sử hành vi xem phim.
- (a) **Rating Controller:** API cho phép người dùng gửi đánh giá (Star Rating), viết bình luận (Review), và API để Player báo cáo tiến độ xem phim (Watch Progress) định kỳ.
 - (b) **Rating Configuration:** Cấu hình các quy tắc kiểm duyệt nội dung (Profanity Filters), giới hạn độ dài bình luận và các quy tắc làm tròn điểm số đánh giá.
 - (c) **Rating Domain:**
 - **Review & Comment Manager:** Xử lý các thao tác thêm/sửa/xóa bình luận, tích hợp logic phát hiện nội dung rác (Spam) hoặc tiết lộ nội dung (Spoiler).
 - **Star Rating Engine:** Tính toán điểm đánh giá trung bình cho từng tập phim và tổng hợp lên cấp độ Mùa (Season) và Phim bộ (Series) theo thời gian thực hoặc định kỳ.
 - **Watch History Manager:** Ghi nhận mốc thời gian (timestamp) người dùng dừng xem để hỗ trợ tính năng "Xem tiếp" (Resume) và đánh dấu trạng thái "Đã xem xong".
 - **Social Interaction Core:** Quản lý các tương tác xã hội như lượt thích (Like), phản hồi (Reply) trên các bình luận của người khác.
9. **Analysis Service:** Hệ thống giám sát, thu thập dữ liệu lớn và tạo báo cáo phục vụ cho việc ra quyết định kinh doanh và vận hành kỹ thuật.
- (a) **Analysis Controller:** Cung cấp API nhận luồng sự kiện (Event Stream) từ phía Client/Backend và các API truy xuất báo cáo thống kê dành cho Dashboard quản trị (Admin Panel).
 - (b) **Analysis Configuration:** Quản lý cấu hình kết nối tới kho dữ liệu phân tích (ClickHouse/Data Warehouse), chu kỳ xoay vòng dữ liệu (Data Retention) và các định nghĩa chỉ số (Metrics definitions).
 - (c) **Analysis Domain:**

- **Event Ingestion Manager:** Đầu mối tiếp nhận hàng loạt sự kiện (logs) với tốc độ cao, thường sử dụng cơ chế đệm (Buffering) trước khi đẩy vào hàng đợi xử lý.
- **Data Aggregation Engine:** Thực hiện các phép toán tổng hợp dữ liệu thô thành các chỉ số có ý nghĩa (ví dụ: Tổng lượt xem theo giờ, Doanh thu theo ngày).
- **Revenue Reporter:** Chuyên trách tính toán và tạo các báo cáo tài chính, so sánh tăng trưởng doanh thu và tỷ lệ chuyển đổi người dùng.
- **User Behavior Tracker:** Phân tích chuỗi hành vi người dùng (User Journey), xác định điểm rời bỏ (Drop-off points) để tối ưu hóa trải nghiệm sản phẩm.
- **Technical Performance Monitor:** Giám sát các chỉ số kỹ thuật như tỷ lệ lỗi phát sinh, thời gian phản hồi API, và chất lượng streaming (QoE).

4.4.3 Hướng Dẫn Thay Đổi

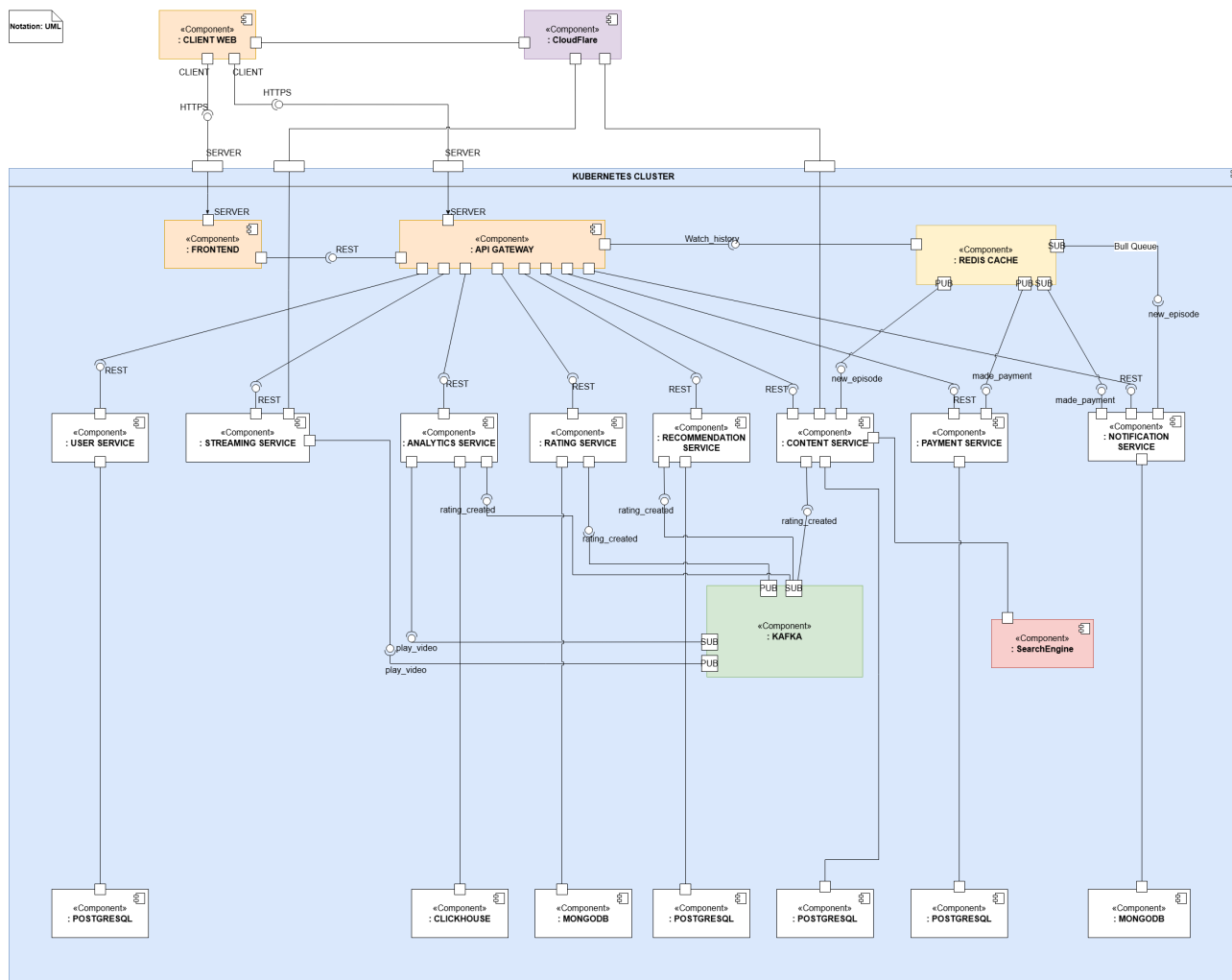
- **Mở rộng cổng thanh toán:** Chỉ cần cài đặt thêm implementation mới trong module *Payment Gateway Adapter* của Payment Service mà không ảnh hưởng đến logic xử lý đơn hàng cốt lõi.
- **Thay đổi thuật toán gợi ý:** Các logic tính toán nằm gói gọn trong *Ranking & Scoring Engine* của Recommendation Service, có thể cập nhật hoặc thay thế mô hình AI mới độc lập.
- **Cấu hình hệ thống:** Các tham số như giá gói cước, giới hạn thiết bị hay API Key của bên thứ 3 được quản lý trong các module *Configuration* riêng biệt của từng dịch vụ, cho phép thay đổi nóng (hot-reload) mà không cần deploy lại toàn bộ code.

4.4.4 Lý Do Thiết Kế

- Hệ thống áp dụng triệt để mô hình **Domain-Driven Design (DDD)**, chia nhỏ hệ thống thành các Bounded Contexts (User, Content, Streaming, Payment...). Mỗi dịch vụ sở hữu trọn vẹn dữ liệu và logic của mình, giúp giảm sự phụ thuộc chéo và dễ dàng bảo trì.
- Việc tách biệt **Streaming Service** xử lý kỹ thuật video khỏi **Content Service** quản lý thông tin phim giúp tối ưu hóa hiệu năng: Content Service nhẹ và nhanh để phục vụ web, trong khi Streaming Service xử lý các tác vụ nặng bất đồng bộ.
- Sử dụng pattern **Controller - Service/Domain - Repository** trong từng Microservice giúp code mạch lạc, dễ kiểm thử (Unit Test) và dễ dàng thay thế các công nghệ lưu trữ hoặc giao tiếp bên dưới khi cần thiết.

4.5 Góc Nhìn Thành Phần - Kết Nối

4.5.1 Sơ đồ chính



Hình 13: Sơ đồ thành phần - kết nối

4.5.2 Danh Sách Thành Phần

- **CLIENT WEB (Web Application):**

- Là ứng dụng web dành cho người dùng cuối, cung cấp giao diện xem video, đánh giá, bình luận và quản lý nội dung cá nhân.
- Thực hiện xác thực người dùng bằng cách giao tiếp với **API Gateway** thông qua giao thức **HTTPS**.
- Gửi yêu cầu đồng bộ đến **API Gateway** bằng **REST** để lấy dữ liệu như danh sách video, đề xuất nội dung và thông tin người dùng.
- Tiếp nhận nội dung tĩnh và tài nguyên từ **CloudFlare CDN** để xem phim và tăng tốc độ load phim.

- **FRONTEND (Next.js):**

- Thực hiện chức năng **Server-Side Rendering (SSR)** để tối ưu **SEO** và hiệu suất tải trang ban đầu.
- Nhận yêu cầu từ **CLIENT WEB** và xử lý trả về trang **HTML** hoàn chỉnh.
- Lấy dữ liệu cần thiết từ **API Gateway** thông qua **REST** trước khi thực hiện render trang.

- Cung cấp nội dung tĩnh thông qua HTTPS và tận dụng **CloudFlare CDN** để phân phối toàn cầu.

- **CloudFlare (CDN):**

- Đóng vai trò là mạng phân phối nội dung toàn cầu, cache và phân phối tài nguyên tĩnh từ **FRONTEND**.
- Giảm độ trễ cho người dùng ở các vị trí địa lý khác nhau bằng cách phục vụ nội dung từ edge server gần nhất.
- Cung cấp bảo mật bổ sung thông qua DDoS protection và SSL/TLS encryption.

- **API GATEWAY (Spring Cloud Gateway):**

- Đóng vai trò là điểm truy cập trung tâm của hệ thống, điều phối tất cả yêu cầu từ **CLIENT WEB** và **FRONTEND**.
- Tiếp nhận yêu cầu REST từ các ứng dụng khách và định tuyến đến các microservice phù hợp.
- Thực thi cơ chế xác thực và kiểm tra quyền truy cập trước khi chuyển tiếp yêu cầu.
- Định tuyến yêu cầu đến các microservice như **USER SERVICE**, **STREAMING SERVICE**, **ANALYTICS SERVICE**, **RATING SERVICE**, **RECOMMENDATION SERVICE**, **CONTENT SERVICE**, **PAYMENT SERVICE** và **NOTIFICATION SERVICE**.
- Theo dõi và ghi log các yêu cầu thông qua kết nối với **REDIS CACHE** để monitoring và debugging.

- **USER SERVICE:**

- Quản lý thông tin người dùng bao gồm đăng ký, xác thực, cập nhật hồ sơ và quản lý phiên làm việc.
- Lưu trữ dữ liệu người dùng trong **POSTGRESQL** database.
- Giao tiếp với **API Gateway** thông qua REST để nhận và xử lý yêu cầu từ client.

- **STREAMING SERVICE:**

- Xử lý việc phát video streaming cho người dùng, bao gồm adaptive bitrate streaming và quản lý session.
- Lấy metadata video từ **MONGODB** và phối hợp với **CONTENT SERVICE** để truy xuất nội dung thực tế.
- Phát các sự kiện play_video lên **KAFKA** để tracking và analytics.
- Sử dụng **CLICKHOUSE** để lưu trữ log streaming và phân tích hành vi xem.

- **ANALYTICS SERVICE:**

- Thu thập và phân tích dữ liệu hành vi người dùng từ các sự kiện được gửi qua **KAFKA**.
- Lưu trữ dữ liệu phân tích trong **CLICKHOUSE** để hỗ trợ truy vấn nhanh và báo cáo thời gian thực.
- Phát sự kiện rating_created lên **KAFKA** khi có đánh giá mới từ người dùng.
- Cung cấp insights và metrics cho các dịch vụ khác thông qua REST API.

- **RATING SERVICE (Spring Boot):**

- Quản lý hệ thống đánh giá và xếp hạng nội dung từ người dùng.
- Lưu trữ dữ liệu rating trong **MONGODB**.
- Tiêu thụ các sự kiện rating_created từ **KAFKA** để cập nhật điểm đánh giá tổng hợp.

- Phát sự kiện `rating_created` để thông báo cho các dịch vụ phụ thuộc như **RECOMMENDATION SERVICE**, **CONTENT SERVICE**, **ANALYTICS SERVICE**.
- **RECOMMENDATION SERVICE (Python/FastAPI):**
 - Xây dựng và cung cấp hệ thống đề xuất nội dung dựa trên machine learning.
 - Tiêu thụ các sự kiện `rating_created` từ **KAFKA** để cập nhật mô hình đề xuất.
 - Lưu trữ dữ liệu mô hình và kết quả đề xuất trong **POSTGRESQL**.
 - Cung cấp API để **API Gateway** truy vấn danh sách nội dung được đề xuất cho người dùng.
- **CONTENT SERVICE:**
 - Quản lý metadata và catalog của tất cả nội dung video trên nền tảng.
 - Lưu trữ thông tin nội dung trong **POSTGRESQL**.
 - Tiêu thụ sự kiện `rating_created` từ **KAFKA** để cập nhật `avg_rating`.
 - Tích hợp với **SearchEngine** như **Elastic Search** để hỗ trợ tìm kiếm nội dung.
 - Phát sự kiện `episode_created` lên **Redis** khi có tập phim mới.
- **PAYMENT SERVICE:**
 - Xử lý các giao dịch thanh toán, subscription và billing cho người dùng.
 - Lưu trữ thông tin giao dịch và subscription trong **POSTGRESQL**.
 - Phát các sự kiện `made_payment` lên **REDIS BULL** để theo dõi và đối chiếu.
 - Tích hợp với các payment gateway bên ngoài để xử lý thanh toán thực tế.
- **NOTIFICATION SERVICE:**
 - Quản lý và gửi thông báo đến người dùng qua nhiều kênh (email, push notification, in-app).
 - Tiêu thụ các sự kiện từ **KAFKA** hay **REDIS BULL** như `new_episode`, `made_payment` để kích hoạt thông báo tương ứng.
 - Lưu trữ lịch sử thông báo trong **MONGODB**.
 - Quản lý template thông báo và lưu trữ cách tùy chỉnh thông báo của mỗi người dùng.
- **KAFKA (Apache Kafka):**
 - Đóng vai trò là message broker trung tâm, xử lý event streaming giữa các microservice.
 - Nhận và phân phối các sự kiện như `play_video`, `rating_created`.
 - Đảm bảo giao tiếp bất đồng bộ, tách rời giữa các service và khả năng xử lý khối lượng lớn events.
 - Hỗ trợ pattern publish-subscribe để nhiều consumer có thể xử lý cùng một event stream.
- **REDIS CACHE (Redis):**
 - Lưu trữ dữ liệu cache để giảm tải cho database và cải thiện tốc độ phản hồi.
 - Được sử dụng bởi **API Gateway** để cache routing rules và authentication tokens.
 - Lưu trữ session data và frequently accessed data như trending videos.
 - Hỗ trợ **PAYMENT SERVICE** và **CONTENT SERVICE** trong việc Pub/Sub với **NOTIFICATION SERVICE** về các sự kiện tương ứng.
- **SearchEngine (Elasticsearch):**

- Cung cấp khả năng tìm kiếm full-text nhanh và mạnh mẽ cho nội dung video.
- Được sử dụng bởi **CONTENT SERVICE** để indexing và searching.
- Hỗ trợ tìm kiếm theo nhiều tiêu chí như tiêu đề, mô tả, tags và metadata.

- **KUBERNETES CLUSTER:**

- Quản lý việc triển khai, orchestration và scaling cho tất cả microservices.
- Cung cấp service discovery, load balancing và self-healing capabilities.
- Duy trì tính ổn định và khả năng mở rộng của toàn bộ hệ thống.
- Quản lý networking giữa các pods và services trong cluster.

- **Các Cơ Sở Dữ Liệu:**

- **POSTGRESQL:** Lưu trữ dữ liệu quan hệ cho **USER SERVICE**, **RECOMMENDATION SERVICE**, **CONTENT SERVICE** và **PAYMENT SERVICE**.
- **MONGODB:** Lưu trữ dữ liệu phi cấu trúc cho **STREAMING SERVICE NOTIFICATION SERVICE** và **RATING SERVICE**, phù hợp với khả năng lưu trữ, truy xuất thông báo, metadata phim và rating data.
- **CLICKHOUSE:** Lưu trữ dữ liệu analytics và time-series cho **ANALYTICS SERVICE**, tối ưu cho OLAP queries.

4.5.3 Hướng Dẫn Thay Đổi

- Mọi chức năng mới phải giao tiếp thông qua **API Gateway**, sử dụng REST protocol; không được kết nối trực tiếp đến các microservice nội bộ để đảm bảo tính nhất quán và bảo mật.
- Các thay đổi liên quan đến xác thực hoặc quyền truy cập phải được cấu hình tại **API Gateway**, đảm bảo các microservice chỉ xử lý business logic và không quản lý authentication logic.
- Khi mở rộng hoặc bổ sung microservice mới, API endpoint phải được đăng ký tại **API Gateway** với routing rules phù hợp.
- Nếu microservice mới cần giao tiếp bất đồng bộ, phải cấu hình topic và consumer group trong **KAFKA** để đảm bảo event flow chính xác.
- Khi thêm loại sự kiện mới vào **KAFKA**, cần document event schema và đảm bảo backward compatibility với các consumer hiện tại.
- Mọi thay đổi về database schema phải được thực hiện thông qua migration scripts và được test kỹ lưỡng trước khi deploy production.
- Khi thêm tính năng caching mới, cần định nghĩa rõ TTL và invalidation strategy trong **REDIS CACHE**.

4.5.4 Lý Do Thiết Kế

- **Sử dụng API Gateway**

- Cung cấp một điểm truy cập thống nhất cho toàn bộ ứng dụng khách, giúp đơn giản hóa việc quản lý giao tiếp giữa client và backend services.
- Tách biệt hoàn toàn các microservice nội bộ khỏi ứng dụng khách, giảm nguy cơ lộ cấu trúc nội bộ hệ thống và tăng cường bảo mật.
- Tập trung xử lý cross-cutting concerns như authentication, authorization, rate limiting và logging tại một điểm duy nhất.

- Hỗ trợ versioning API và backward compatibility khi cần thay đổi hoặc nâng cấp services.
- Tạo điều kiện dễ dàng mở rộng hệ thống và tích hợp load balancing khi lưu lượng truy cập tăng cao.

- **Sử dụng Event-Driven Architecture với Kafka**

- Cho phép các microservice giao tiếp bất đồng bộ, giảm coupling và tăng khả năng mở rộng độc lập của từng service.
- Đảm bảo reliability thông qua message persistence và replay capability khi service consumer gặp sự cố.
- Hỗ trợ real-time data streaming cho analytics và monitoring, cho phép phản ứng nhanh với các sự kiện hệ thống.
- Tạo audit trail đầy đủ của các sự kiện trong hệ thống, hỗ trợ debugging và compliance requirements.
- Cho phép xây dựng event sourcing pattern cho các business domain phức tạp.

- **Sử dụng Đa Dạng Database (Polyglot Persistence)**

- **PostgreSQL** được chọn cho dữ liệu quan hệ cần ACID guarantees như user data, payment transactions và subscription management.
- **MongoDB** phù hợp với dữ liệu semi-structured như video metadata và ratings, cung cấp flexibility trong schema evolution.
- **ClickHouse** tối ưu cho analytics workload với khả năng xử lý large-scale time-series data và OLAP queries hiệu quả.
- Mỗi database được chọn dựa trên đặc tính của workload, đảm bảo performance tối ưu cho từng use case cụ thể.

- **Sử dụng Redis Cache**

- Giảm đáng kể độ trễ cho các truy vấn thường xuyên bằng cách cache dữ liệu ở in-memory layer.
- Giảm tải cho database backend, cho phép hệ thống xử lý lưu lượng cao hơn với cùng infrastructure.
- Hỗ trợ session management và distributed locking cho các tác vụ cần coordination giữa services.
- Cung cấp TTL-based expiration để đảm bảo data freshness và tự động cleanup.

- **Sử dụng CloudFlare CDN**

- Cải thiện đáng kể user experience bằng cách phục vụ static content từ edge locations gần người dùng nhất.
- Giảm bandwidth cost và load cho origin servers thông qua aggressive caching.
- Cung cấp built-in DDoS protection và Web Application Firewall để bảo vệ ứng dụng.
- Hỗ trợ HTTP/3 và modern protocols để tối ưu performance.

- **Sử dụng Microservices Architecture**

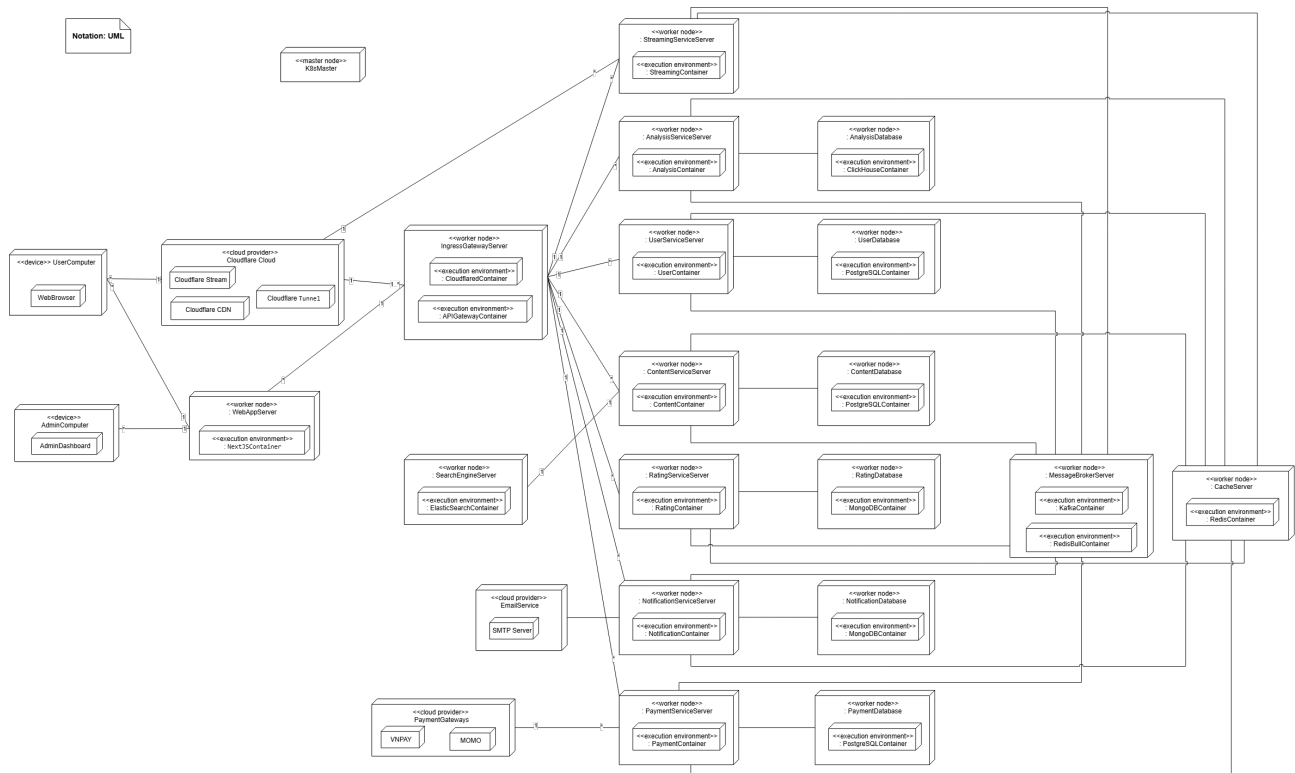
- Cho phép các team phát triển độc lập trên các bounded context khác nhau, tăng velocity và reduce dependencies.
- Hỗ trợ scaling độc lập cho từng service dựa trên workload thực tế, tối ưu resource utilization.
- Tăng resilience thông qua isolation - lỗi ở một service không làm sập toàn bộ hệ thống.
- Cho phép technology diversity - mỗi service có thể chọn technology stack phù hợp nhất với yêu cầu của nó.

• Sử dụng Kubernetes

- Tự động hóa deployment, scaling và management của containerized applications.
- Cung cấp self-healing capabilities thông qua automatic restart và rescheduling của failed pods.
- Hỗ trợ rolling updates và rollbacks, giảm downtime khi deploy phiên bản mới.
- Cung cấp service discovery và load balancing tự động giữa các pod instances.

4.6 Góc Nhìn Triển Khai

4.6.1 Sơ Đồ Chính



Hình 14: Sơ đồ góc nhìn triển khai hệ thống Streaming

4.6.2 Danh Sách Thành Phần

Hệ thống được triển khai trên nền tảng Kubernetes Cluster, kết hợp với các dịch vụ Cloud bên ngoài để tối ưu hóa hiệu năng và khả năng lưu trữ.

- **KubernetesMaster** Điều phối toàn bộ cụm cluster. Chịu trách nhiệm quản lý trạng thái, lập lịch (scheduling) cho các Pods và giám sát sức khỏe của các Worker Nodes trong hệ thống.
- **Cloudflare Cloud (Cloud Provider)**
 - **Cloudflare Stream & CDN:** Cung cấp mạng phân phối nội dung (CDN) và dịch vụ streaming video để giảm tải băng thông cho hệ thống gốc.
 - **Cloudflare Tunnel:** Tạo đường hầm bảo mật kết nối từ môi trường Internet vào Ingress Gateway mà không cần mở port công khai.
- **IngressGatewayServer** Đóng vai trò là điểm nhập duy nhất cho traffic từ bên ngoài vào cluster.
- **CloudflaredContainer:** Agent kết nối với Cloudflare Tunnel.

- **APIGatewayContainer:** Định tuyến yêu cầu đến các microservice đích, thực hiện xác thực cơ bản và cân bằng tải.
- **WebAppServer (NextJSContainer)**
 - Triển khai ứng dụng Next.js phục vụ giao diện người dùng (WebBrowser) và giao diện quản trị (AdminDashboard).
 - Hỗ trợ Server-Side Rendering (SSR) để tối ưu hóa SEO và trải nghiệm người dùng.
- **StreamingServiceServer (StreamingContainer)** Xử lý logic nghiệp vụ liên quan đến luồng phát video, cấp phát manifest file (HLS/DASH) và tương tác với Cloudflare Stream.
- **AnalysisServiceServer**
 - **AnalysisContainer:** Thu thập dữ liệu hành vi người dùng và log hệ thống.
 - **AnalysisDatabase (ClickHouseContainer):** Cơ sở dữ liệu dạng cột (Columnar DB) chuyên dụng để lưu trữ và truy vấn khối lượng lớn dữ liệu phân tích (OLAP) với tốc độ cao.
- **UserServiceServer**
 - **UserContainer:** Quản lý thông tin tài khoản, hồ sơ cá nhân và xác thực người dùng.
 - **UserDatabase (PostgreSQLContainer):** Lưu trữ dữ liệu quan hệ về người dùng.
- **ContentServiceServer**
 - **ContentContainer:** Quản lý metadata của phim, tập phim, thể loại và danh sách phát.
 - **ContentDatabase (PostgreSQLContainer):** Lưu trữ thông tin cấu trúc của nội dung video.
- **RatingServiceServer**
 - **RatingContainer:** Xử lý logic đánh giá, bình luận của người dùng về nội dung.
 - **RatingDatabase (MongoDBContainer):** Lưu trữ dữ liệu phi cấu trúc (document-based) của các bài đánh giá và phản hồi.
- **SearchEngineServer (ElasticSearchContainer)** Cung cấp công cụ tìm kiếm toàn văn (Full-text search), hỗ trợ tính năng gợi ý và tìm kiếm nội dung theo từ khóa phức tạp.
- **NotificationServiceServer**
 - **NotificationContainer:** Quản lý việc tạo và gửi thông báo (email, push notification).
 - **NotificationDatabase (MongoDBContainer):** Lưu trữ lịch sử và trạng thái các thông báo.
- **PaymentServiceServer**
 - **PaymentContainer:** Xử lý giao dịch thanh toán, đăng ký gói cước. Kết nối với cổng thanh toán bên ngoài (VNPAY, MOMO).
 - **PaymentDatabase (PostgreSQLContainer):** Lưu trữ lịch sử giao dịch và trạng thái đơn hàng.
- **MessageBrokerServer**
 - **KafkaContainer:** Hệ thống stream processing để xử lý các sự kiện thời gian thực giữa các microservice.
 - **RedisBullContainer:** Hàng đợi (Queue) để xử lý các tác vụ nền (background jobs) thông qua thư viện Bull.
- **CacheServer (RedisContainer)** Lưu trữ bộ nhớ đệm (caching) cho các dữ liệu truy xuất thường xuyên nhằm giảm độ trễ phản hồi.

- **External Services (Cloud Providers)**

- **EmailService (SMTP Server):** Dịch vụ gửi email xác thực và thông báo.
- **PaymentGateways (VNPAY, MOMO):** Các cổng thanh toán điện tử tích hợp.

Thiết bị đầu cuối:

- **UserComputer (WebBrowser):** Thiết bị của người dùng cuối, truy cập ứng dụng để xem nội dung.
- **AdminComputer (AdminDashboard):** Thiết bị của quản trị viên, truy cập dashboard để quản lý hệ thống.

4.6.3 Hướng Dẫn Thay Đổi

- Mô hình triển khai sử dụng Kubernetes cho phép dễ dàng mở rộng (Scale Out) các service cụ thể (ví dụ: tăng số lượng pod cho *StreamingContainer* khi lượng người xem tăng cao).
- Khi thêm một Microservice mới: Cần định nghĩa Deployment và Service tương ứng trong Kubernetes, đồng thời cấu hình Ingress Route nếu cần truy cập từ bên ngoài.
- Việc thay đổi cấu hình hạ tầng mạng hoặc CDN được thực hiện thông qua Cloudflare Dashboard và cấu hình Cloudflared Tunnel.

4.6.4 Lý Do Thiết Kế

Kiến trúc triển khai Microservices trên Kubernetes được lựa chọn vì các lý do sau:

- **Tính đa hình trong lưu trữ (Polyglot Persistence):** Mỗi dịch vụ sử dụng loại cơ sở dữ liệu phù hợp nhất với đặc thù dữ liệu của nó (PostgreSQL cho dữ liệu quan hệ, MongoDB cho dữ liệu linh hoạt, ClickHouse cho phân tích, Elasticsearch cho tìm kiếm).
- **Khả năng mở rộng độc lập:** Các module như *Streaming* hay *Search* có thể được cấp phát tài nguyên độc lập tùy theo tải thực tế mà không ảnh hưởng đến các module khác như *Payment*.
- **Hiệu năng cao:** Sử dụng Caching (Redis) và Message Broker (Kafka) giúp hệ thống phản hồi nhanh và xử lý tác vụ bất đồng bộ hiệu quả.

5 Cở sở dữ liệu

Để đảm bảo tính độc lập và tuân thủ kiến trúc Microservices, các thực thể dữ liệu được phân chia và quản lý bởi từng dịch vụ cụ thể như sau:

5.1 Dịch vụ xác thực và phân quyền (Auth Service)

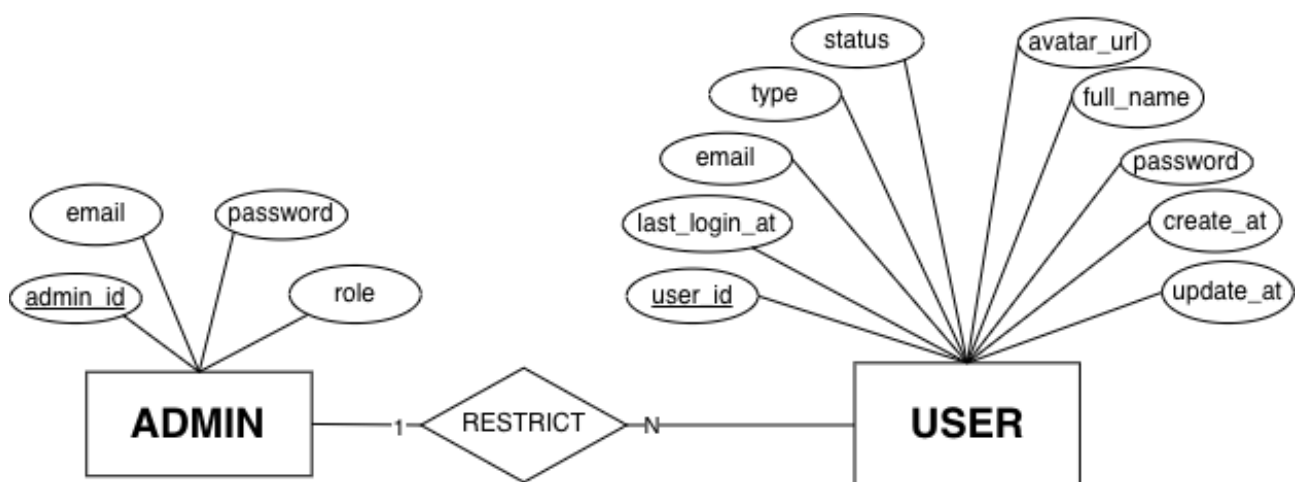
Dịch vụ Auth đóng vai trò là "người gác cổng" của hệ thống FlixZone, chịu trách nhiệm quản lý định danh (Identity) và kiểm soát truy cập (Access Management).

Chức năng nghiệp vụ (Domain Functions):

- **Identity Management:** Quản lý vòng đời tài khoản của người dùng cuối (End-User) và quản trị viên (Admin), bao gồm đăng ký, cập nhật hồ sơ và trạng thái hoạt động.
- **Authentication (Xác thực):** Xác minh danh tính người dùng thông qua Email/Password hoặc OAuth (Google/Facebook), cấp phát Access Token (JWT) để truy cập hệ thống.
- **Authorization (Phân quyền):** Phân định quyền hạn dựa trên vai trò (Role-based Access Control), đảm bảo người dùng chỉ truy cập được các tài nguyên cho phép (Ví dụ: Guest, Free User, Premium User, Admin).
- **Account Security:** Quản lý việc khóa/mở khóa tài khoản khi có vi phạm hoặc yêu cầu từ quản trị viên.

5.1.1 Mô hình dữ liệu

Mô hình dữ liệu mức logic của Auth Service tập trung vào các thực thể định danh và mối quan hệ quản lý giữa chúng.



Hình 15: Sơ đồ EERD cho Auth Service

Tên thực thể	Mô tả
USER	Đại diện cho người dùng hệ thống (End-user). Lưu trữ thông tin cá nhân, thông tin đăng nhập và trạng thái tài khoản.
ADMIN	Đại diện cho quản trị viên hệ thống. Lưu trữ thông tin xác thực để truy cập vào trang quản trị.

Bảng 31: Danh sách thực thể Auth Service

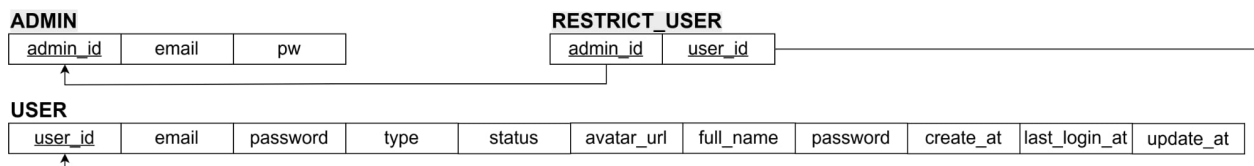
5.1.1.1 Danh sách thực thể

Tên quan hệ	Thực thể liên quan	Mô tả
RESTRICT	ADMIN → USER	Quan hệ thể hiện hành động quản trị: Admin có quyền hạn chế (ban/block) hoặc thay đổi trạng thái hoạt động của User khi phát hiện vi phạm.

Bảng 32: Danh sách quan hệ Auth Service

5.1.1.2 Danh sách quan hệ

5.1.2 Ánh xạ mô hình dữ liệu



Hình 16: Sơ đồ mapping EERD cho Auth Service

5.1.2.1 Bảng: users Lưu trữ thông tin người dùng. Đây là bảng trung tâm để xác định danh tính (Subject) trong các mã thông báo (Token).

Tên trường	Kiểu dữ liệu	Mô tả
user_id	UUID	Khóa chính. Định danh duy nhất cho người dùng trên toàn hệ thống.
email	VARCHAR(255)	Địa chỉ email đăng nhập (Unique).
password_hash	VARCHAR	Mật khẩu đã được mã hóa (Bcrypt/Argon2).
full_name	VARCHAR(100)	Tên hiển thị của người dùng.
avatar_url	TEXT	Đường dẫn ảnh đại diện (Lưu trên Object Storage).
type	ENUM	Phân loại người dùng: 'REGULAR', 'PREMIUM'.
status	ENUM	Trạng thái tài khoản: 'ACTIVE', 'RESTRICTED', 'PENDING'.
created_at	TIMESTAMP	Thời điểm tạo tài khoản.
updated_at	TIMESTAMP	Thời điểm cập nhật thông tin gần nhất.
last_login_at	TIMESTAMP	Thời điểm đăng nhập gần nhất.

Bảng 33: Chi tiết bảng users

5.1.2.2 Bảng: admins Lưu trữ tài khoản có quyền truy cập vào hệ thống quản trị (Dashboard). Bảng này được tách biệt với bảng users để tăng cường bảo mật.



Tên trường	Kiểu dữ liệu	Mô tả
admin_id	UUID	Khóa chính. Định danh duy nhất cho quản trị viên.
email	VARCHAR(255)	Email dùng để đăng nhập trang quản trị.
password_hash	VARCHAR	Mật khẩu quản trị đã được mã hóa.
role	VARCHAR(50)	Quyền hạn cụ thể: 'SUPER_ADMIN', 'CONTENT_MANAGER', 'ANALYST'.
created_at	TIMESTAMP	Thời điểm tạo tài khoản admin.

Bảng 34: Chi tiết bảng admins

5.2 Dịch vụ quản lý nội dung (Content Service)

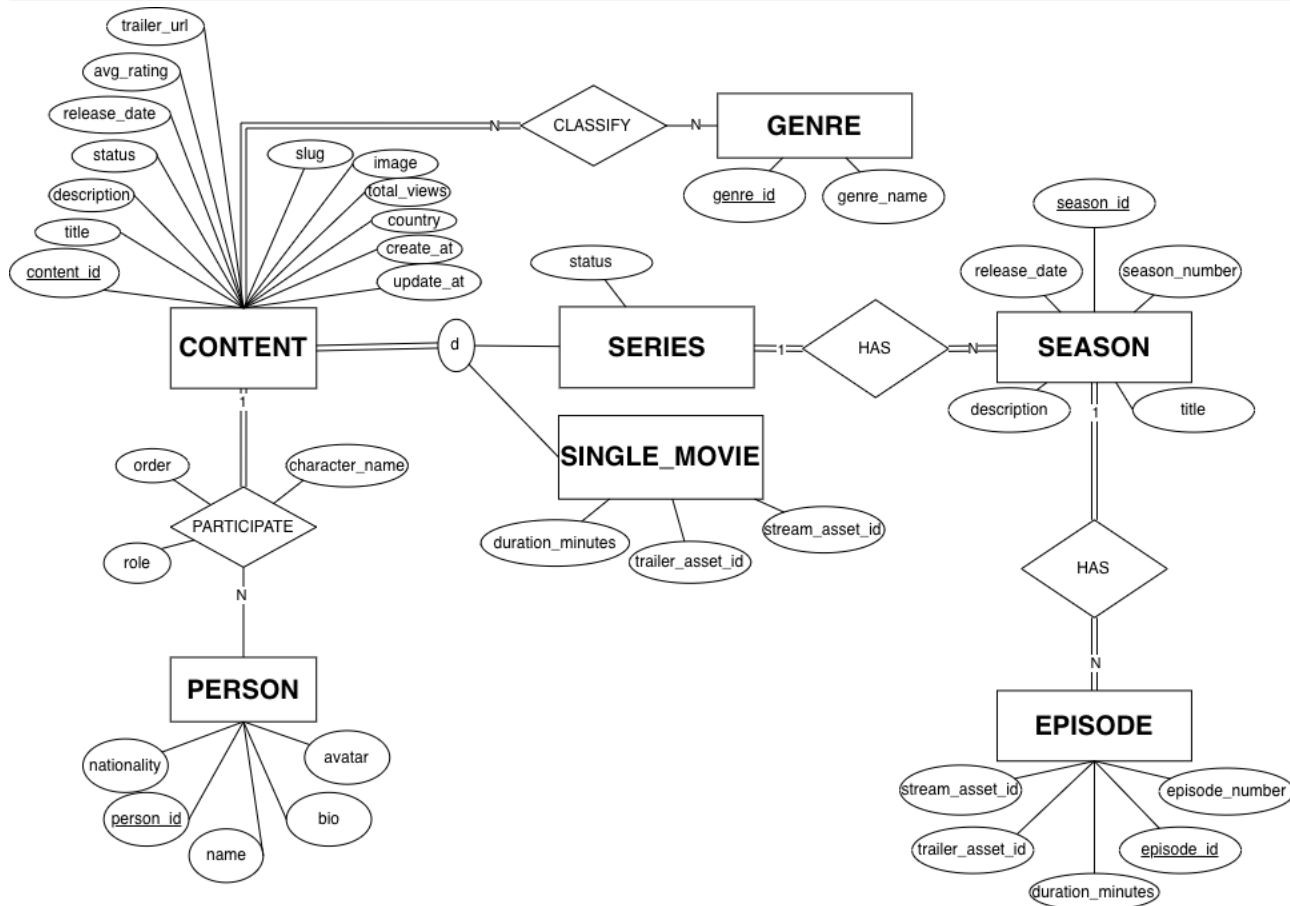
Content Service đóng vai trò là kho lưu trữ tập trung (Central Repository) cho toàn bộ siêu dữ liệu (Metadata) về phim ảnh trong hệ thống FlixZone. Dịch vụ này được thiết kế để xử lý cấu trúc dữ liệu đa dạng của phim lẻ và phim bộ.

Chức năng nghiệp vụ (Domain Functions):

- **Metadata Management (CRUD):** Quản lý vòng đời thông tin của Phim lẻ (Movies), Phim bộ (TV Series), Mùa (Seasons) và Tập phim (Episodes).
- **Content Organization:** Phân loại nội dung theo Thể loại (Genre) và quản lý thông tin Ê-kíp sản xuất (Diễn viên, Đạo diễn).
- **Search & Discovery Support:** Cung cấp dữ liệu chuẩn hóa cho các dịch vụ Tìm kiếm (Elasticsearch) và Gợi ý (Recommendation) để indexing.
- **Asset Linkage:** Ánh xạ giữa thông tin phim (Metadata) với các tài nguyên media vật lý (Video/Subtitles) được lưu trữ tại Streaming Service.

5.2.1 Mô hình dữ liệu

Mô hình dữ liệu logic của Content Service sử dụng kỹ thuật *Class Inheritance* (Kế thừa lớp) để tối ưu hóa việc lưu trữ các thuộc tính chung.



Hình 17: Sơ đồ EERD cho Content Service

Tên thực thể	Mô tả
CONTENT	Thực thể cha (Base Entity). Chứa các thuộc tính chung nhất mà cả phim lẻ và phim bộ đều có (Tên, mô tả, ảnh bìa, năm phát hành).
SINGLE_MOVIE	Thực thể con (Sub-entity). Đại diện cho phim lẻ, kế thừa từ CONTENT và bổ sung thêm thời lượng phim.
SERIES	Thực thể con (Sub-entity). Đại diện cho phim bộ, kế thừa từ CONTENT và chứa danh sách các mùa phim.
SEASON	Mùa phim. Một đơn vị phân cấp nằm trong Series.
EPISODE	Tập phim. Đơn vị nội dung nhỏ nhất có thể xem được (watchable unit) trong một Series.
GENRE	Danh mục thể loại (Hành động, Tình cảm, Viễn tưởng...).
PERSON	Hồ sơ nghệ sĩ (Diễn viên, Đạo diễn, Biên kịch...).

Bảng 35: Danh sách thực thể Content Service

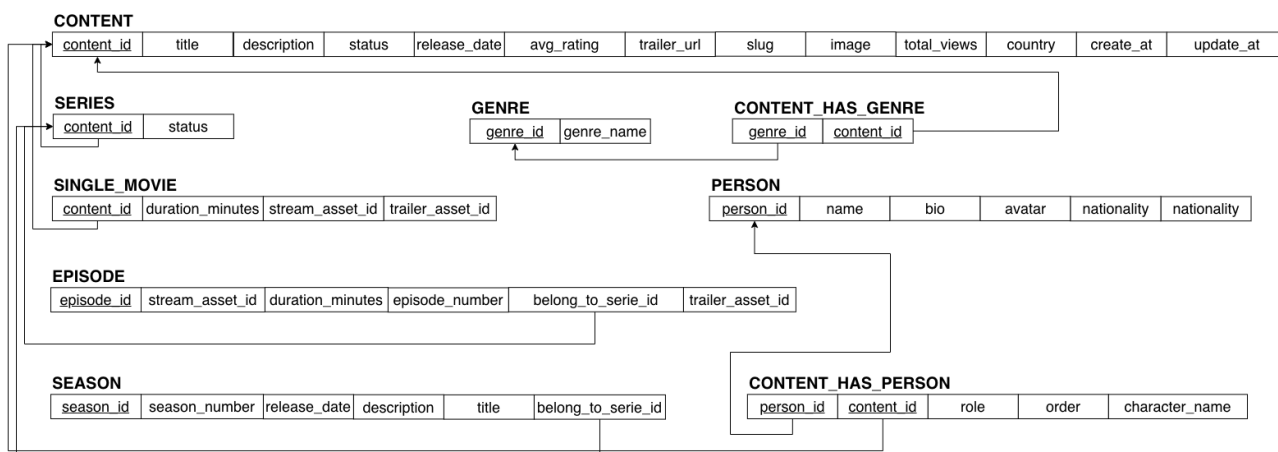
5.2.1.1 Danh sách thực thể

Tên quan hệ	Thực thể liên quan	Mô tả
disjoint (Inheritance)	MOVIE/SERIES → CONTENT	Quan hệ kế thừa. Một Movie hoặc Series là một Content.
HAS	SERIES → SEASON	Quan hệ phân cấp cha-con (One-to-Many). Một Series có nhiều Season.
HAS	SEASON → EPISODE	Quan hệ phân cấp cha-con (One-to-Many). Một Season có nhiều Episode.
PARTICIPATE	CONTENT ↔ PERSON	Quan hệ nhiều-nhiều (Many-to-Many). Một nghệ sĩ có thể tham gia nhiều phim với các vai trò khác nhau.
CLASSIFY	CONTENT ↔ GENRE	Quan hệ phân loại nhiều-nhiều. Một phim có thể thuộc nhiều thể loại.

Bảng 36: Danh sách quan hệ Content Service

5.2.1.2 Danh sách quan hệ

5.2.2 Ánh xạ mô hình dữ liệu



Hình 18: Sơ đồ mapping EERD cho Content Service

5.2.2.1 Bảng: contents (Lớp cha) Lưu trữ siêu dữ liệu (metadata) chung cho tất cả các loại nội dung video.

Tên trường	Kiểu dữ liệu	Mô tả
content_id	UUID	Khóa chính (PK). Định danh duy nhất cho nội dung.
title	VARCHAR(255)	Tên hiển thị của bộ phim.
slug	VARCHAR(255)	Đường dẫn URL thân thiện (SEO-friendly), ví dụ: <i>iron-man-2008</i> .
description	TEXT	Mô tả nội dung phim.
release_date	DATE	Ngày phát hành công chiếu.
avg_rating	DECIMAL(3, 2)	Điểm đánh giá trung bình.
total_views	BIGINT	Tổng lượt xem tích lũy.
image	VARCHAR	Đường dẫn ảnh bìa/poster.
trailer_url	VARCHAR	Link xem trailer (trên Youtube hoặc nguồn công khai).
country	VARCHAR(50)	Quốc gia sản xuất.
status	ENUM	Trạng thái hiển thị: <i>'DRAFT'</i> , <i>'PUBLISHED'</i> , <i>'ARCHIVED'</i> .
created_at	TIMESTAMP	Thời điểm tạo bản ghi.
updated_at	TIMESTAMP	Thời điểm cập nhật bản ghi gần nhất.

Bảng 37: Chi tiết bảng contents

5.2.2.2 Bảng: single_movies (Lớp con) Chứa thông tin đặc thù của phim lẻ, kế thừa ID từ bảng contents.

Tên trường	Kiểu dữ liệu	Mô tả
content_id	UUID	Khóa chính (PK), đồng thời là Khóa ngoại (FK) tham chiếu đến contents .
duration_minutes	INT	Thời lượng phim (tính bằng phút).
stream_asset_id	UUID	ID của video phim gốc (Lưu tại Streaming Service).
trailer_asset_id	UUID	ID của video trailer (Lưu tại Streaming Service).

Bảng 38: Chi tiết bảng single_movies

5.2.2.3 Bảng: series (Lớp con) Đại diện cho phim bộ, kế thừa ID từ bảng contents.

Tên trường	Kiểu dữ liệu	Mô tả
content_id	UUID	Khóa chính (PK), đồng thời là Khóa ngoại (FK) tham chiếu đến contents .
status	ENUM	Trạng thái sản xuất: <i>'ONGOING'</i> , <i>'ENDED'</i> .

Bảng 39: Chi tiết bảng series

5.2.2.4 Bảng: seasons & episodes Cấu trúc phân cấp nội dung của phim bộ.

Tên trường	Kiểu dữ liệu	Mô tả
season_id	UUID	Khóa chính (PK).
belong_to_serie_id	UUID	Khóa ngoại (FK) tham chiếu đến series .
season_number	INT	Số thứ tự mùa phim (Mùa 1, Mùa 2...).
title	VARCHAR	Tiêu đề mùa phim (nếu có).
description	TEXT	Mô tả nội dung mùa phim.
release_date	DATE	Ngày phát hành mùa phim.

Bảng 40: Chi tiết bảng seasons

Tên trường	Kiểu dữ liệu	Mô tả
episode_id	UUID	Khóa chính (PK).
belong_to_serie_id	UUID	Khóa ngoại (FK) tham chiếu đến seasons .
episode_number	INT	Số tập phim.
duration_minutes	INT	Thời lượng tập phim.
stream_asset_id	UUID	ID video tập phim (từ Streaming Service).
trailer_asset_id	UUID	ID video trailer của tập (nếu có).

Bảng 41: Chi tiết bảng episodes

5.2.2.5 Bảng danh mục và người tham gia phim: genres & persons Lưu trữ thông tin thể loại và diễn viên, đạo diễn.

Tên trường	Kiểu dữ liệu	Mô tả
genre_id	UUID	Khóa chính.
genre_name	VARCHAR	Tên thể loại (Hành động, HÀi hước...).

Bảng 42: Chi tiết bảng genres

Tên trường	Kiểu dữ liệu	Mô tả
person_id	UUID	Khóa chính.
name	VARCHAR	Tên nghệ sĩ.
bio	TEXT	Tiểu sử.
avatar	VARCHAR	Link ảnh đại diện.
nationality	VARCHAR	Quốc tịch.

Bảng 43: Chi tiết bảng persons

5.2.2.6 Bảng: content_has_genres (Map từ quan hệ CLASSIFY) Bảng trung gian hiện thực hóa mối quan hệ nhiều-nhiều giữa phim và thể loại.

Tên trường	Kiểu dữ liệu	Mô tả
content_id	UUID	Khóa chính (PK) & Khóa ngoại (FK). Tham chiếu trực tiếp đến trường content_id của bảng contents. Xác định bộ phim cần phân loại.
genre_id	UUID	Khóa chính (PK) & Khóa ngoại (FK). Tham chiếu trực tiếp đến trường genre_id của bảng genres. Xác định thể loại tương ứng.

Bảng 44: Chi tiết bảng content_genres

5.2.2.7 Bảng: content_persons (Map từ quan hệ PARTICIPATE) Bảng lưu trữ thông tin ê-kíp sản xuất, liên kết giữa phim và nghệ sĩ. Do một nghệ sĩ có thể đóng nhiều vai trò (vừa là đạo diễn vừa là diễn viên), khóa chính sẽ là tổ hợp của cả 3 trường: content, person và role.

Tên trường	Kiểu dữ liệu	Mô tả
content_id	UUID	PK, FK. Tham chiếu đến bảng contents. Xác định bộ phim mà nghệ sĩ tham gia.
person_id	UUID	PK, FK. Tham chiếu đến bảng persons. Xác định danh tính nghệ sĩ.
role	VARCHAR(50)	PK. Vai trò cụ thể trong dự án (Ví dụ: 'ACTOR', 'DIRECTOR').
character_name	VARCHAR(100)	Tên nhân vật trong phim (chỉ có giá trị khi role là 'ACTOR', ngược lại có thể để NULL).
order	INT	Thứ tự ưu tiên hiển thị (Billing Order). Ví dụ: Diễn viên chính có order=1, diễn viên phụ có order cao hơn. Order càng cao, độ ưu tiên càng thấp

Bảng 45: Chi tiết bảng content_persons

5.3 Dịch vụ xử lý và truyền phát video (Streaming Service)

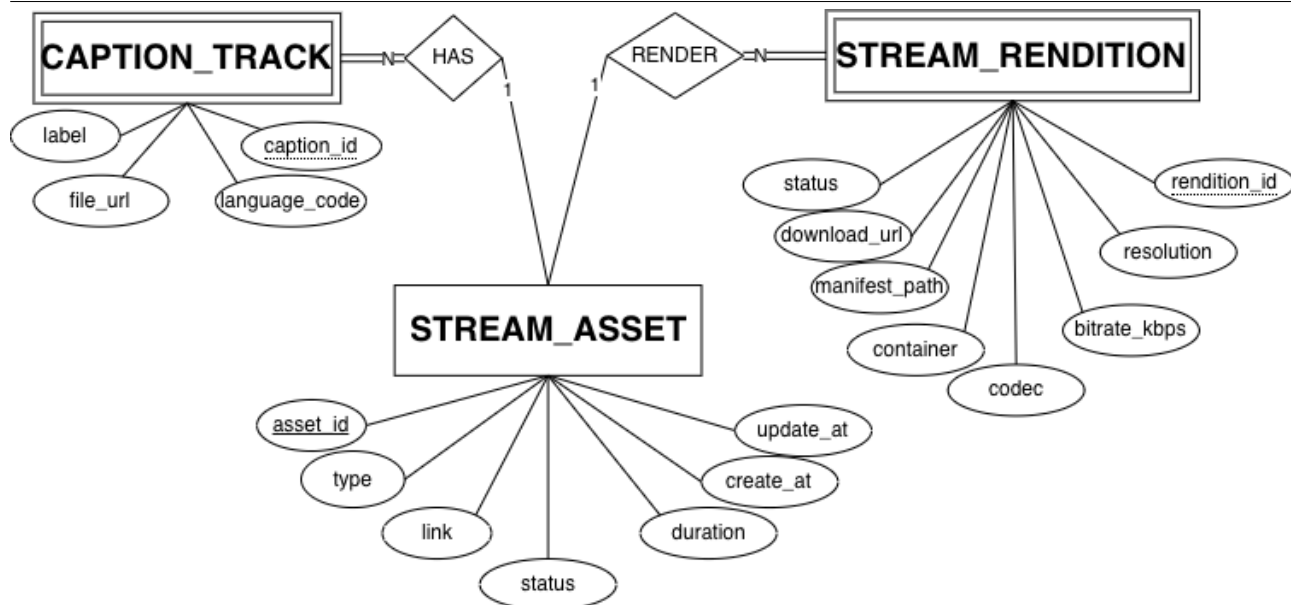
Streaming Service là thành phần chịu trách nhiệm quản lý vòng đời kỹ thuật của video, từ lúc tải lên (Ingest), chuyển mã (Transcode) sang định dạng thích ứng (Adaptive Bitrate Streaming), đến việc lưu trữ và phân phối nội dung (Delivery) qua CDN.

Chức năng nghiệp vụ (Domain Functions):

- **Asset Management:** Quản lý kho tài nguyên video gốc (Raw/Source files) và các tệp phụ trợ (Subtitle, Thumbnail).
- **Transcoding Orchestration:** Điều phối quá trình chuyển đổi video gốc thành nhiều phiên bản chất lượng khác nhau (360p, 720p, 1080p...) và đóng gói theo chuẩn HLS/DASH để tối ưu hóa băng thông người xem.
- **Metadata Extraction:** Trích xuất thông tin kỹ thuật của video như thời lượng (duration), codec, bitrate, độ phân giải.
- **Delivery & Security:** Cung cấp các đường dẫn phát (manifest URL) và tích hợp cơ chế bảo vệ bản quyền (DRM/Signed Token) khi giao tiếp với Client.

5.3.1 Mô hình dữ liệu

Mô hình dữ liệu của Streaming Service tập trung vào thực thể **Asset** (Tài sản số) và các biến thể của nó.



Hình 19: Sơ đồ EERD cho Streaming Service

Tên thực thể	Mô tả
STREAM_ASSET	Thực thể gốc đại diện cho một video được tải lên hệ thống. Nó đóng vai trò là "container" quản lý tất cả các phiên bản chuyển mã và phụ đề đi kèm.
STREAM_RENDITION	Các phiên bản chuyển mã (Transcoded Versions) của Asset. Mỗi rendition đại diện cho một mức chất lượng cụ thể (VD: 1080p High Bitrate) để phục vụ thuật toán Adaptive Bitrate.
CAPTION_TRACK	Các tệp phụ đề (Subtitles/Closed Captions) đi kèm với video, hỗ trợ đa ngôn ngữ.

Bảng 46: Danh sách thực thể Streaming Service

5.3.1.1 Danh sách thực thể

Tên quan hệ	Thực thể liên quan	Mô tả
RENDER	ASSET → RENDITION	Quan hệ 1-N. Một video gốc được render ra nhiều phiên bản chất lượng khác nhau.
HAS	ASSET → CAPTION	Quan hệ 1-N. Một video có thể đi kèm nhiều track phụ đề (Tiếng Việt, Tiếng Anh...).
LINK_TO	EXT (Movie/Ep) → ASSET	Quan hệ tham chiếu logic từ Content Service. Streaming Service không lưu khóa ngoại của phim, mà Content Service sẽ lưu stream_asset_id để liên kết.

Bảng 47: Danh sách quan hệ Streaming Service

5.3.1.2 Danh sách các mối quan hệ

5.3.2 Ảnh xạ mô hình dữ liệu

STREAM_ASSET

asset_id	type	link	status	duration	create_at	update_at
----------	------	------	--------	----------	-----------	-----------

CAPTION_TRACK

caption_id	language_code	file_url	label	asset_id
------------	---------------	----------	-------	----------

STREAM_RENDITION

rendition_id	resolution	bitrate_kbps	codec	container	manifest_path	download_url	status	asset_id
--------------	------------	--------------	-------	-----------	---------------	--------------	--------	----------

Hình 20: Sơ đồ mapping EERD cho Stream Service

5.3.2.1 Bảng: stream_assets Quản lý file nguồn và liên kết với Cloud Provider. Bảng này được bổ sung các trường định danh từ Cloudflare và cờ đánh dấu bản quyền.

Tên trường	Kiểu dữ liệu	Mô tả
asset_id	UUID	PK. Định danh duy nhất của tài nguyên video trong hệ thống nội bộ.
provider_video_id	VARCHAR(100)	ID video do Cloudflare Stream cấp phát (VD: d2b38...). Dùng để gọi API lấy manifest, thumbnail hoặc xóa video.
provider_name	VARCHAR(50)	Tên nhà cung cấp hạ tầng (Mặc định: 'CLOUDFLARE'). Giúp hệ thống dễ dàng mở rộng sang AWS/YouTube sau này.
type	VARCHAR(50)	Loại asset: 'MAIN_MOVIE', 'TRAILER'.
status	ENUM	Trạng thái xử lý: 'UPLOADING', 'PROCESSING', 'READY', 'ERROR'.
link	VARCHAR	Đường dẫn tới file gốc (Source URL) hoặc ID trên Cloudflare Stream.
duration	FLOAT	Tổng thời lượng video (giây).
created_at	TIMESTAMP	Thời điểm tạo.
updated_at	TIMESTAMP	Thời điểm cập nhật trạng thái gần nhất.

Bảng 48: Chi tiết bảng stream_assets

5.3.2.2 Bảng: stream_renditions Lưu thông tin các phiên bản chất lượng (Giữ nguyên logic nhưng bổ sung chi tiết về Manifest).

Tên trường	Kiểu dữ liệu	Mô tả
rendition_id	UUID	PK.
asset_id	UUID	FK. Tham chiếu tới stream_assets .
resolution	VARCHAR(20)	Độ phân giải (VD: '1080p', '720p').
bitrate_kbps	INT	Tốc độ bit trung bình.
codec	VARCHAR(20)	Chuẩn nén video (VD: 'h264', 'vp9', 'av1').
container	VARCHAR(10)	Định dạng container (VD: 'ts' cho HLS, 'm4s' cho DASH).
download_url	VARCHAR	Đường dẫn tải xuống file vật lý (Dành cho tính năng Download Offline của Premium User).
manifest_path	VARCHAR	Đường dẫn (url) file playlist .m3u8 hoặc .mpd của riêng rendition này.
status	ENUM	Trạng thái: 'READY', 'PROCESSING'.

Bảng 49: Chi tiết bảng stream_renditions

5.3.2.3 Bảng: caption_tracks (Không thay đổi nhiều, chỉ chuẩn hóa tên trường).

Tên trường	Kiểu dữ liệu	Mô tả
caption_id	UUID	PK.
asset_id	UUID	FK. Tham chiếu tới stream_assets .
language_code	VARCHAR(10)	Mã ngôn ngữ chuẩn ISO (vi, en, ja).
label	VARCHAR(50)	Tên hiển thị trên Player (VD: "Tiếng Việt - Mặc định").
file_url	VARCHAR	Đường dẫn file VTT/SRT trên CDN.

Bảng 50: Chi tiết bảng caption_tracks

5.4 Dịch vụ thông báo (Notification Service)

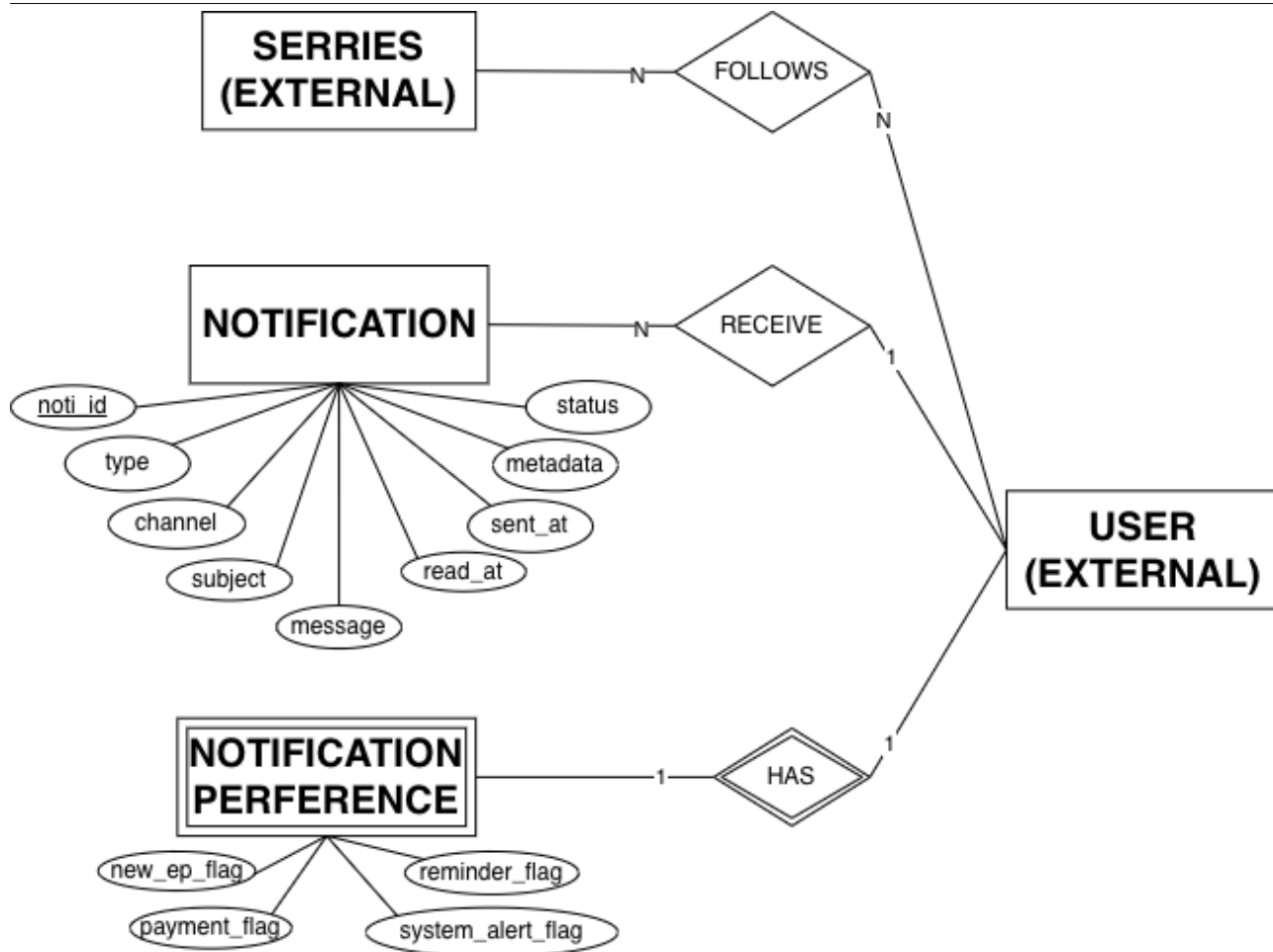
Notification Service là trung tâm điều phối thông tin của FlixZone, chịu trách nhiệm gửi thông báo đến người dùng qua đúng kênh (Email, In-App) vào đúng thời điểm, dựa trên cấu hình cá nhân hóa.

Chức năng nghiệp vụ (Domain Functions):

- **Multi-channel Dispatcher:** Định tuyến thông báo qua các kênh khác nhau: Email (SendGrid/SMTP) và thông báo trong ứng dụng (In-App).
- **User Preference Management:** Quản lý cài đặt nhận tin của người dùng (Ví dụ: Chỉ nhận tin về phim mới, tắt tin quảng cáo) để tránh spam (User Fatigue).
- **Template:** Quản lý các mẫu thông báo động (HTML Email, Push payload), cho phép thay thế nội dung (tên user, tên phim) linh hoạt mà không cần sửa code.
- **Device Token Management:** Lưu trữ và quản lý token thiết bị để phục vụ chức năng gửi Push Notification.

5.4.1 Mô hình dữ liệu

Mô hình dữ liệu tập trung vào việc lưu trữ lịch sử gửi tin và cấu hình người dùng.



Hình 21: Sơ đồ EERD cho Notification Service

Tên thực thể	Mô tả
NOTIFICATION	Lưu trữ nội dung và trạng thái của từng thông báo đã/đang gửi đi. Phục vụ cho tính năng "Hộp thư" (Inbox) của người dùng.
NOTIFICATION_PREF	(Preference) Lưu trữ cấu hình bật/tắt thông báo cho từng loại sự kiện của từng người dùng.
TEMPLATE	Lưu trữ các mẫu thông báo chuẩn (Tiêu đề, nội dung HTML) để tái sử dụng. <i>(phần này được lưu riêng trong container của notification service server)</i>

Bảng 51: Danh sách thực thể Notification Service

5.4.1.1 Danh sách thực thể

Tên quan hệ	Thực thể liên quan	Mô tả
RECEIVE	USER → NOTIFICATION	Quan hệ 1-N. Một người dùng nhận nhiều thông báo theo thời gian.
HAS_PREF	USER ↔ NOTI_PREF	Quan hệ 1-1. Mỗi người dùng có một bộ cấu hình duy nhất.
FOLLOW(SUBSCRIBE)	USER ↔ SERRIES	Quan hệ N-N. Mỗi người dùng có thể theo dõi nhiều series phim. Một series phim được theo dõi bởi nhiều người.

Bảng 52: Danh sách quan hệ Notification Service

5.4.1.2 Danh sách quan hệ

5.4.2 Ảnh xạ mô hình dữ liệu

NOTIFICATION

<u>noti_id</u>	type	channel	subject	message	readAt	sentAt	metadata	status	user_id
----------------	------	---------	---------	---------	--------	--------	----------	--------	---------

NOTIFICATION_PREFERENCE

<u>noti_pref_id</u>	new_ep_flag	new_recommend_flag	payment_flag	reminder_flag	readAt	sentAt	user_id
---------------------	-------------	--------------------	--------------	---------------	--------	--------	---------

SERIES_SUBSCRIPTION

<u>_id</u>	user_id	content_id	title	email	user_name	is_active	create_at	update_at
------------	---------	------------	-------	-------	-----------	-----------	-----------	-----------

Hình 22: Sơ đồ mapping EERD cho Notification Service

5.4.2.1 Collection: notifications Lưu lịch sử thông báo.

Tên trường	Kiểu dữ liệu	Mô tả
<u>_id</u>	ObjectId	PK. ID duy nhất của thông báo.
user_id	UUID	FK (Logical). Tham chiếu đến User Service. Người nhận thông báo.
type	String	Loại sự kiện: 'NEW_EPISODE', 'PAYMENT_SUCCESS', 'SYSTEM_ALERT'.
channel	String	Kênh gửi: 'EMAIL', 'IN_APP'.
subject	String	Tiêu đề thông báo.
message	String	Nội dung thông báo (hoặc HTML body cho email).
metadata	Object (JSON)	Dữ liệu kèm theo cho template hệ thống. VD: { "movie_id": "123", "action": "open_detail" }.
status	String	Trạng thái: 'PENDING', 'SENT', 'FAILED', 'READ'.
sent_at	Timestamp	Thời điểm gửi thành công.
read_at	Timestamp	Thời điểm người dùng đã xem.

Bảng 53: Chi tiết collection notifications

5.4.2.2 Collection: notification_preferences Lưu cấu hình nhận tin.

Tên trường	Kiểu dữ liệu	Mô tả
<u>_id</u>	ObjectId	PK. ID duy nhất của cấu hình.
user_id	UUID	FK. Tham chiếu đến User Service. Người nhận thông báo.
new_ep_flag	Boolean	Nhận thông báo khi series người dùng theo dõi có tập phim mới
payment_flag	Boolean	Nhận thông báo liên quan đến việc thanh toán premium
system_alert_flag	Boolean	Nhận thông báo đến việc bị cảnh báo khi có hành vi không chuẩn mực (<i>comment thiếu văn hóa</i>)
reminder_flag	Boolean	Nhận thông báo đến việc nhắc người dùng đã lâu không vào hệ thống xem phim (VD: <i>we miss you!</i>)

Bảng 54: Chi tiết collection notification_preferences

5.4.2.3 Collection: series_subscriptions Lưu thông lượt theo dõi của người dùng cho các series phim.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	PK. ID duy nhất của lượt follow.
user_id	UUID	FK. Tham chiếu đến User Service. Người nhận thông báo.
content_id	UUID	FK. Tham chiếu đến Content Service. Series được theo dõi.
title	String	Tiêu đề thông báo.
email	String	Email người dùng.
user_name	String	Tên người dùng.
is_active	Boolean	Trạng thái của lượt theo dõi (sẽ toggle khi người dùng unfollow hoặc follow lại).
creat_at	Timestamp	Thời điểm người dùng ấn nút theo dõi.
update_at	Timestamp	Thời điểm người dùng thay đổi quyết định (unfollow hoặc follow lại).

Bảng 55: Chi tiết collection series_subscriptions

5.5 Dịch vụ đánh giá và tương tác (Rating Service)

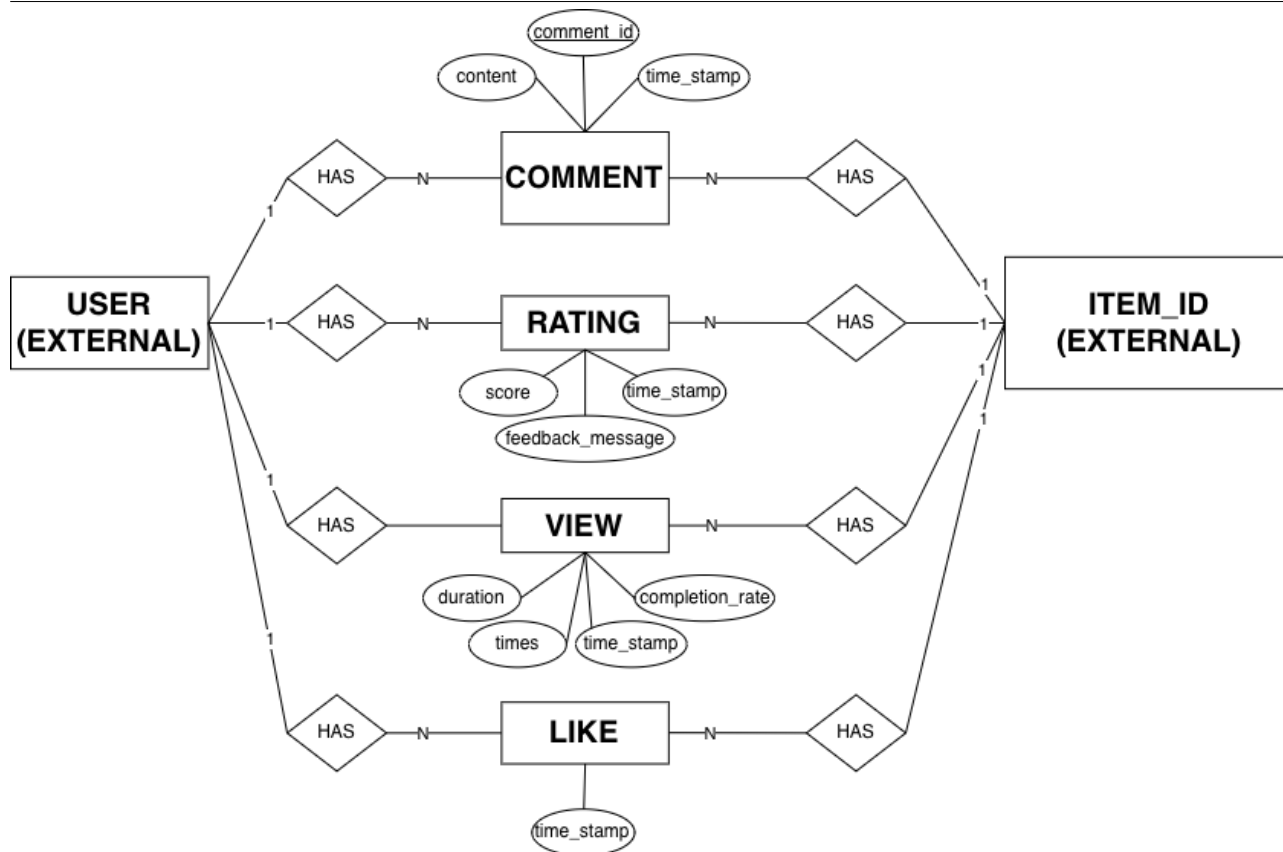
Rating Service quản lý các tương tác trực tiếp của người dùng đối với nội dung đang xem. Dựa trên thiết kế ERD, dịch vụ này tách biệt ba luồng dữ liệu chính: Bình luận, Đánh giá (kèm phản hồi), Lượt xem và Lượt thích. Hệ thống sử dụng MongoDB để lưu trữ các dữ liệu này nhằm đảm bảo khả năng ghi nhận (write) nhanh và linh hoạt.

Chức năng nghiệp vụ (Domain Functions):

- **Commenting:** Cho phép người dùng thảo luận, viết nội dung text về video đang xem.
- **Rating & Feedback:** Cho phép người dùng chấm điểm (Score) và gửi kèm tin nhắn phản hồi (Feedback message) về chất lượng nội dung.
- **Tracking views:** Ghi nhận lượt xem của người dùng đối với video.
- **Liking:** Ghi nhận hành động "Thích" của người dùng đối với video.

5.5.1 Mô hình dữ liệu

Mô hình dữ liệu logic bao gồm 3 thực thể chính xoay quanh mối quan hệ giữa Người dùng và Tài nguyên video (Stream Asset).



Hình 23: Sơ đồ EERD cho Rating Service

Tên thực thể	Mô tả
COMMENT	Lưu trữ nội dung văn bản mà người dùng thảo luận về video. Tách biệt với điểm số đánh giá.
RATING	Lưu trữ điểm số (Score) và tin nhắn phản hồi ngắn (Feedback message) của người dùng.
VIEW	Lưu trữ lượt xem của người dùng đối với phim.
LIKE	Ghi nhận trạng thái "Thích" của người dùng với video tại một thời điểm cụ thể.

Bảng 56: Danh sách thực thể Rating Service

5.5.1.1 Danh sách thực thể

Tên quan hệ	Thực thể liên quan	Mô tả
HAS (Comment)	USER → COMMENT → ITEM	Người dùng viết bình luận cho một bộ phim lẻ hoặc một tập phim bộ. Một User có thể viết nhiều Comment, một bộ phim lẻ hoặc một tập phim bộ có nhiều Comment.
HAS (Rating)	USER → RATING → ITEM	Người dùng gửi đánh giá cho một bộ phim lẻ hoặc một tập phim bộ.
HAS (View)	USER → VIEW → ITEM	Người dùng gửi đánh giá cho một bộ phim lẻ hoặc một tập phim bộ.
HAS (Like)	USER → LIKE → ITEM	Người dùng thích một bộ phim lẻ hoặc một tập phim bộ.

Bảng 57: Danh sách quan hệ Rating Service

5.5.1.2 Danh sách quan hệ

5.5.2 Ánh xạ mô hình dữ liệu

RATING

_id	user_id	score	feedback_message	time_stamp	item_id
-----	---------	-------	------------------	------------	---------

LIKE

_id	user_id	time_stamp	item_id
-----	---------	------------	---------

COMMENT

_id	user_id	content	time_stamp	item_id
-----	---------	---------	------------	---------

VIEW

_id	user_id	duration	times	time_stamp	completion_rate	item_id
-----	---------	----------	-------	------------	-----------------	---------

Hình 24: Sơ đồ mapping EERD cho Rating Service

5.5.2.1 Collection: comments Tương ứng với thực thể COMMENT trong ERD.

Tên trường	Kiểu dữ liệu	Mô tả
_id (comment_id)	ObjectId	PK. Định danh duy nhất của bình luận.
user_id	UUID	FK (External). Tham chiếu đến User Service.
item_id	UUID	FK (External). Tham chiếu đến Content Service. Video đang được bình luận.
content	String	Nội dung bình luận.
time_stamp	DateTime	Thời điểm tạo bình luận.

Bảng 58: Chi tiết collection comments

5.5.2.2 Collection: ratings Tương ứng với thực thể RATING trong ERD.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	PK.
user_id	UUID	FK (External). Tham chiếu đến User Service.
item_id	UUID	FK (External). Tham chiếu đến Content Service. Video đang được đánh giá.
score	Int	Điểm số đánh giá (từ 1-5).
feedback_message	String	Tin nhắn phản hồi đi kèm (ví dụ: "Phim hay", "Chất lượng kém").
time_stamp	DateTime	Thời điểm đánh giá.

Bảng 59: Chi tiết collection ratings

5.5.2.3 Collection: views Tương ứng với thực thể VIEW trong ERD.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	PK.
user_id	UUID	FK (External). Tham chiếu đến User Service.
item_id	UUID	FK (External). Tham chiếu đến Content Service. Video đang được đánh giá.
duration	Float	Thời lượng xem (giây).
completeion_rate	Float	Tỉ lệ hoàn thành của lần xem, có giá trị từ 0 đến 1.
times	Int	Số thứ tự của lần xem (1 = lần đầu, 2 = lần hai, ...).
time_stamp	DateTime	Thời điểm đánh giá.

Bảng 60: Chi tiết collection ratings

5.5.2.4 Collection: likes Tương ứng với thực thể LIKE trong ERD.

Tên trường	Kiểu dữ liệu	Mô tả
_id	ObjectId	PK.
user_id	UUID	FK (External). Tham chiếu đến User Service.
item_id	UUID	FK (External). Tham chiếu đến Content Service.
time_stamp	DateTime	Thời điểm người dùng nhấn nút Thích.

Bảng 61: Chi tiết collection likes

5.6 Dịch vụ thanh toán (Payment Service)

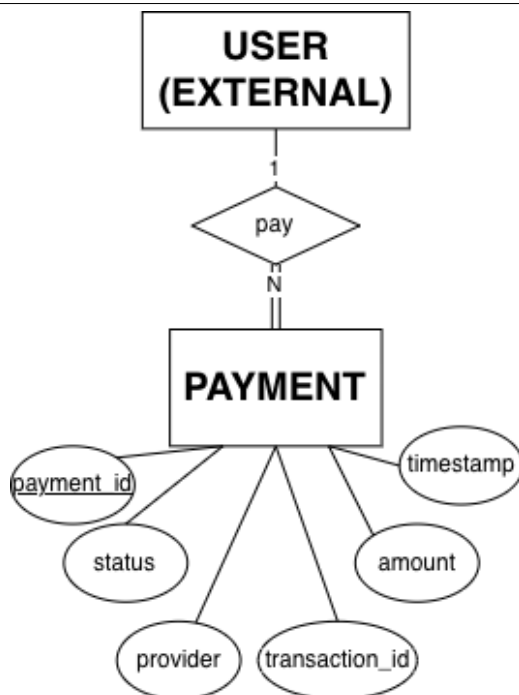
Payment Service là thành phần quan trọng xử lý các giao dịch tài chính trong hệ thống FlixZone. Dịch vụ này đóng vai trò trung gian, kết nối hệ thống với các cổng thanh toán điện tử (Payment Gateways) để thực hiện quy trình nâng cấp tài khoản người dùng lên Premium.

Chức năng nghiệp vụ (Domain Functions):

- **Transaction Processing:** Khởi tạo và xử lý vòng đời của một giao dịch thanh toán (Từ lúc chờ thanh toán → Thành công/Thất bại).
- **Gateway Integration:** Tích hợp API của các nhà cung cấp bên thứ ba (VNPay, MoMo) để lấy mã QR hoặc đường dẫn thanh toán.
- **Payment Verification (IPN):** Tiếp nhận tín hiệu xác thực (Instant Payment Notification) từ cổng thanh toán để đảm bảo tính toàn vẹn và chống gian lận.
- **History Tracking:** Lưu trữ biên lai điện tử phục vụ việc đối soát và hiển thị lịch sử cho người dùng.

5.6.1 Mô hình dữ liệu

Mô hình dữ liệu của Payment Service được thiết kế đơn giản nhưng chặt chẽ, tập trung vào thực thể Giao dịch (Payment).



Hình 25: Sơ đồ EERD cho Payment Service

Tên thực thể	Mô tả
PAYMENT	Đại diện cho một biên lai giao dịch. Lưu trữ đầy đủ thông tin về số tiền, trạng thái, nhà cung cấp và mã tham chiếu giao dịch.

Bảng 62: Danh sách thực thể Payment Service

5.6.1.1 Danh sách thực thể

Tên quan hệ	Thực thể liên quan	Mô tả
PAY	USER → PAYMENT	Quan hệ 1-N. Một người dùng có thể thực hiện nhiều lần thanh toán (ví dụ: mua gói tháng này, tháng sau mua tiếp, hoặc thanh toán lại khi lỗi).

Bảng 63: Danh sách quan hệ Payment Service

5.6.1.2 Danh sách quan hệ

5.6.2 Ánh xạ mô hình dữ liệu

PAYMENT

<u>payment_id</u>	user_id	provider	transaction_id	amount	timestamp	status
-------------------	---------	----------	----------------	--------	-----------	--------

Hình 26: Sơ đồ mapping EERD cho Payment Service

5.6.2.1 Bảng: payments Tương ứng với thực thể PAYMENT trong ERD.

Tên trường	Kiểu dữ liệu	Mô tả
payment_id	UUID	PK. Mã định danh duy nhất của giao dịch trong hệ thống FlixZone (Order ID).
user_id	UUID	FK (External). Tham chiếu đến User Service. Xác định người thực hiện thanh toán.
amount	DECIMAL(15, 2)	Số tiền thanh toán (VND).
provider	VARCHAR(50)	Tên cổng thanh toán được chọn: 'VNPAY', 'MOMO', 'ZALOPAY'.
transaction_id	VARCHAR(100)	Mã giao dịch tham chiếu từ phía Provider (Transaction No). Dùng để đối soát khi có khiếu nại.
status	VARCHAR(20)	Trạng thái giao dịch: 'PENDING' (Chờ), 'SUCCESS' (Thành công), 'FAILED' (Thất bại).
timestamp	TIMESTAMP	Thời điểm khởi tạo giao dịch.

Bảng 64: Chi tiết bảng payments

5.7 Dịch vụ phân tích dữ liệu (Analytics Service)

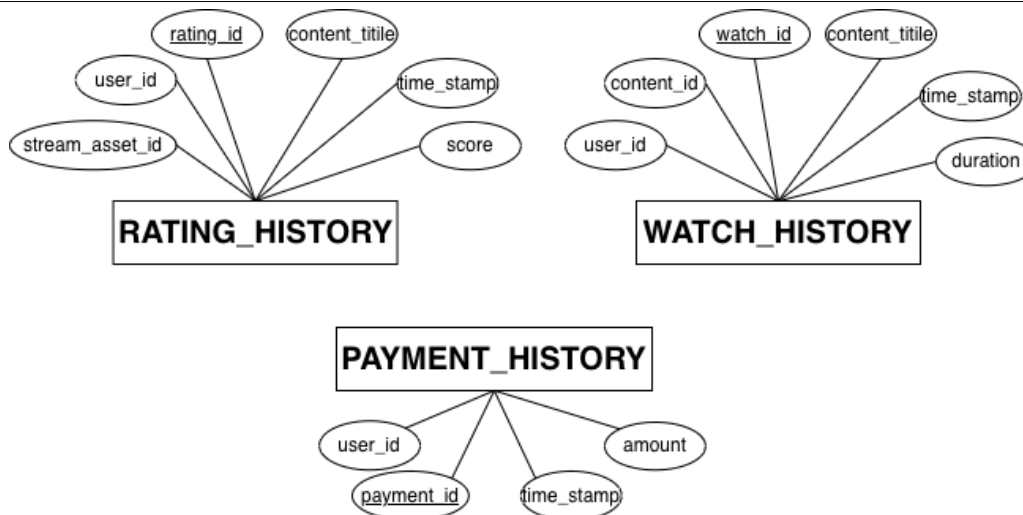
Analytics Service đóng vai trò là kho dữ liệu tập trung (Data Warehouse) của hệ thống FlixZone. Dịch vụ này không trực tiếp tạo ra dữ liệu mà hoạt động theo cơ chế hướng sự kiện (Event-driven): đăng ký (Subscribe) và lắng nghe các sự kiện phát sinh từ Rating Service và Payment Service để lưu trữ phục vụ cho các báo cáo quản trị (Business Intelligence).

Chức năng nghiệp vụ (Domain Functions):

- **Data Ingestion (Thu nạp dữ liệu):** Tiếp nhận dòng sự kiện (Event Stream) từ Kafka về các hành động xem phim, đánh giá và thanh toán của người dùng theo thời gian thực.
- **Historical Archiving:** Lưu trữ lịch sử hoạt động lâu dài (Time-series data) để phân tích xu hướng người dùng theo tháng/quý/năm.
- **Revenue Analysis:** Tổng hợp và tính toán doanh thu từ các giao dịch thanh toán để hiển thị trên Dashboard Admin.
- **Content Performance Tracking:** Phân tích hiệu suất nội dung thông qua lượt xem (Views), thời lượng xem (Watch time) và điểm đánh giá trung bình.

5.7.1 Mô hình dữ liệu

Mô hình dữ liệu của Analytics Service được thiết kế dạng **Star Schema** đơn giản hóa, tập trung vào các bảng Sự kiện (Fact Tables). Dữ liệu tại đây được thiết kế phi chuẩn hóa (Denormalized) - lưu trữ thừa thông tin (như tên phim) để tối ưu tốc độ đọc mà không cần JOIN với các Service khác.



Hình 27: Sơ đồ EERD cho Analytics Service

Tên thực thể	Mô tả
WATCH_HISTORY	Lưu trữ từng phiên xem phim của người dùng. Dữ liệu này được đồng bộ từ sự kiện Tracking của Rating Service. Dùng để tính toán "Tổng giờ xem" hoặc "Top phim thịnh hành".
RATING_HISTORY	Lưu trữ lịch sử đánh giá sao. Giúp phân tích xu hướng chất lượng nội dung theo thời gian (Ví dụ: Phim A tháng trước 5 sao, tháng này tụt xuống 3 sao).
PAYMENT_HISTORY	Lưu trữ nhật ký giao dịch tài chính được đồng bộ từ Payment Service. Dùng để vẽ biểu đồ doanh thu.

Bảng 65: Danh sách thực thể Analytics Service

5.7.2 Ánh xạ mô hình dữ liệu

Do đặc thù cần xử lý lượng dữ liệu lớn và truy vấn tổng hợp (Aggregation) nhanh, Analytics Service sử dụng **ClickHouse** (cơ sở dữ liệu dạng cột - OLAP) để lưu trữ.

RATING_HISTORY

<u>rating_id</u>	user_id	item_id	content_title	time_stamp	score
------------------	---------	---------	---------------	------------	-------

PAYMENT_HISTORY

<u>payment_id</u>	user_id	time_stamp	amount
-------------------	---------	------------	--------

WATCH_HISTORY

<u>watch_id</u>	user_id	item_id	content_title	time_stamp	duration
-----------------	---------	---------	---------------	------------	----------

Hình 28: Sơ đồ mapping cho Analytics Service

5.7.2.1 Bảng: watch_history Lưu vết hành vi xem phim (Sub từ Rating Service).

Tên trường	Kiểu dữ liệu	Mô tả
watch_id	UUID	PK. Định danh duy nhất của phiên xem.
user_id	UUID	ID người dùng thực hiện hành vi.
content_id	UUID	ID của bộ phim/tập phim.
content_title	VARCHAR	Denormalized Field: Tên phim được lưu cứng tại thời điểm xem. Giúp Admin xem báo cáo nhanh mà không cần query sang Content Service.
duration	INT	Thời lượng người dùng đã xem trong phiên này (giây).
time_stamp	DATETIME	Thời điểm ghi nhận sự kiện (Dùng để lọc báo cáo theo ngày/tháng).

Bảng 66: Chi tiết bảng watch_history

5.7.2.2 Bảng: rating_history Lưu vết đánh giá chất lượng (Sub từ Rating Service).

Tên trường	Kiểu dữ liệu	Mô tả
rating_id	UUID	PK. Định danh lượt đánh giá.
user_id	UUID	ID người dùng đánh giá.
stream_asset_id	UUID	ID của file video (Asset) được đánh giá (nhằm phân tích chất lượng kỹ thuật của nguồn phát).
content_title	VARCHAR	Tên nội dung tại thời điểm đánh giá.
score	INT	Điểm số đánh giá (1-5).
time_stamp	DATETIME	Thời điểm đánh giá.

Bảng 67: Chi tiết bảng rating_history

5.7.2.3 Bảng: payment_history Lưu vết dòng tiền (Sub từ Payment Service).

Tên trường	Kiểu dữ liệu	Mô tả
payment_id	UUID	PK. Mã giao dịch gốc.
user_id	UUID	ID người thực hiện thanh toán.
amount	DECIMAL	Số tiền thực tế thu được (Doanh thu). Dùng cho các hàm SUM() để tính tổng doanh thu.
time_stamp	DATETIME	Thời điểm giao dịch thành công.

Bảng 68: Chi tiết bảng payment_history

5.8 Dịch vụ gợi ý (Recommendation Service)

Recommendation Service là trí tuệ của hệ thống FlixZone, chịu trách nhiệm thu thập dữ liệu hành vi người dùng, huấn luyện mô hình AI (NCF) và đưa ra danh sách phim phù hợp nhất cho từng cá nhân.

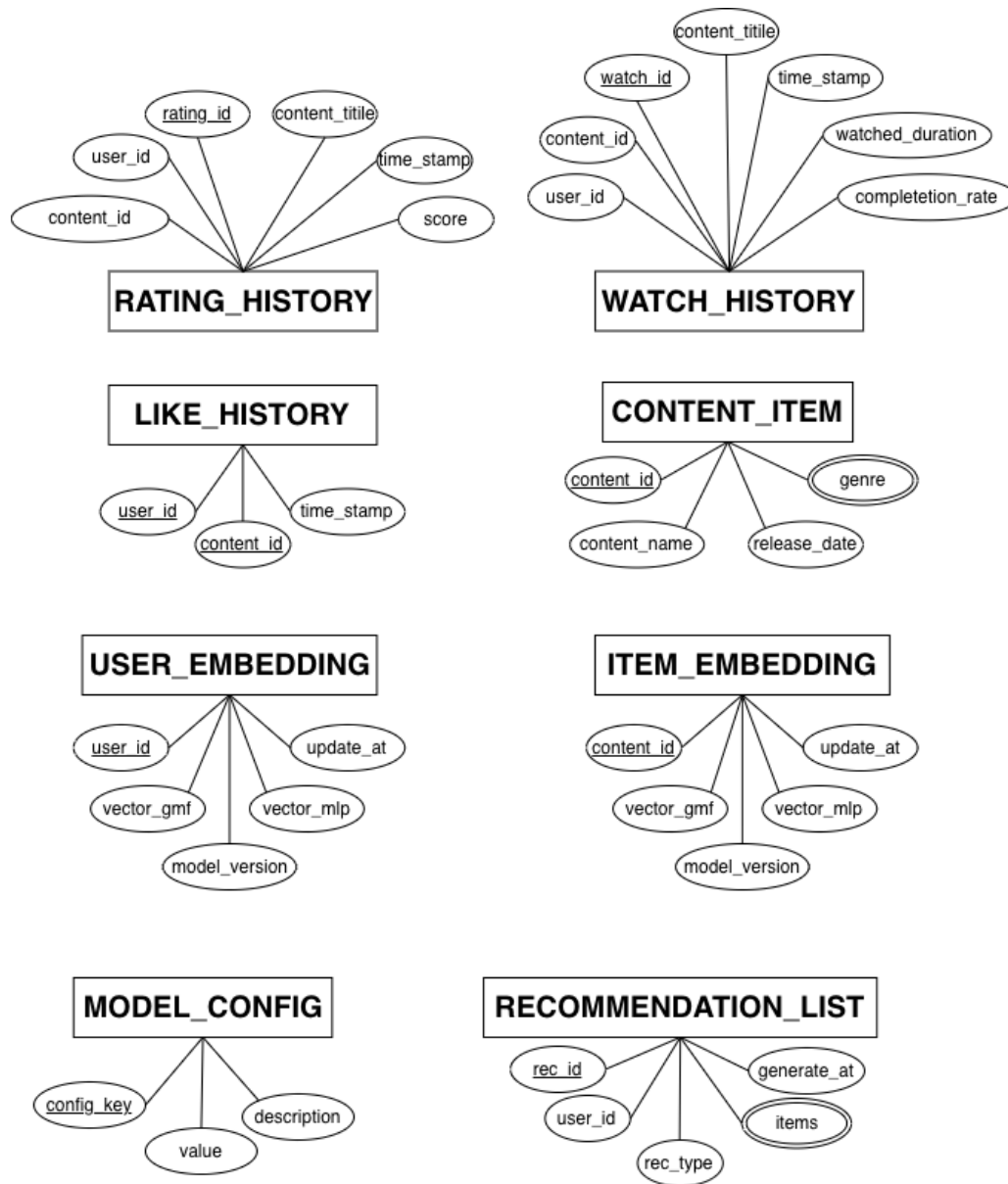
Chức năng nghiệp vụ (Domain Functions):

- **Data Collection (Thu thập):** Tổng hợp dữ liệu tương tác từ nhiều nguồn (xem, đánh giá, thích) và dữ liệu biên lai nội dung để tạo tập huấn luyện (Training Set).
- **Model Management:** Quản lý phiên bản mô hình và lưu trữ các vector đặc trưng (Embeddings) của người dùng và phim sau khi huấn luyện.

- **Recommendation Serving:** Tính toán hoặc truy xuất danh sách phim gợi ý (Top-K) và trả về cho Client hiển thị.
- **Configuration:** Quản lý các tham số động của thuật toán và hệ thống mà không cần can thiệp vào code.

5.8.1 Mô hình dữ liệu

Mô hình dữ liệu được chia thành 3 nhóm phục vụ cho quy trình Machine Learning pipeline.



Hình 29: Sơ đồ EERD cho Recommendation Service

Tên thực thể	Mô tả
HISTORY (Watch, Rating, Like)	Nhóm các thực thể lưu trữ lịch sử tương tác. Đây là nguồn dữ liệu đầu vào (Input Data) để mô hình học được sở thích người dùng (Implicit & Explicit Feedback).
CONTENT_ITEM	Bản sao (Replica) rút gọn của dữ liệu phim từ Content Service. Giúp Recommendation Service có thể lấy thông tin phim nhanh chóng khi huấn luyện mà không cần gọi API sang Service khác.
EMBEDDING (User, Item)	Lưu trữ các vector tiềm ẩn (Latent Vectors) - kết quả của quá trình huấn luyện mô hình NCF.
RECOMMENDATION_LIST	Lưu trữ kết quả gợi ý cuối cùng (Output) đã được tính toán sẵn để phục vụ truy vấn nhanh từ phía người dùng.

Bảng 69: Danh sách thực thể Recommendation Service

5.8.2 Ánh xạ mô hình dữ liệu

Cơ sở dữ liệu của Recommendation Service được xây dựng trên nền tảng **PostgreSQL**. Dữ liệu được chia thành 3 nhóm chức năng chính: Dữ liệu đầu vào (Input), Dữ liệu mô hình (Model) và Dữ liệu kết quả (Output).

RATING_HISTORY

rating_id	user_id	item_id	content_title	time_stamp	score
-----------	---------	---------	---------------	------------	-------

LIKE_HISTORY

payment_id	user_id	time_stamp
------------	---------	------------

WATCH_HISTORY

watch_id	user_id	item_id	content_title	time_stamp	duration	completion_rate
----------	---------	---------	---------------	------------	----------	-----------------

USER_EMBEDDING

user_id	vector_gmf	model_version	vector_mlp	update_at
---------	------------	---------------	------------	-----------

ITEM_EMBEDDING

content_id	vector_gmf	model_version	vector_mlp	update_at
------------	------------	---------------	------------	-----------

MODEL_CONFIG

config_key	value	description
------------	-------	-------------

USER_EMBEDDING

rec_id	user_id	rec_type	generate_at	update_at	items
--------	---------	----------	-------------	-----------	-------

Hình 30: Sơ đồ mapping cho Recommendation Service

5.8.2.1 Nhóm 1: Dữ liệu Huấn luyện (Training Data) Các bảng trong nhóm này lưu trữ dữ liệu lịch sử tương tác của người dùng, đóng vai trò là nguồn nuôi dưỡng (Feed) cho thuật toán AI. Dữ liệu được đồng bộ (Subscribe) từ Rating Service và Content Service thông qua Kafka.

Bảng: watch_history

Lưu trữ hành vi xem phim (Implicit Feedback).



Tên trường	Kiểu dữ liệu	Mô tả
watch_id	UUID	PK. Định danh duy nhất của phiên xem.
user_id	UUID	ID người dùng thực hiện hành vi.
content_id	UUID	ID nội dung được xem.
content_title	VARCHAR	Tên phim (Lưu trữ phi chuẩn hóa để truy vấn nhanh).
watched_duration	INT	Thời gian thực tế người dùng đã xem (giây).
completion_rate	FLOAT	Tỷ lệ hoàn thành (0.0 - 1.0). Chỉ số quan trọng để mô hình hiểu mức độ hứng thú.
time_stamp	TIMESTAMP	Thời điểm ghi nhận sự kiện.

Bảng 70: Chi tiết bảng watch_history

Bảng: rating_history

Lưu trữ hành vi đánh giá (Explicit Feedback).

Tên trường	Kiểu dữ liệu	Mô tả
rating_id	UUID	PK. Định danh lượt đánh giá.
user_id	UUID	ID người dùng đánh giá.
content_id	UUID	ID bộ phim.
stream_asset_id	UUID	ID file video cụ thể (nếu cần phân tích chất lượng nguồn phát).
content_title	VARCHAR	Tên phim.
score	INT	Điểm số đánh giá (1-5 sao).
time_stamp	TIMESTAMP	Thời điểm đánh giá.

Bảng 71: Chi tiết bảng rating_history

Bảng: like_history

Lưu trữ hành vi "Thích" (Binary Feedback).

Tên trường	Kiểu dữ liệu	Mô tả
user_id	UUID	PK, FK. ID người dùng.
content_id	UUID	PK, FK. ID bộ phim được thích.
time_stamp	TIMESTAMP	Thời điểm người dùng nhấn nút thích.

Bảng 72: Chi tiết bảng like_history

Bảng: content_item

Bảng đệm (Cache) chứa thông tin phim cơ bản. Dữ liệu tại đây được phi chuẩn hóa (Denormalized) để phục vụ việc lọc ứng viên (Candidate Generation) nhanh nhất có thể.

Tên trường	Kiểu dữ liệu	Mô tả
content_id	UUID	PK. ID bộ phim.
content_name	VARCHAR	Tên hiển thị của phim.
genres	VARCHAR[]	Array (Mảng). Danh sách các thể loại của phim. Ví dụ: ['Action', 'Sci-Fi'].
release_date	DATE	Ngày phát hành.

Bảng 73: Chi tiết bảng content_item

5.8.2.2 Nhóm 2: Dữ liệu Mô hình (Model Artifacts) Lưu trữ các tham số vector đặc trưng (Embeddings) của người dùng và phim sau khi được huấn luyện bởi mạng nơ-ron NeuMF.

Bảng: user_embedding

Lưu vector đặc trưng hành vi người dùng.

Tên trường	Kiểu dữ liệu	Mô tả
user_id	UUID	PK. Định danh người dùng.
vector_gmf	FLOAT[]	Mảng số thực đại diện cho vector GMF (Generalized Matrix Factorization).
vector_mlp	FLOAT[]	Mảng số thực đại diện cho vector MLP (Multi-Layer Perceptron).
model_version	VARCHAR	Phiên bản của model (Ví dụ: v1.0).
update_at	TIMESTAMP	Thời điểm cập nhật vector.

Bảng 74: Chi tiết bảng user_embedding

Bảng: item_embedding

Lưu vector đặc trưng nội dung phim.

Tên trường	Kiểu dữ liệu	Mô tả
content_id	UUID	PK. Định danh bộ phim.
vector_gmf	FLOAT[]	Mảng số thực vector GMF của phim.
vector_mlp	FLOAT[]	Mảng số thực vector MLP của phim.
model_version	VARCHAR	Phiên bản model.
update_at	TIMESTAMP	Thời điểm cập nhật vector.

Bảng 75: Chi tiết bảng item_embedding

Bảng: model_config

Lưu cấu hình động cho hệ thống gợi ý.

Tên trường	Kiểu dữ liệu	Mô tả
config_key	VARCHAR	PK. Tên tham số cấu hình (VD: <i>top_k_items</i>).
value	VARCHAR	Giá trị cấu hình.
description	TEXT	Mô tả ý nghĩa của tham số.

Bảng 76: Chi tiết bảng model_config

5.8.2.3 Nhóm 3: Dữ liệu Kết quả (Serving Layer) Lưu trữ danh sách phim đã được tính toán điểm phù hợp, sẵn sàng trả về cho Client.

Bảng: recommendation_list

Chứa danh sách Top- K phim gợi ý cho từng người dùng.

Tên trường	Kiểu dữ liệu	Mô tả
rec_id	UUID	PK . Định danh bản ghi gợi ý.
user_id	UUID	FK . ID người dùng nhận gợi ý.
rec_type	VARCHAR	Loại danh sách (VD: <i>HOME</i> , <i>SIMILAR_TO_WATCHED</i>).
items	JSONB	Danh sách chứa ID phim và điểm dự đoán. Cấu trúc ví dụ: [content_id: "uuid", score: 0.95]
generated_at	TIMESTAMP	Thời điểm danh sách được tạo ra.

Bảng 77: Chi tiết bảng recommendation_list

6 Hệ thống gợi ý phim cá nhân hóa

6.1 Mục tiêu

Hệ thống gợi ý phim cá nhân hóa của FlixZone được xây dựng nhằm đề xuất các bộ phim phù hợp nhất cho từng người dùng dựa trên hành vi sử dụng và lịch sử tương tác thực tế trong hệ thống. Mục tiêu chính của hệ thống là:

- Cá nhân hóa trải nghiệm xem phim cho từng người dùng thay vì hiển thị danh sách phim giống nhau cho tất cả.
- Tăng mức độ tương tác (engagement), thời gian xem và khả năng quay lại nền tảng.
- Hỗ trợ người dùng khám phá các nội dung mới phù hợp với sở thích cá nhân.

Về mặt kỹ thuật, bài toán được mô hình hóa như một bài toán dự đoán mức độ quan tâm (preference prediction) giữa người dùng và phim, từ đó xếp hạng và lựa chọn ra các phim có xác suất được người dùng yêu thích cao nhất.

6.2 Dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống gợi ý được thu thập trực tiếp từ cơ sở dữ liệu của FlixZone, phản ánh hành vi thực tế của người dùng trong quá trình sử dụng nền tảng. Các nhóm dữ liệu chính bao gồm:

- **Thông tin người dùng (User):**
 - UserID: định danh duy nhất của người dùng.
 - Thông tin gói dịch vụ (Free / Premium).
 - Ngày đăng ký, trạng thái tài khoản.
- **Thông tin phim (Movie):**
 - MovieID: định danh duy nhất của bộ phim.
 - Thể loại (genres).
 - Quốc gia sản xuất, năm phát hành.
 - Thời lượng phim, độ tuổi phân loại.
- **Dữ liệu tương tác người dùng–phim (User–Movie Interaction):**
 - Lịch sử xem phim (watch history).
 - Thời gian xem, tỷ lệ xem hoàn thành.
 - Hành vi dừng, tua, xem lại.
 - Lượt thích (like).
 - Điểm đánh giá (rating) nếu người dùng cung cấp.

Từ các dữ liệu trên, hệ thống xây dựng ma trận tương tác người dùng–phim, trong đó mỗi phần tử phản ánh mức độ quan tâm tổng hợp của người dùng đối với một bộ phim cụ thể.

6.3 Dữ liệu đầu ra

Đầu ra của hệ thống gợi ý phim cá nhân hóa trong FlixZone bao gồm:

- Điểm ưu tiên (recommendation score) giữa mỗi người dùng và mỗi bộ phim, biểu diễn mức độ phù hợp dự đoán.

- Danh sách Top- K phim được xếp hạng theo mức độ phù hợp giảm dần cho từng người dùng.

Danh sách phim gợi ý này được sử dụng trực tiếp để hiển thị trên các khu vực như “Phim đề xuất cho bạn” trong giao diện người dùng của FlixZone.

6.4 Phương pháp

Hệ thống gợi ý phim cá nhân hóa của FlixZone được hiện thực dựa trên phương pháp **Lọc cộng tác bằng mạng nơ-ron (Neural Collaborative Filtering – NCF)**. Phương pháp này cho phép mô hình hóa trực tiếp mối quan hệ giữa người dùng và phim thông qua dữ liệu tương tác thực tế, đồng thời khắc phục hạn chế của các mô hình tuyến tính truyền thống.

6.4.1 Biểu diễn dữ liệu

Trong hệ thống FlixZone, mỗi người dùng u và mỗi bộ phim i được biểu diễn bởi một vector embedding trong không gian tiềm ẩn:

$$p_u \in \mathbb{R}^d, \quad q_i \in \mathbb{R}^d$$

Các embedding này không được thiết kế thủ công mà được học tự động từ dữ liệu lịch sử tương tác người dùng–phim, bao gồm hành vi xem, mức độ hoàn thành, lượt thích và đánh giá.

Trước khi đưa vào mô hình, dữ liệu tương tác được chuẩn hóa thành các cặp $(u, i, y_{u,i})$, trong đó $y_{u,i}$ là giá trị thể hiện mức độ quan tâm của người dùng u đối với phim i .

6.4.2 Mô hình tương tác người dùng–phim

Để học được cả mối quan hệ tuyến tính và phi tuyến giữa người dùng và phim, FlixZone sử dụng kiến trúc **Neural Matrix Factorization (NeuMF)**, bao gồm hai nhánh song song:

- **Nhánh Generalized Matrix Factorization (GMF):** Mô hình hóa tương tác tuyến tính bằng cách thực hiện phép nhân từng phần tử (Hadamard) giữa embedding người dùng và phim:

$$\phi_{u,i}^{GMF} = p_u \odot q_i$$

- **Nhánh Multi-Layer Perceptron (MLP):** Mô hình hóa các tương tác phi tuyến bậc cao bằng cách nối hai vector embedding:

$$z_1 = [p_u; q_i]$$

Vector này được truyền qua nhiều tầng ẩn phi tuyến để trích xuất biểu diễn tương tác phức tạp giữa người dùng và phim.

Hai biểu diễn trên được kết hợp tại tầng cuối nhằm tận dụng ưu điểm của cả hai cách tiếp cận:

$$\phi_{u,i} = [\phi_{u,i}^{GMF}; \phi_{u,i}^{MLP}]$$

6.4.3 Dự đoán mức độ phù hợp

Từ biểu diễn kết hợp $\phi_{u,i}$, mô hình sinh ra một giá trị $\hat{y}_{u,i}$ biểu diễn mức độ phù hợp dự đoán giữa người dùng u và phim i :

$$\hat{y}_{u,i} = f(\phi_{u,i})$$

Trong đó $f(\cdot)$ là hàm kích hoạt phù hợp với loại dữ liệu đầu vào, có thể là:

- Hàm sigmoid khi mô hình hóa bài toán gợi ý dựa trên hành vi (implicit feedback).
- Hàm tuyến tính khi mô hình hóa bài toán dự đoán điểm đánh giá.

Giá trị $\hat{y}_{u,i}$ được sử dụng làm điểm ưu tiên để xếp hạng các bộ phim cho từng người dùng.

6.4.4 Huấn luyện mô hình

Mô hình được huấn luyện offline trên dữ liệu lịch sử của FlixZone bằng cách tối ưu hàm mất mát giữa giá trị dự đoán $\hat{y}_{u,i}$ và giá trị thực $y_{u,i}$. Tùy theo dạng dữ liệu, hệ thống có thể sử dụng:

- Mean Squared Error (MSE) cho dữ liệu đánh giá.
- Binary Cross-Entropy cho dữ liệu hành vi.

Thuật toán tối ưu Adam được sử dụng để cập nhật các tham số của mô hình. Quá trình huấn luyện được thực hiện định kỳ nhằm đảm bảo mô hình phản ánh kịp thời sự thay đổi trong hành vi người dùng.

6.4.5 Triển khai trong hệ thống FlixZone

Sau khi huấn luyện, mô hình gợi ý được triển khai dưới dạng một **Recommendation Service** độc lập trong kiến trúc microservices của FlixZone. Service này:

- Nhận đầu vào là UserID.
- Truy xuất embedding tương ứng của người dùng.
- Tính toán điểm phù hợp cho tập các phim ứng viên.
- Trả về danh sách Top- K phim được gợi ý cho người dùng.

Cách tiếp cận này cho phép FlixZone mở rộng quy mô hệ thống gợi ý, dễ dàng tích hợp với các dịch vụ khác và hỗ trợ cá nhân hóa nội dung theo thời gian thực.

Nhiệm vụ	Mô tả	Thành viên phụ trách
Phát triển Frontend Web App	Xây dựng giao diện người dùng bằng Next.js, thiết kế UI/UX cho trang chủ, tìm kiếm phim, xem phim, quản lý tài khoản.	Nguyễn Hải Trung
Hỗ trợ phát triển Backend	Hỗ trợ phát triển các API endpoints, tích hợp frontend với backend services.	Nguyễn Hải Trung
Viết tài liệu hệ thống	Biên soạn tài liệu kiến trúc phần mềm, mô tả use case, sơ đồ hành vi.	Lâm Vũ
Thiết kế cơ sở dữ liệu	Thiết kế mô hình dữ liệu, sơ đồ ERD cho các dịch vụ, định nghĩa quan hệ và ràng buộc.	Lâm Vũ
Phát triển Recommendation Service	Xây dựng hệ thống đề xuất phim ứng với mỗi user dựa trên lịch sử xem, thể loại, rating, like phim	Lâm Vũ
Thiết kế kiến trúc hệ thống	Thiết kế kiến trúc Event-Driven Microservices, định nghĩa các góc nhìn kiến trúc.	Võ Quang Đại Việt
Đưa ra quyết định kỹ thuật	Phân tích và lựa chọn công nghệ phù hợp, ghi nhận trong ADR.	Võ Quang Đại Việt
Phát triển Users Service	Xây dựng dịch vụ xác thực và quản lý người dùng, JWT authentication và OAuth2.	Võ Quang Đại Việt
Phát triển Content Service	Xây dựng dịch vụ quản lý nội dung phim và tích hợp Elasticsearch.	Võ Quang Đại Việt
Phát triển Streaming Service	Xây dựng dịch vụ xử lý và truyền phát video, tích hợp Cloudflare Stream.	Võ Quang Đại Việt
Phát triển Rating Service	Xây dựng dịch vụ quản lý đánh giá người dùng và đồng bộ qua message queue.	Võ Quang Đại Việt
Phát triển Notification Service	Xây dựng dịch vụ gửi thông báo và xử lý sự kiện tập mới.	Võ Quang Đại Việt
Cấu hình API Gateway	Cài đặt Apache APISIX để định tuyến, xác thực và rate limiting.	Võ Quang Đại Việt
Cấu hình Message Queue	Thiết lập Redis/Bull Queue và Apache Kafka cho giao tiếp bất đồng bộ.	Võ Quang Đại Việt
Cấu hình Elasticsearch	Tích hợp Elasticsearch cho tìm kiếm toàn văn và fuzzy search.	Võ Quang Đại Việt
Triển khai hệ thống	Đóng gói và triển khai bằng Docker và Docker Compose.	Võ Quang Đại Việt
Hỗ trợ viết tài liệu	Hỗ trợ biên soạn tài liệu kỹ thuật, ADR và activity diagrams.	Võ Quang Đại Việt

Bảng 78: Phân công nhiệm vụ phát triển hệ thống FlixZone

7 Kết Luận

7.1 Những Gì Đã Làm Được

Trong giai đoạn này, nhóm đã hoàn thành việc thiết kế và phát triển hệ thống **FlixZone** – một nền tảng xem phim trực tuyến theo kiến trúc Event-Driven Microservices. Các thành quả chính bao gồm:

7.1.1 Thiết Kế Kiến Trúc

- Xây dựng kiến trúc microservices hoàn chỉnh với 6 dịch vụ độc lập: Users Service, Content Service, Streaming Service, Rating Service, Notification Service và Recommendation Service.
- Thiết kế hệ thống API Gateway với Apache APISIX để quản lý định tuyến, xác thực và bảo mật.
- Triển khai cơ chế giao tiếp bất đồng bộ kết hợp Redis/Bull Queue cho tác vụ nghiệp vụ và Apache Kafka cho streaming events.
- Áp dụng Polyglot Persistence với PostgreSQL cho dữ liệu quan hệ và MongoDB cho dữ liệu linh hoạt.

7.1.2 Phát Triển Backend

- Hoàn thành Users Service với đầy đủ chức năng đăng ký, đăng nhập, xác thực JWT và OAuth2 với Google.
- Phát triển Content Service với khả năng quản lý phim, phim bộ, mùa, tập và tích hợp Elasticsearch cho tìm kiếm toàn văn với fuzzy matching.
- Xây dựng Streaming Service tích hợp Cloudflare Stream cho xử lý và phân phối video với adaptive bitrate streaming.
- Triển khai Rating Service với cơ chế đồng bộ điểm đánh giá qua message queue.
- Phát triển Notification Service với khả năng gửi thông báo đa kênh (in-app, email) và quản lý subscriptions.

7.1.3 Phát Triển Frontend

- Xây dựng giao diện người dùng với Next.js, bao gồm trang chủ, tìm kiếm, chi tiết phim và trình phát video.
- Tích hợp frontend với backend thông qua RESTful API.

7.1.4 Tài Liệu Kỹ Thuật

- Biên soạn tài liệu kiến trúc phần mềm với đầy đủ các góc nhìn: phân rã, thành phần-kết nối và triển khai.
- Thiết kế 09 sơ đồ hoạt động (activity diagrams) cho các nghiệp vụ quan trọng.
- Ghi nhận 8 Hồ Sơ Quyết Định Kiến Trúc (ADR) với phân tích chi tiết các lựa chọn công nghệ.
- Thiết kế mô hình cơ sở dữ liệu với sơ đồ ERD cho từng dịch vụ.

7.1.5 Triển Khai

- Container hóa toàn bộ hệ thống với Docker và Docker Compose.
- Cấu hình môi trường phát triển hoàn chỉnh với tất cả dependencies (PostgreSQL, MongoDB, Redis, Elasticsearch, Kafka).

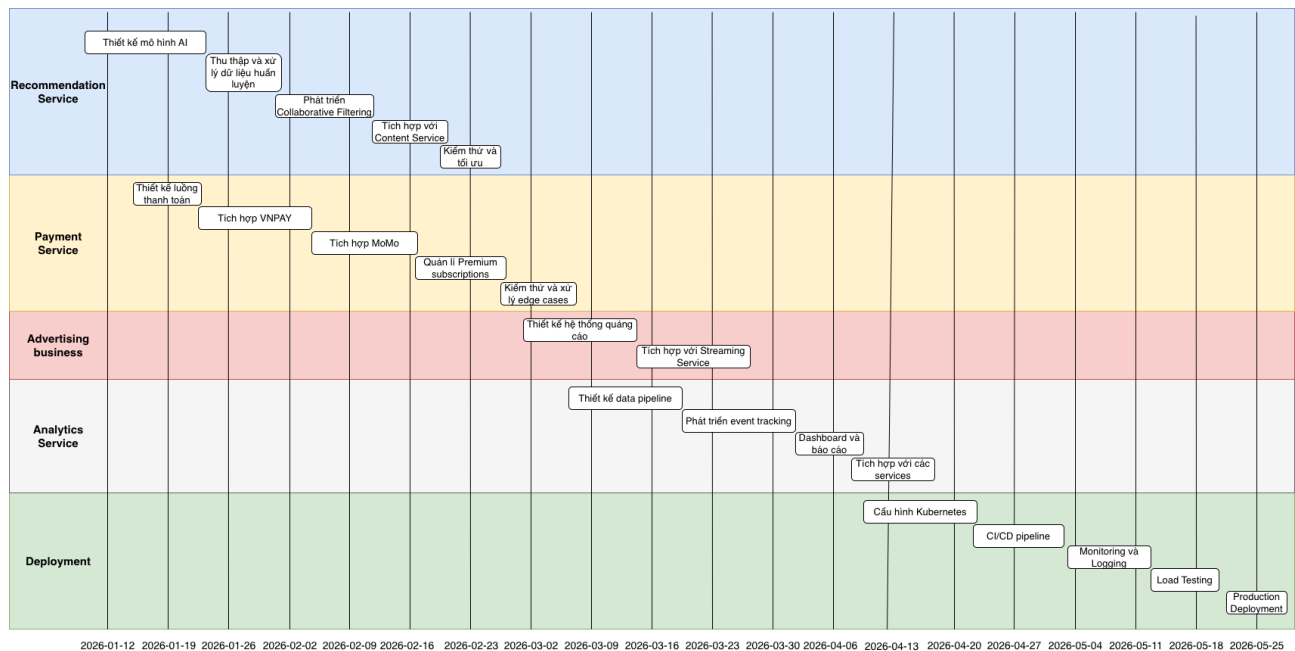
7.2 Nhận Xét

Hệ thống FlixZone đã đạt được mục tiêu đề ra là xây dựng một nền tảng xem phim hoàn chỉnh theo kiến trúc microservices, vượt xa mức độ của một bài tập lập trình cơ bản. Việc áp dụng các pattern và công nghệ hiện đại như Event-Driven Architecture, API Gateway, Message Queue và Polyglot Persistence giúp hệ thống có khả năng mở rộng tốt và phù hợp với quy mô doanh nghiệp thực tế.

8 Hưởng Phát Triển Tiếp Theo

Trong các giai đoạn tiếp theo, nhóm dự kiến sẽ:

- Hoàn thiện hệ thống gợi ý phim cá nhân hóa dựa trên AI với collaborative filtering.
- Tích hợp thanh toán với VNPAY và MoMo cho tài khoản Premium.
- Phát triển module quảng cáo (Ads Service) cho người dùng miễn phí.
- Triển khai Analytics Service để thu thập và phân tích dữ liệu xem phim.
- Mở rộng hệ thống lên Kubernetes cho môi trường production.
- Tối ưu hiệu năng và thực hiện load testing.



Hình 31: Sơ đồ Gantt cho hướng phát triển trong tương lai

Tài liệu

- [1] Grand View Research. *Video Streaming Market Size, Share & Trends Analysis Report By Streaming Type, By Platform, By Revenue Model, By Deployment, By User, By Region, And Segment Forecasts, 2024 - 2030*. 2024. Truy cập tại: <https://www.grandviewresearch.com/industry-analysis/video-streaming-market>
- [2] Statista. *Video Streaming (SVoD) - Worldwide | Statista Market Forecast*. 2024. Truy cập tại: <https://www.statista.com/outlook/dmo/digital-media/video-on-demand/video-streaming-svod/worldwide>
- [3] Cục Phát thanh, Truyền hình và Thông tin điện tử, Bộ Thông tin và Truyền thông. *Báo cáo tổng kết hoạt động dịch vụ phát thanh, truyền hình trên mạng Internet năm 2023*. Hà Nội, 2023.
- [4] Kantar Media. *Vietnam Media Landscape 2023: OTT and Streaming Trends*. Kantar Vietnam, 2023.
- [5] Newman, Sam. *Building Microservices: Designing Fine-Grained Systems*. 2nd Edition. Sebastopol, CA: O'Reilly Media, 2021. ISBN: 978-1492034025.
- [6] Richardson, Chris. *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications, 2018. ISBN: 978-1617294549.
- [7] Fowler, Martin and Lewis, James. "Microservices: A Definition of This New Architectural Term". *martinfowler.com*, March 25, 2014. Truy cập tại: <https://martinfowler.com/articles/microservices.html>
- [8] Mauro, Tony. "Adopting Microservices at Netflix: Lessons for Architectural Design". *NGINX Blog*, February 19, 2015. Truy cập tại: <https://www.nginx.com/blog/microservices-at-netflix-architectural-best-practices/>
- [9] Richardson, Chris. "Pattern: API Gateway / Backends for Frontends". *Microservices.io*, 2024. Truy cập tại: <https://microservices.io/patterns/apigateway.html>
- [10] Pantos, R. and May, W. *RFC 8216: HTTP Live Streaming*. Internet Engineering Task Force (IETF), August 2017. DOI: 10.17487/RFC8216. Truy cập tại: <https://datatracker.ietf.org/doc/html/rfc8216>
- [11] International Organization for Standardization. *ISO/IEC 23009-1:2022 - Information technology — Dynamic adaptive streaming over HTTP (DASH) — Part 1: Media presentation description and segment formats*. Geneva: ISO, 2022.
- [12] Nygren, Erik, Ramesh K. Sitaraman, and Jennifer Sun. "The Akamai Network: A Platform for High-Performance Internet Applications". *ACM SIGOPS Operating Systems Review* 44, no. 3 (2010): 2-19. DOI: 10.1145/1842733.1842736.
- [13] Gomez-Uribe, Carlos A. and Hunt, Neil. "The Netflix Recommender System: Algorithms, Business Value, and Innovation". *ACM Transactions on Management Information Systems* 6, no. 4 (2015): Article 13, 1-19. DOI: 10.1145/2843948.
- [14] Covington, Paul, Jay Adams, and Emre Sargin. "Deep Neural Networks for YouTube Recommendations". In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16)*, 191-198. New York: ACM, 2016. DOI: 10.1145/2959100.2959190.
- [15] Ricci, Francesco, Lior Rokach, and Bracha Shapira (Eds.). *Recommender Systems Handbook*. 2nd Edition. New York: Springer, 2015. ISBN: 978-1489976369.
- [16] VNPAY JSC. *VNPAY Payment Gateway - Tài liệu tích hợp API v2.1.0*. 2024. Truy cập tại: <https://sandbox.vnpayment.vn/apis/docs/thanh-toan-pay/pay.html>
- [17] M_Service JSC. *MoMo Payment Platform - Developer Documentation*. 2024. Truy cập tại: <https://developers.momo.vn/v3/docs/payment/onboarding/introduction/>
- [18] Stripe, Inc. "Idempotent Requests". *Stripe API Documentation*, 2024. Truy cập tại: https://docs.stripe.com/api/idempotent_requests

- [19] Richardson, Chris. “Pattern: Saga”. *Microservices.io*, 2024. Truy cập tại: <https://microservices.io/patterns/data/saga.html>
- [20] Richards, Mark and Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O'Reilly Media, 2020. ISBN: 978-1492043454.
- [21] Martin, Robert C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall, 2008. ISBN: 978-0132350884.
- [22] Evans, Eric. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston: Addison-Wesley Professional, 2003. ISBN: 978-0321125217.

9 Phụ lục

9.1 Hồ sơ quyết định kiến trúc

ADR-01: Lựa Chọn Kiến Trúc Event-Driven Microservices

1. **Vấn đề:** Hệ thống FlixZone cần xử lý lượng lớn yêu cầu phát video trực tuyến, tìm kiếm nội dung, quản lý đánh giá và gửi thông báo đến hàng ngàn người dùng theo dõi phim bộ. Đồng thời, hệ thống phải thu thập và xử lý playback events real-time cho analytics. Kiến trúc cần đảm bảo khả năng mở rộng độc lập cho từng dịch vụ, độ tin cậy cao khi xử lý các sự kiện nghiệp vụ quan trọng, và cho phép các nhóm phát triển triển khai độc lập.

2. **Quyết định:** Áp dụng kiến trúc **Event-Driven Microservices**.

- Hệ thống được chia thành nhiều microservice độc lập (Users Service, Content Service, Streaming Service, Rating Service, Notification Service, Analytics Service), mỗi dịch vụ chịu trách nhiệm một nghiệp vụ riêng và có cơ sở dữ liệu độc lập.
- API Gateway (Apache APISIX) là điểm vào duy nhất, đảm nhiệm xác thực, định tuyến và bảo vệ hệ thống.
- **Redis/Bull Queue** làm hàng đợi cho tác vụ nghiệp vụ: thông báo tập mới, cập nhật đánh giá, gửi email với retry mechanism và delayed jobs.
- **Apache Kafka** làm hàng đợi cho streaming events: playback events, watch progress, user activity với throughput cao và khả năng replay.
- Elasticsearch cung cấp khả năng tìm kiếm toàn văn với độ phù hợp cao cho danh mục phim.

Ưu điểm:

- Xử lý bất đồng bộ hiệu quả, phù hợp với các nghiệp vụ như gửi thông báo hàng loạt khi có tập mới.
- Hỗ trợ streaming events với throughput cao cho real-time analytics thông qua Kafka.
- Các dịch vụ hoàn toàn tách biệt, hỗ trợ triển khai độc lập và mở rộng linh hoạt.
- Khả năng phục hồi cao: dữ liệu sự kiện vẫn được lưu giữ trong hàng đợi khi một dịch vụ gặp sự cố.
- Kafka cho phép replay events để tái xử lý analytics hoặc debug issues.

Nhược điểm:

- Chi phí cơ sở hạ tầng và vận hành cao hơn kiến trúc monolithic.
- Độ phức tạp vận hành cao với hai hệ thống message queue (Redis/Bull và Kafka).
- Một số nghiệp vụ phải chấp nhận mô hình nhất quán cuối cùng (eventual consistency).
- Khó kiểm thử tích hợp và sửa lỗi do các luồng xử lý phân tán.

3. **Trạng thái: ĐÃ PHÊ DUYỆT**

4. **Giả định:**

- (a) Hệ thống có thể triển khai trên hạ tầng container và Docker Compose hoặc Kubernetes.
- (b) Đội ngũ kỹ thuật có khả năng vận hành các công nghệ như Redis, Kafka, Elasticsearch, PostgreSQL và MongoDB.
- (c) Tính mở rộng và độ tin cậy là yêu cầu trọng yếu trong nghiệp vụ streaming video.
- (d) Chi phí vận hành cao hơn được chấp nhận để đổi lấy hiệu năng và khả năng mở rộng.
- (e) Hệ thống cần xử lý hàng ngàn playback events mỗi giây cho real-time analytics.

5. **Các lựa chọn thay thế:**

- (a) **Monolithic Architecture:** Toàn bộ hệ thống được xây dựng trong một ứng dụng duy nhất.

• **Ưu điểm:**

- Đơn giản triển khai và debug.
- Phù hợp giai đoạn MVP với team nhỏ.

- **Nhược điểm:**

- Khó mở rộng khi lưu lượng tăng; thay đổi một module ảnh hưởng toàn hệ thống.
- Không phù hợp cho xử lý streaming events với throughput cao.
- Không thể scale độc lập từng tính năng.

(b) **Service-Based Architecture (SBA):** Hệ thống được chia thành các dịch vụ tương đối lớn, giao tiếp qua REST hoặc RPC.

- **Ưu điểm:**

- Phân chia trách nhiệm rõ hơn Monolithic Architecture.
- Vận hành đơn giản hơn microservices.

- **Nhược điểm:**

- Dịch vụ vẫn phụ thuộc đồng bộ; dễ xảy ra lỗi dây chuyền.
- Không tối ưu cho xử lý sự kiện hoặc streaming analytics.

6. **Lập luận:**

- **Hỗ trợ xử lý bất đồng bộ:** Kiến trúc Event-Driven cho phép gửi thông báo đến hàng ngàn người dùng khi có tập mới mà không chặn luồng xử lý chính của Content Service.
- **Streaming analytics real-time:** Kafka cung cấp throughput cao để xử lý hàng ngàn playback events mỗi giây, cho phép Analytics Service thu thập dữ liệu xem phim real-time.
- **Giảm phụ thuộc và triển khai độc lập:** Các microservice giao tiếp thông qua sự kiện nên không phụ thuộc trực tiếp vào nhau. Việc cập nhật Rating Service không ảnh hưởng đến Streaming Service.
- **Mở rộng linh hoạt theo tải thực tế:** Khi lưu lượng playback events tăng, chỉ cần tăng Kafka consumers. Khi thông báo tăng, chỉ cần scale Notification Service.
- **Đảm bảo khả năng phục hồi:** Bull Queue lưu jobs để retry khi Notification Service gián đoạn. Kafka lưu events để replay khi Analytics Service cần tái xử lý.

7. **Các quyết định liên quan:**

- Hệ quả: ADR-02 - Lựa Chọn Sử Dụng API Gateway.
- Hệ quả: ADR-03 - Lựa Chọn Cloudflare Stream cho Video Streaming.
- Hệ quả: ADR-04 - Lựa Chọn Elasticsearch cho Tìm Kiếm Nội Dung.
- Hệ quả: ADR-05 - Lựa Chọn Message Queue Kết Hợp (Redis/Bull + Kafka).

ADR-02: Lựa Chọn Sử Dụng API Gateway

1. **Vấn đề:** Hệ thống FlixZone cần một điểm kiểm soát tập trung để bảo vệ các microservice trước các rủi ro như truy cập trái phép, tấn công từ chối dịch vụ. Do hệ thống xử lý thông tin nhạy cảm (thông tin người dùng, lịch sử xem, đánh giá cá nhân), cần có cơ chế xác thực và phân quyền thống nhất trước khi yêu cầu đến các microservice nội bộ.

2. **Quyết định:** Áp dụng **API Gateway (Apache APISIX)** làm điểm truy cập duy nhất cho toàn bộ client. API Gateway thực hiện:

- Xác thực JWT và kiểm tra quyền trước khi chuyển tiếp yêu cầu đến microservice.
- Định tuyến yêu cầu đến các microservice tương ứng dựa trên đường dẫn URL.
- Triển khai rate limiting, cân bằng tải và path rewriting.
- Cấu hình dạng YAML standalone, dễ quản lý và version control.

Ưu điểm:

- Cung cấp điểm kiểm soát tập trung cho xác thực, ủy quyền và bảo vệ dữ liệu nhạy cảm trước khi yêu cầu đi vào microservice.
- Giảm độ phức tạp cho microservice bằng cách tách các mối quan tâm chung như bảo mật, rate limiting và cân bằng tải.
- Hiệu năng cao với khả năng xử lý hàng chục ngàn request/giây.
- Hỗ trợ plugin phong phú và dễ mở rộng.

Nhược điểm:

- Có thể trở thành điểm nghẽn hoặc điểm lỗi trung tâm nếu không được mở rộng và giám sát phù hợp.
- Gia tăng độ trễ do mỗi yêu cầu phải đi qua thêm một lớp trung gian.

3. **Trạng thái: ĐÃ PHÊ DUYỆT**

4. **Giải thích:**

- (a) Hệ thống cần duy trì khả năng mở rộng khi số lượng người dùng tăng cao.
- (b) Kiến trúc hỗ trợ triển khai container với Docker.
- (c) Microservices không được thiết kế để giao tiếp trực tiếp với ứng dụng khách.

5. **Các lựa chọn thay thế:**

- (a) **Kong Gateway:** API Gateway phổ biến với hệ sinh thái plugin phong phú.

• **Ưu điểm:**

- Cộng đồng lớn, documentation tốt.
- Hỗ trợ nhiều plugin có sẵn.

• **Nhược điểm:**

- Cấu hình phức tạp hơn, yêu cầu database riêng (PostgreSQL/Cassandra).
- Hiệu năng thấp hơn APISIX trong một số benchmark.

- (b) **Nginx với Lua scripting:** Web server hiệu năng cao với khả năng mở rộng qua Lua.

• **Ưu điểm:**

- Hiệu năng rất cao, linh hoạt cấu hình.
- Không phụ thuộc công cụ bên ngoài.

• **Nhược điểm:**

- Không có sẵn các tính năng API Gateway, cần tự phát triển nhiều.
- Khó bảo trì khi logic phức tạp.

6. **Lập luận:**

- **Điểm kiểm soát tập trung:** API Gateway cung cấp điểm kiểm soát cho toàn bộ lưu lượng từ

Web và Mobile App, đảm bảo xác thực và bảo vệ dữ liệu nhạy cảm.

- **Cấu hình đơn giản:** Apache APISIX hỗ trợ chế độ standalone với cấu hình YAML, không cần database riêng, dễ triển khai và version control.
- **Hiệu năng cao:** APISIX được xây dựng trên Nginx + Lua với khả năng xử lý tốt hơn Kong trong nhiều benchmark.
- **Phù hợp với Event-Driven Microservices:** Tạo ra một điểm điều phối rõ ràng, giảm phức tạp cho từng microservice.

7. Các quyết định liên quan:

- Tiền đề: ADR-01 - Lựa Chọn Kiến Trúc Event-Driven Microservices.

ADR-03: Lựa Chọn Cloudflare Stream cho Video Streaming

- Vấn đề:** Hệ thống FlixZone cần phân phối video chất lượng cao đến người dùng với độ trễ thấp, hỗ trợ adaptive bitrate streaming, và có khả năng mở rộng toàn cầu. Việc tự xây dựng hạ tầng video streaming đòi hỏi chi phí lớn về phần cứng, băng thông và nhân sự vận hành.
- Quyết định:** Sử dụng **Cloudflare Stream** làm nền tảng video streaming.
 - Video được tải lên Cloudflare Stream thông qua API và tự động xử lý (transcoding, tạo nhiều độ phân giải).
 - Sử dụng signed URLs với thời hạn để bảo vệ nội dung khỏi truy cập trái phép.
 - Cloudflare CDN phân phối video đến người dùng từ edge server gần nhất.
 - Webhook thông báo trạng thái xử lý video về hệ thống backend.
- Ưu điểm:**
 - Giảm đáng kể chi phí hạ tầng và vận hành so với tự xây dựng.
 - Hỗ trợ adaptive bitrate streaming tự động điều chỉnh chất lượng theo tốc độ mạng.
 - CDN toàn cầu đảm bảo độ trễ thấp cho người dùng ở mọi khu vực.
 - Tự động transcoding sang nhiều độ phân giải (480p, 720p, 1080p, 4K).
- Nhược điểm:**
 - Phụ thuộc vào nhà cung cấp bên thứ ba.
 - Chi phí tăng theo lưu lượng video phát.
- Trạng thái: ĐÃ PHÊ DUYỆT**
- Giả định:**
 - Chi phí sử dụng dịch vụ bên thứ ba được chấp nhận để đổi lấy độ tin cậy và khả năng mở rộng.
 - Hệ thống cần hỗ trợ playback trên nhiều thiết bị và trình duyệt.
 - Nội dung video cần được bảo vệ khỏi truy cập trái phép.
- Các lựa chọn thay thế:**
 - AWS CloudFront + MediaConvert:** Giải pháp video streaming từ Amazon Web Services.
 - Ưu điểm:**
 - Tích hợp tốt với hệ sinh thái AWS.
 - Linh hoạt cấu hình và tùy chỉnh.
 - Nhược điểm:**
 - Cấu hình phức tạp hơn, cần quản lý nhiều dịch vụ riêng biệt (S3, MediaConvert, CloudFront).
 - Chi phí có thể cao hơn nếu không tối ưu cấu hình.
 - Tự xây dựng với nginx-rtmp:** Server streaming tự quản lý.
 - Ưu điểm:**
 - Toàn quyền kiểm soát, không phụ thuộc bên thứ ba.
 - Chi phí cố định không phụ thuộc lưu lượng.
 - Nhược điểm:**
 - Chi phí hạ tầng và vận hành rất cao.
 - Cần đội ngũ chuyên môn về video streaming.
 - Khó đảm bảo độ trễ thấp cho người dùng toàn cầu.
- Lập luận:**
 - Giải pháp end-to-end:** Cloudflare Stream cung cấp từ upload, transcoding đến delivery trong một nền tảng duy nhất, giảm độ phức tạp tích hợp.
 - Bảo vệ nội dung:** Signed URLs đảm bảo chỉ người dùng được ủy quyền mới có thể xem video, phù hợp với mô hình bảo vệ nội dung streaming.

- **Webhook tích hợp:** Webhook giúp hệ thống backend cập nhật trạng thái video real-time, phù hợp với kiến trúc event-driven.
 - **Chi phí hợp lý:** So với tự xây dựng, chi phí vận hành thấp hơn đáng kể và có thể dự đoán được.
7. Các quyết định liên quan:
- Tiền đề: ADR-01 - Lựa Chọn Kiến Trúc Event-Driven Microservices.

ADR-04: Lựa Chọn Elasticsearch cho Tìm Kiếm Nội Dung

- Vấn đề:** Hệ thống FlixZone cần cung cấp khả năng tìm kiếm phim và phim bộ với độ phù hợp cao, hỗ trợ tìm kiếm mờ (fuzzy search) khi người dùng gõ sai chính tả, và có thể lọc theo nhiều tiêu chí (thể loại, năm, đánh giá). Việc sử dụng truy vấn SQL truyền thống với LIKE không đáp ứng được yêu cầu về hiệu năng và độ chính xác.
 - Quyết định:** Sử dụng **Elasticsearch** làm công cụ tìm kiếm chính.
 - Toàn bộ nội dung (phim lẻ, phim bộ) được đánh chỉ mục vào Elasticsearch.
 - Tìm kiếm sử dụng multi-match query với fuzzy matching.
 - Kết quả được xếp hạng theo độ phù hợp (relevance score).
 - PostgreSQL được sử dụng làm fallback khi Elasticsearch không khả dụng.**Ưu điểm:**
 - Tìm kiếm toàn văn với độ phù hợp cao, hỗ trợ fuzzy search.
 - Hiệu năng tìm kiếm nhanh ngay cả với danh mục lớn.
 - Hỗ trợ aggregation để lọc và phân loại kết quả.
 - Khả năng mở rộng ngang khi lượng dữ liệu tăng.**Nhược điểm:**
 - Cần đồng bộ dữ liệu giữa PostgreSQL và Elasticsearch.
 - Tăng độ phức tạp hạ tầng và vận hành.
 - Chấp nhận độ trễ ngắn trong đồng bộ dữ liệu (eventual consistency).
 - Trạng thái: ĐÃ PHÊ DUYỆT**
 - Giả định:**
 - Danh mục nội dung sẽ tăng lên hàng ngàn phim và phim bộ.
 - Người dùng thường tìm kiếm bằng từ khóa và lọc theo thể loại, năm.
 - Chấp nhận độ trễ ngắn trong đồng bộ dữ liệu giữa PostgreSQL và Elasticsearch.
 - Các lựa chọn thay thế:**
 - PostgreSQL Full-Text Search:** Tính năng tìm kiếm toàn văn tích hợp sẵn trong PostgreSQL.
 - Ưu điểm:**
 - Không cần hạ tầng bổ sung, tích hợp sẵn.
 - Dễ triển khai và bảo trì.
 - Nhược điểm:**
 - Hiệu năng thấp hơn với dữ liệu lớn.
 - Không hỗ trợ fuzzy search tốt như Elasticsearch.
 - Algolia:** Dịch vụ tìm kiếm SaaS với API đơn giản.
 - Ưu điểm:**
 - Dễ tích hợp, hiệu năng cao.
 - Hỗ trợ typo tolerance tốt.
 - Nhược điểm:**
 - Chi phí cao theo lượng truy vấn.
 - Phụ thuộc bên thứ ba, không thể tự host.
- Lập luận:**
 - Fuzzy matching mạnh mẽ:** Elasticsearch cung cấp khả năng tìm kiếm mờ, phù hợp khi người dùng gõ sai chính tả như "avangers" thay vì "avengers".
 - Nested query cho thể loại:** Hỗ trợ nested query cho việc lọc theo thể loại (genres.name), phù hợp với cấu trúc dữ liệu phim.

- **Tự host và kiểm soát:** Có thể tự host và kiểm soát hoàn toàn, giảm chi phí so với giải pháp SaaS như Algolia.
- **Fallback mechanism:** PostgreSQL làm fallback đảm bảo hệ thống vẫn hoạt động khi Elasticsearch gặp sự cố.

7. Các quyết định liên quan:

- Tiền đề: ADR-01 - Lựa Chọn Kiến Trúc Event-Driven Microservices.

ADR-05: Lựa Chọn Message Queue Kết Hợp

1. **Vấn đề:** Hệ thống FlixZone có hai loại yêu cầu xử lý bất đồng bộ khác nhau: (1) Các tác vụ nghiệp vụ như gửi thông báo, cập nhật đánh giá cần xử lý tin cậy với retry mechanism; (2) Các sự kiện streaming như playback events, watch progress cần throughput cao và khả năng xử lý real-time. Việc sử dụng một công nghệ duy nhất không đáp ứng tối ưu cả hai yêu cầu.
2. **Quyết định:** Sử dụng kết hợp **Redis/Bull Queue** và **Apache Kafka** cho các use case khác nhau.

- **Redis/Bull Queue** cho tác vụ nghiệp vụ:

- Gửi thông báo tập mới (new-episode queue)
- Cập nhật đánh giá trung bình (rating-updated queue)
- Gửi email xác thực, reset password
- Xử lý webhook từ Cloudflare Stream

- **Apache Kafka** cho streaming events:

- Video playback events (play, pause, seek, complete)
- Watch progress tracking
- Real-time analytics data
- User activity logs

Ưu điểm:

- Tối ưu từng công cụ cho use case phù hợp.
- Redis/Bull cung cấp retry mechanism, delayed jobs, và job prioritization cho tác vụ nghiệp vụ.
- Kafka cung cấp throughput cực cao và khả năng replay events cho streaming analytics.
- Tách biệt luồng xử lý giúp dễ scale và debug.

Nhược điểm:

- Tăng độ phức tạp vận hành với hai hệ thống message queue.
- Đội ngũ cần có kiến thức về cả Redis/Bull và Kafka.
- Chi phí hạ tầng cao hơn.

3. **Trạng thái: ĐÃ PHÊ DUYỆT**

4. **Giải định:**

- (a) Hệ thống cần xử lý hàng ngàn playback events mỗi giây khi lượng người dùng tăng.
- (b) Đội ngũ có khả năng vận hành cả Redis và Kafka.
- (c) Chi phí vận hành cao hơn được chấp nhận để đổi lấy hiệu năng và tính phù hợp.

5. **Các lựa chọn thay thế:**

- (a) **Chỉ sử dụng Redis/Bull:** Dùng Redis cho tất cả message queue.

- **Ưu điểm:**

- Đơn giản, dễ vận hành.
- Đủ cho hầu hết tác vụ nghiệp vụ.

- **Nhược điểm:**

- Không tối ưu cho streaming events với throughput cao.
- Không hỗ trợ replay events như Kafka.

- (b) **Chỉ sử dụng Kafka:** Dùng Kafka cho tất cả message queue.

- **Ưu điểm:**

- Thống nhất một công nghệ.
- Throughput cao cho mọi use case.

- **Nhược điểm:**

- Overkill cho tác vụ đơn giản như gửi email.

- Không có sẵn retry mechanism và delayed jobs như Bull.
- Cấu hình phức tạp hơn cho job queue đơn giản.

(c) **RabbitMQ:** Message broker truyền thống.

- **Ưu điểm:**

- Linh hoạt với nhiều exchange types.
- Hỗ trợ nhiều protocols.

- **Nhược điểm:**

- Không tối ưu cho cả hai use case như giải pháp kết hợp.
- Throughput thấp hơn Kafka cho streaming.

6. **Lập luận:**

- **Right tool for the right job:** Redis/Bull được thiết kế cho job queue với retry, delay, và priority - hoàn hảo cho tác vụ nghiệp vụ. Kafka được thiết kế cho event streaming với throughput cao - hoàn hảo cho playback events.
- **Tách biệt concern:** Luồng notification và rating update hoàn toàn độc lập với luồng streaming analytics, giúp dễ debug và scale riêng biệt.
- **Khả năng replay:** Kafka cho phép replay events để tái xử lý analytics hoặc debug issues, điều mà Redis không hỗ trợ tốt.
- **Tối ưu chi phí:** Redis/Bull nhẹ hơn cho tác vụ đơn giản, không cần dùng Kafka cho việc gửi email hay notification.

7. **Các quyết định liên quan:**

- Tiền đề: ADR-01 - Lựa Chọn Kiến Trúc Event-Driven Microservices.

ADR-06: Lựa Chọn Polyglot Persistence (PostgreSQL + MongoDB)

1. **Vấn đề:** Hệ thống FlixZone có nhiều loại dữ liệu với đặc tính khác nhau: (1) Dữ liệu có cấu trúc chặt chẽ như người dùng, phim, thể loại cần tính toán vẹn và quan hệ phức tạp; (2) Dữ liệu linh hoạt như thông báo, đánh giá, lịch sử xem có schema thay đổi thường xuyên. Việc sử dụng một loại database duy nhất không đáp ứng tối ưu cả hai yêu cầu.

2. **Quyết định:** Áp dụng **Polyglot Persistence** với **PostgreSQL** và **MongoDB**.

- **PostgreSQL** cho dữ liệu có cấu trúc và quan hệ:
 - Users Service: Thông tin người dùng, credentials, profiles.
 - Content Service: Phim, phim bộ, mùa, tập, thể loại, quan hệ nhiều-nhiều.
 - Watchlist: Danh sách yêu thích với foreign key đến users và content.
- **MongoDB** cho dữ liệu linh hoạt và document-based:
 - Notification Service: Thông báo, preferences, subscriptions.
 - Rating Service: Đánh giá người dùng với aggregation.
 - Streaming Service: Video metadata, watch history.

Ưu điểm:

- Tối ưu từng database cho use case phù hợp.
- PostgreSQL đảm bảo ACID cho dữ liệu quan trọng như người dùng và nội dung.
- MongoDB linh hoạt schema cho dữ liệu thay đổi thường xuyên như notifications.
- Mỗi microservice có database riêng, đảm bảo tính độc lập (Database per Service pattern).

Nhược điểm:

- Tăng độ phức tạp vận hành với nhiều loại database.
- Đội ngũ cần có kiến thức về cả SQL và NoSQL.
- Khó thực hiện transactions cross-database.

3. **Trạng thái:** **ĐÃ PHÊ DUYỆT**

4. **Giả định:**

- Mỗi microservice chỉ truy cập database của mình (Database per Service).
- Không cần transactions cross-service; sử dụng eventual consistency qua events.
- Đội ngũ có kinh nghiệm với cả PostgreSQL và MongoDB.

5. **Các lựa chọn thay thế:**

(a) **Chỉ sử dụng PostgreSQL:** Một database relational cho toàn hệ thống.

• Ưu điểm:

- Đơn giản vận hành, một công nghệ duy nhất.
- ACID transactions cho mọi dữ liệu.

• Nhược điểm:

- Schema cứng nhắc, khó thay đổi cho notifications và watch history.
- Không tối ưu cho document-based queries.

(b) **Chỉ sử dụng MongoDB:** Một database NoSQL cho toàn hệ thống.

• Ưu điểm:

- Schema linh hoạt, dễ thay đổi.
- Horizontal scaling tốt.

• Nhược điểm:

- Không hỗ trợ tốt quan hệ phức tạp như content-genres many-to-many.
- Thiếu ACID transactions cho dữ liệu quan trọng.

6. Lập luận:

- **Right database for right data:** Content data có quan hệ phức tạp (phim $\rightarrow$ mùa $\rightarrow$ tập, phim $\Leftrightarrow$ thể loại) phù hợp với PostgreSQL. Notifications có schema linh hoạt phù hợp với MongoDB.
- **Database per Service:** Mỗi microservice có database riêng giúp triển khai và scale độc lập, phù hợp với kiến trúc microservices.
- **Performance tối ưu:** PostgreSQL với indexing và joins hiệu quả cho content queries. MongoDB với aggregation pipeline cho notification statistics.
- **Eventual consistency chấp nhận được:** Các service giao tiếp qua events, không cần cross-database transactions.

7. Các quyết định liên quan:

- Tiền đề: ADR-01 — Lựa Chọn Kiến Trúc Event-Driven Microservices.

ADR-07: Lựa Chọn JWT cho Authentication

- Vấn đề:** Hệ thống FlixZone cần cơ chế xác thực người dùng stateless phù hợp với kiến trúc microservices, hỗ trợ đa nền tảng (Web, Mobile), và cho phép các service xác thực request mà không cần gọi về central auth server cho mỗi request.
 - Quyết định:** Sử dụng **JSON Web Token (JWT)** với cơ chế **Access Token + Refresh Token**.
 - Access Token:** JWT ngắn hạn (15-30 phút) chứa thông tin user (id, email, role).
 - Refresh Token:** Token dài hạn (7 ngày) để lấy access token mới.
 - Tokens được lưu trong HTTP-only cookies để bảo vệ khỏi XSS.
 - API Gateway xác thực JWT trước khi forward request đến microservices.
 - Hỗ trợ OAuth2 với Google cho social login.**Ưu điểm:**
 - Stateless: Không cần lưu session trên server, dễ scale horizontally.
 - Self-contained: Token chứa đủ thông tin user, microservices không cần gọi Users Service.
 - Cross-platform: Hoạt động tốt trên Web, Mobile, và API clients.
 - Standardized: JWT là chuẩn mở (RFC 7519) được hỗ trợ rộng rãi.**Nhược điểm:**
 - Không thể revoke token ngay lập tức (phải chờ hết hạn hoặc dùng blacklist).
 - Token size lớn hơn session ID truyền thống.
 - Cần bảo vệ secret key cẩn thận.
 - Trạng thái: ĐÃ PHÊ DUYỆT**
 - Giả định:**
 - Hệ thống cần scale horizontally với nhiều instances.
 - Chấp nhận việc token không thể revoke ngay lập tức (mitigate bằng access token ngắn hạn).
 - Đội ngũ có kiến thức về JWT và bảo mật token.
 - Các lựa chọn thay thế:**
 - Session-based Authentication:** Lưu session trên server với session ID trong cookie.
 - Ưu điểm:**
 - Dễ revoke session ngay lập tức.
 - Session ID nhỏ gọn.
 - Nhược điểm:**
 - Stateful: Cần shared session store (Redis) cho multiple instances.
 - Microservices phải gọi session store cho mỗi request.
 - Không phù hợp với kiến trúc microservices stateless.
 - OAuth2 với Keycloak/Auth0:** Sử dụng Identity Provider bên ngoài.
 - Ưu điểm:**
 - Feature đầy đủ: SSO, MFA, user management.
 - Tách biệt authentication khỏi application.
 - Nhược điểm:**
 - Thêm dependency và complexity.
 - Chi phí nếu dùng managed service.
 - Overkill cho requirements hiện tại.
- Lập luận:**
 - Phù hợp microservices:** JWT stateless cho phép mỗi service xác thực độc lập mà không cần central session store.
 - API Gateway validation:** APISIX xác thực JWT một lần, các microservices tin tưởng request

đã được authenticate.

- **Refresh token pattern:** Access token ngắn hạn giảm thiểu rủi ro nếu token bị lộ. Refresh token cho phép user không cần đăng nhập lại thường xuyên.
- **HTTP-only cookies:** Bảo vệ tokens khỏi XSS attacks, an toàn hơn localStorage.
- **OAuth2 integration:** Dễ dàng tích hợp Google login với cùng JWT flow.

7. Các quyết định liên quan:

- Tiền đề: ADR-01 — Lựa Chọn Kiến Trúc Event-Driven Microservices.
- Hỗ trợ: ADR-02 — Lựa Chọn Sử Dụng API Gateway.

ADR-08: Lựa Chọn Containerization với Docker

1. **Vấn đề:** Hệ thống FlixZone bao gồm nhiều microservices với các dependencies khác nhau (Node.js, PostgreSQL, MongoDB, Redis, Elasticsearch, Kafka). Cần một giải pháp đóng gói và triển khai đảm bảo tính nhất quán giữa môi trường development, testing và production, đồng thời hỗ trợ scale từng service độc lập.

2. **Quyết định:** Sử dụng **Docker** và **Docker Compose** cho containerization.

- Mỗi microservice được đóng gói trong Docker container riêng với Dockerfile.
- Docker Compose orchestrate toàn bộ hệ thống trong môi trường development và staging.
- Multi-stage builds để tối ưu image size cho production.
- Kubernetes-ready: Có thể migrate sang K8s khi cần scale lớn hơn.

Ưu điểm:

- Đảm bảo consistency giữa các môi trường (dev, staging, prod).
- Isolation: Mỗi service chạy trong container riêng, không conflict dependencies.
- Dễ dàng scale từng service độc lập.
- Reproducible builds: Dockerfile định nghĩa chính xác environment.
- Docker Compose đơn giản hóa việc khởi chạy toàn bộ stack.

Nhược điểm:

- Overhead tài nguyên so với chạy trực tiếp trên host.
- Đội ngũ cần có kiến thức về Docker và container networking.
- Debug có thể phức tạp hơn với multiple containers.

3. **Trạng thái:** **ĐÃ PHÊ DUYỆT**

4. **Giả định:**

- (a) Đội ngũ có kiến thức cơ bản về Docker và containerization.
- (b) Môi trường production hỗ trợ Docker hoặc Kubernetes.
- (c) Quy mô hiện tại phù hợp với Docker Compose, sẽ migrate sang K8s khi cần.

5. **Các lựa chọn thay thế:**

(a) **Virtual Machines (VMs):** Mỗi service chạy trên VM riêng.

• Ưu điểm:

- Isolation hoàn toàn ở mức OS.
- Quen thuộc với nhiều đội ngũ ops.

• Nhược điểm:

- Overhead tài nguyên rất lớn (mỗi VM cần full OS).
- Khởi động chậm, không phù hợp cho microservices.
- Khó scale nhanh.

(b) **Bare Metal / Direct Deploy:** Chạy services trực tiếp trên server.

• Ưu điểm:

- Không có container overhead.
- Đơn giản cho ứng dụng nhỏ.

- **Nhược điểm:**

- Không đảm bảo consistency giữa environments.
- Dependency conflicts giữa các services.
- Khó reproduce issues và debug.

6. **Lập luận:**

- **Microservices-ready:** Docker là standard cho microservices deployment, mỗi service isolated trong container riêng.
- **Developer experience:** ‘docker-compose up’ khởi chạy toàn bộ stack (services + databases + message queues) trong vài phút.
- **CI/CD friendly:** Docker images dễ dàng build, test và deploy trong pipeline.
- **Scalability path:** Docker Compose cho dev/staging, có thể migrate sang Kubernetes cho production scale lớn.

7. **Các quyết định liên quan:**

- Tiền đề: ADR-01 — Lựa Chọn Kiến Trúc Event-Driven Microservices.